

GUIABURROS - Manual Técnico de Seguridad Informática

especializado en Debian 13 Trixie - versión 2026

Nivel 0: Plan de Seguridad Global

Version: 1.0 **Fecha:** Febrero 2026 **Sistema Operativo:** Debian 13 "Trixie" **Autor:** Óscar S. con IAs: MiniMax Agent, Perplexity AI, Gemini AI y Deepseek AI.

Tabla de Contenidos

- 1. [Introduccion y Objetivos del Manual]
- 2. [Alcance]
- 3. [Politica de Seguridad General]
- 4. [Roles y Responsabilidades (Matriz RACI)]
- 5. [Clasificacion de Activos e Informacion]
- 6. [Gestion de Riesgos y Presupuesto]
- 7. [Procedimiento Basico de Respuesta a Incidentes]
- 8. Checklist Nivel 0

1.1. Introduccion y Objetivos del Manual

Que es la Seguridad Informatica

La **seguridad informatica** es el conjunto de medidas, practicas y herramientas disenadas para proteger los sistemas de computacion, las redes y los datos contra accesos no autorizados, ataques maliciosos, danos o robos. Su objetivo principal es garantizar tres principios fundamentales conocidos como la **triada CIA**:

Principio	Descripcion	Ejemplo en Debian
Confidencialidad	Solo las personas autorizadas pueden acceder a la informacion	Permisos de archivos con chmod 600 para datos sensibles
Integridad	Los datos no pueden ser modificados sin autorizacion	Uso de sumas de verificacion (checksums) en paquetes APT
Disponibilidad	Los sistemas y datos estan accesibles cuando se necesitan	Configuracion de servicios redundantes y copias de seguridad

En este manual, el término Nivel 0-5 se usa como niveles de madurez/capas de implantación (modelo propio), no como niveles de clasificación de información ni niveles TCSEC/ISO.

Conceptos Fundamentales

Antes de continuar, es importante comprender algunos terminos clave:

- **Sistema Operativo (SO):** Es el software principal que gestiona el hardware de una computadora y proporciona servicios a otros programas. Debian 13 "Trixie" es un sistema operativo basado en Linux, conocido por su estabilidad y seguridad.
- **Linux:** Es el nucleo (kernel) sobre el cual se construyen sistemas operativos como Debian. Es de codigo abierto, lo que significa que cualquiera puede revisar, modificar y mejorar su codigo fuente.
- **Vulnerabilidad:** Es una debilidad en un sistema que puede ser explotada por un atacante para comprometer la seguridad.
- **Amenaza:** Es cualquier circunstancia o evento con el potencial de causar dano a un sistema informatico.
- **Ataque:** Es la accion deliberada de explotar una vulnerabilidad para comprometer un sistema.

Objetivos del Manual

Este manual tiene como proposito:

1. **Establecer una base solida de conocimientos** en seguridad informatica para administradores de sistemas Debian, sin importar su nivel de experiencia previo.
2. **Proporcionar un marco de trabajo estructurado** para implementar medidas de seguridad de forma progresiva y ordenada.
3. **Definir politicas y procedimientos claros** que puedan ser adoptados en entornos domesticos, educativos o empresariales pequenos y medianos.
4. **Crear conciencia sobre los riesgos** asociados al uso de sistemas informaticos conectados en red.
5. **Ofrecer herramientas practicas y verificables** para evaluar y mejorar continuamente la postura de seguridad.

Por que Debian 13 "Trixie"

Debian es una de las distribuciones de Linux mas antiguas y respetadas, conocida por:

- **Estabilidad:** Los paquetes de software pasan por rigurosas pruebas antes de ser incluidos.
- **Seguridad:** Cuenta con un equipo dedicado de seguridad que responde rapidamente a vulnerabilidades.

- **Transparencia:** Todo el código es abierto y auditable.
 - **Comunidad activa:** Miles de desarrolladores y usuarios contribuyen a su mejora continua.
-

1.2. Alcance

Definición del Alcance

El alcance de este plan de seguridad define **que elementos están cubiertos** por las políticas y procedimientos descritos en este manual. Comprender el alcance es esencial para saber donde aplicar las medidas de seguridad.

Elementos Cubiertos

1. Computadoras Personales (PCs)

Son las estaciones de trabajo individuales donde los usuarios realizan sus tareas diarias. En el contexto de este manual:

- **Equipos de escritorio:** Computadoras fijas con Debian 13 instalado.
- **Portátiles:** Equipos móviles que pueden conectarse a diferentes redes.
- **Servidores locales:** Máquinas dedicadas a proporcionar servicios específicos.

Ejemplo práctico en Debian:

```
# Verificar la versión de Debian instalada
cat /etc/os-release
```

```
# Salida esperada (parcial):
# PRETTY_NAME="Debian GNU/Linux 13 (trixie)"
# NAME="Debian GNU/Linux"
# VERSION_ID="13"
```

2. Routers

Un **router** (enrutador) es un dispositivo de red que conecta diferentes redes entre sí y dirige el tráfico de datos entre ellas. Funciona como un “director de tráfico” que decide por donde deben viajar los paquetes de información.

Funciones principales: - Conectar la red local (LAN) a Internet (WAN) - Asignar direcciones IP mediante DHCP - Proporcionar una primera línea de defensa mediante el firewall integrado

Ejemplo de verificación de la puerta de enlace (router) en Debian:

```
# Ver la ruta por defecto (dirección del router)
ip route show default
```

```
# Salida esperada:  
# default via 192.168.1.1 dev eth0 proto dhcp metric 100
```

3. Switches

Un **switch** (conmutador) es un dispositivo de red que conecta multiples dispositivos dentro de una misma red local (LAN). A diferencia del router, el switch no conecta redes diferentes, sino que permite la comunicacion entre dispositivos de la misma red.

Características: - Opera en la Capa 2 (enlace de datos) del modelo OSI - Utiliza direcciones MAC para dirigir el trafico - Puede ser gestionado (configurable) o no gestionado (plug-and-play)

Verificacion de interfaces de red en Debian:

```
# Listar todas las interfaces de red  
ip link show  
  
# Ver las direcciones MAC de las interfaces  
ip link show | grep "link/ether"
```

4. Servidores (Fisicos o Virtualizados)

Un **servidor** es una maquina dedicada a proporcionar servicios a otros equipos en la red. Puede ser: - **Servidor fisico:** Hardware dedicado instalado en un rack o armario de comunicaciones. - **Servidor virtualizado:** Entorno de computacion simulado por software que funciona como si fuera una computadora fisica independiente. Permite ejecutar multiples sistemas operativos en un solo equipo fisico.

Un **servidor web** es el software que atiende solicitudes HTTP/HTTPS. Los mas comunes son: - **nginx:** Servidor web de alto rendimiento que tambien puede funcionar como proxy inverso y balanceador de carga. Ampliamente utilizado por su eficiencia y bajo consumo de recursos. - **Apache HTTP Server:** Servidor web modular y extensible, muy flexible y soporta multiples modulos.

Ventajas de la virtualizacion: - **Aislamiento:** Los problemas en un servidor virtualizado no afectan a otros ni al sistema anfitrión. - **Flexibilidad:** Se pueden crear, clonar y eliminar rapidamente. - **Pruebas seguras:** Ideal para probar configuraciones sin riesgo.

Ejemplo de instalacion de un servidor web en Debian:

```
# Actualizar lista de paquetes  
sudo apt update  
  
# Opcion 1: Instalar nginx  
sudo apt install nginx -y
```

```
# Opcion 2: Instalar Apache
# sudo apt install apache2 -y
```

```
# Verificar el estado del servicio (ejemplo con nginx)
sudo systemctl status nginx
```

```
# Comprobar que el servidor web responde
curl -I http://localhost
```

Ejemplo de archivo de configuracion basico de nginx:

```
# /etc/nginx/sites-available/mi-sitio
server {
    listen 80;
    server_name mi-sitio.local;

    root /var/www/mi-sitio;
    index index.html;

    # Cabeceras de seguridad basicas
    add_header X-Frame-Options "SAMEORIGIN";
    add_header X-Content-Type-Options "nosniff";

    location / {
        try_files $uri $uri/ =404;
    }
}
```

Ejemplo de archivo de configuracion basico de Apache:

```
# /etc/apache2/sites-available/mi-sitio.conf
<VirtualHost *:80>
    ServerName mi-sitio.local
    DocumentRoot /var/www/mi-sitio

    # Cabeceras de seguridad basicas
    Header set X-Frame-Options "SAMEORIGIN"
    Header set X-Content-Type-Options "nosniff"

    <Directory /var/www/mi-sitio>
        Require all granted
    </Directory>
</VirtualHost>
```

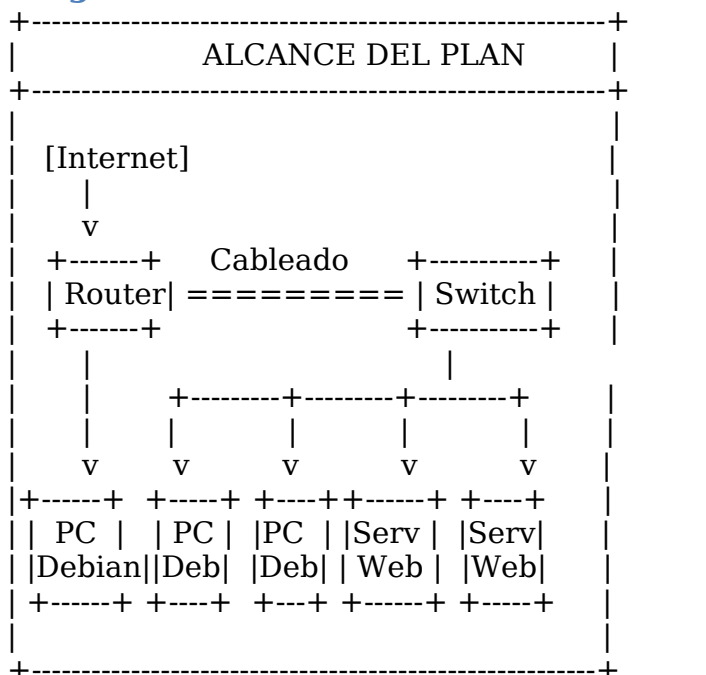
5. Cableado de Red

El **cableado estructurado** comprende toda la infraestructura fisica que permite la comunicacion entre dispositivos:

- **Cables Ethernet (Cat5e/Cat6/Cat6a):** Conectan dispositivos a switches y routers.
- **Cables de fibra optica:** Utilizados para conexiones de alta velocidad y larga distancia.
- **Patch panels:** Paneles de conexion que organizan el cableado.
- **Rosetas y conectores RJ-45:** Puntos de conexion en paredes y equipos.

Consideraciones de seguridad para el cableado: - Evitar que los cables pasen por zonas accesibles al publico - Etiquetar correctamente cada cable y conexion - Proteger fisicamente los armarios de comunicaciones (racks)

Diagrama del Alcance



Exclusiones del Alcance

Los siguientes elementos **NO estan cubiertos** por este plan en su version actual:

- Dispositivos moviles (smartphones, tablets)
- Servicios en la nube de terceros (AWS, Azure, GCP)
- Aplicaciones de usuario final (navegadores, suites ofimaticas)
- Seguridad fisica de las instalaciones (mas alla del cableado)

1.3. Política de Seguridad General

Que es una Política de Seguridad

Una **política de seguridad** es un documento formal que establece las reglas, directrices y principios que rigen como una organización protege sus activos informáticos. Define el “que” y el “por que” de la seguridad, mientras que los procedimientos definen el “como”.

Declaración de Política

“Todos los sistemas informáticos basados en Debian 13 dentro del alcance de este plan deben ser configurados, mantenidos y operados de manera que protejan la confidencialidad, integridad y disponibilidad de la información, cumpliendo con las mejores prácticas de la industria y las regulaciones aplicables.”

Principios Fundamentales

1. Principio de Mínimo Privilegio

Cada usuario y proceso debe tener **únicamente los permisos necesarios** para realizar su función, nada más.

Ejemplo en Debian:

MAL: Dar acceso root a un usuario normal
usermod -aG sudo usuario_normal # NO HACER sin justificación

BIEN: Crear un usuario con permisos limitados para una tarea específica
`sudo useradd -m -s /bin/bash operador_backup`
`sudo chown operador_backup:operador_backup /var/backups/mi_aplicacion`

2. Defensa en Profundidad

No depender de una única medida de seguridad. Implementar **múltiples capas de protección** para que si una falla, las demás sigan protegiendo el sistema.

Capas de defensa en un servidor Debian con servidor web:

Capa	Medida	Herramienta en Debian
1	Firewall de red	Router/iptables
2	Firewall de host	nftables/ufw
3	Seguridad de aplicación	Configuración del servidor web (nginx, Apache, etc.)
4	Autenticación	PAM, SSH keys
5	Cifrado	TLS/SSL, LUKS
6	Monitoreo	fail2ban, auditd

3. Seguridad por Defecto

Los sistemas deben ser **seguros desde su instalacion inicial**. Las funcionalidades inseguras deben requerir habilitacion explicita.

Ejemplo de configuracion segura por defecto en SSH:

/etc/ssh/sshd_config - Configuracion segura por defecto

Deshabilitar acceso root directo

PermitRootLogin no

Usar solo autenticacion por clave

PasswordAuthentication no

PubkeyAuthentication yes

Limitar intentos de autenticacion

MaxAuthTries 3

Tiempo de gracia reducido

LoginGraceTime 30

Reiniciar el servicio para aplicar cambios

sudo systemctl restart sshd

4. Actualizaciones Oportunas

Los sistemas deben mantener el software **actualizado** para corregir vulnerabilidades conocidas.

Configuracion de actualizaciones automaticas en Debian:

Instalar el paquete de actualizaciones automaticas

sudo apt install unattended-upgrades -y

Habilitar las actualizaciones automaticas

sudo dpkg-reconfigure -plow unattended-upgrades

Verificar la configuracion

cat /etc/apt/apt.conf.d/20auto-upgrades

Políticas Específicas

Política de Contraseñas

Requisito	Valor Mínimo
Longitud mínima	12 caracteres
Complejidad	Mayúsculas, minúsculas, números, símbolos
Vigencia máxima	90 días

Requisito	Valor Minimo
Historial	No reutilizar las ultimas 5 contraseñas
Bloqueo por intentos fallidos	Despues de 5 intentos

Implementacion en Debian con PAM:

Instalar libpam-pwquality

```
sudo apt install libpam-pwquality -y
```

Configurar requisitos de contraseña

/etc/security/pwquality.conf

minlen = 12

dcredit = -1 *# Al menos 1 digito*

ucredit = -1 *# Al menos 1 mayuscula*

lcredit = -1 *# Al menos 1 minuscula*

ocredit = -1 *# Al menos 1 simbolo*

Politica de Acceso Remoto

- Todo acceso remoto debe realizarse mediante **SSH con autenticacion por clave publica**.
- Las conexiones deben originarse desde direcciones IP autorizadas cuando sea posible.
- Se debe utilizar **autenticacion de dos factores (2FA)** para sistemas criticos.

Politica de Copias de Seguridad

- Los datos criticos deben respaldarse **diariamente**.
- Las copias deben almacenarse en una ubicacion **separada fisicamente**.
- Se deben realizar **pruebas de restauracion** trimestralmente.

Script basico de backup en Debian:

```
#!/bin/bash
```

```
# /usr/local/bin/backup_diario.sh
```

```
FECHA=$(date +%Y%m%d)
```

```
ORIGEN="/var/www"
```

```
DESTINO="/var/backups/web"
```

Crear copia comprimida

```
tar -czf ${DESTINO}/backup_${FECHA}.tar.gz ${ORIGEN}
```

Eliminar copias mayores a 30 dias

```
find ${DESTINO} -name "backup_*.tar.gz" -mtime +30 -delete
```

Registrar en log

```
echo "$(date): Backup completado" >> /var/log/backups.log
```

1.4. Roles y Responsabilidades (Matriz RACI)

Que es una Matriz RACI

La **Matriz RACI** es una herramienta de gestion que define claramente las responsabilidades de cada rol en un proyecto o proceso. El acronimo RACI significa:

Letra	Significado	Descripcion
R	Responsable (Responsable)	La persona que ejecuta la tarea. Hace el trabajo.
A	Accountable (Autoridad)	La persona con autoridad final y que rinde cuentas. Solo una por tarea.
C	Consulted (Consultado)	Personas cuya opinion se solicita antes de tomar decisiones. Comunicacion bidireccional.
I	Informed (Informado)	Personas que deben ser notificadas de los resultados. Comunicacion unidireccional.

Roles Definidos

1. Director de TI / Responsable de Seguridad

Descripcion: Persona con autoridad maxima sobre las decisiones de seguridad informatica. Responsable de aprobar politicas y asignar recursos.

Responsabilidades principales: - Aprobar el plan de seguridad - Asignar presupuesto - Reportar a la direccion general

2. Administrador de Sistemas

Descripcion: Profesional tecnico encargado de la instalacion, configuracion y mantenimiento de los sistemas Debian.

Responsabilidades principales: - Implementar medidas de seguridad - Aplicar actualizaciones - Configurar servicios (servidor web, SSH, firewall) - Monitorear los sistemas

3. Administrador de Red

Descripcion: Responsable de la infraestructura de red, incluyendo routers, switches y cableado.

Responsabilidades principales: - Configurar equipos de red - Segmentar la red - Mantener el firewall perimetral - Gestionar el cableado

4. Usuario Final

Descripción: Cualquier persona que utiliza los sistemas para realizar su trabajo diario.

Responsabilidades principales: - Seguir las políticas de seguridad - Reportar incidentes sospechosos - Proteger sus credenciales - Completar capacitaciones de seguridad

5. Auditor de Seguridad

Descripción: Persona (interna o externa) encargada de verificar el cumplimiento de las políticas de seguridad.

Responsabilidades principales: - Realizar auditorías periódicas - Identificar vulnerabilidades - Emitir recomendaciones - Verificar la implementación de correcciones

Matriz RACI para Actividades de Seguridad

Actividad	Director TI	Admin. Sistemas	Admin. Red	Usuario	Auditor
Aprobar políticas de seguridad	A	C	C	I	C
Instalar actualizaciones de seguridad	A	R	R	I	I
Configurar firewall de host (nftables)	I	R/A	C	-	I
Configurar firewall de red (router)	I	C	R/A	-	I
Gestionar cuentas de usuario	A	R	-	I	I
Responder a incidentes de seguridad	A	R	R	C	I
Realizar copias de seguridad	I	R/A	-	-	I
Monitorear logs del sistema	I	R	R	-	C
Configurar servidor	A	R	C	-	I

Actividad	Director TI	Admin. Sistemas	Admin. Red	Usuario	Auditor
web de forma segura					
Gestionar certificados SSL/TLS	I	R/A	-	-	I
Realizar auditorias de seguridad	A	C	C	-	R
Capacitar a usuarios	A	R	R	R	C
Documentar procedimientos	A	R	R	-	C
Gestionar acceso fisico al cableado	A	C	R	-	I
Planificar recuperacion ante desastres	A	R	R	I	C

Ejemplo Practico: Actualizacion de Seguridad Critica

Cuando se publica una vulnerabilidad critica que afecta a un servidor web (nginx, Apache, etc.):

1. **Auditor (I):** Es informado de la vulnerabilidad a traves de canales oficiales de Debian Security.
2. **Director de TI (A):** Recibe notificacion y autoriza la actualizacion de emergencia.
3. **Administrador de Sistemas (R):** Ejecuta la actualizacion:

Verificar si hay actualizaciones de seguridad disponibles
`sudo apt update`

Ver actualizaciones pendientes
`apt list --upgradable | grep -E "(nginx|apache2)"`

Aplicar la actualizacion (ejemplo con nginx)
`sudo apt upgrade nginx -y`

O con Apache

```
# sudo apt upgrade apache2 -y

# Verificar la nueva version
nginx -v
# o apache2 -v

# Reiniciar el servicio
sudo systemctl restart nginx
# o sudo systemctl restart apache2
```

4. **Administrador de Red (C):** Es consultado sobre posibles impactos en la red.
 5. **Usuarios (I):** Son informados de una breve interrupcion del servicio.
-

1.5. Clasificacion de Activos e Informacion

Que son los Activos Informaticos

Los **activos informaticos** son todos los recursos que tienen valor para una organizacion y que estan relacionados con el procesamiento, almacenamiento o transmision de informacion. Estos pueden ser:

- **Activos de hardware:** Servidores, PCs, routers, switches, cables.
- **Activos de software:** Sistemas operativos, aplicaciones, bases de datos.
- **Activos de informacion:** Documentos, bases de datos, configuraciones.
- **Activos humanos:** Personal con conocimientos criticos.
- **Activos de servicio:** Servicios que dependen de la infraestructura (web, correo).

Inventario de Activos

Es fundamental mantener un **inventario actualizado** de todos los activos. A continuacion se presenta una plantilla:

ID	Tipo	Nombre	Ubicacion	Propietario	Criticidad	Ultima Revision
H W- 00 1	Hard ware	Servido r Web Princip al	Rack A, U12	Admin. Sistemas	Alta	2026-02-01
H W- 00 2	Hard ware	Router Princip al	Rack A, U1	Admin. Red	Critica	2026-02-01
H W- 00	Hard ware	Switch Core	Rack A, U3	Admin. Red	Alta	2026-02-01

ID	Tipo	Nombre	Ubicacion	Propietario	Criticidad	Ultima Revision
3						
S W- 00 1	Soft ware	Debian 13	Servidor Web	Admin. Sistemas	Alta	2026-02-01
S W- 00 2	Soft ware	Servido r Web (nginx/ Apache)	Servidor Web	Admin. Sistemas	Alta	2026-02-01
SR V- 00 1	Servi dor	Servido r Web Prod (Virtuali zado)	Servidor Web	Admin. Sistemas	Alta	2026-02-01
IN F- 00 1	Infor maci on	BD Clientes	Servidor BD	Director TI	Critica	2026-02-01

Script para generar inventario basico en Debian:

```
#!/bin/bash
# /usr/local/bin/inventario_sistema.sh

echo "=== INVENTARIO DEL SISTEMA ==="
echo "Fecha: $(date)"
echo ""
echo "=== INFORMACION DEL SISTEMA ==="
hostnamectl
echo ""
echo "=== VERSION DE DEBIAN ==="
cat /etc/os-release | grep -E "^(PRETTY_NAME|VERSION)"
echo ""
echo "=== HARDWARE ==="
echo "CPU: $(lscpu | grep 'Model name' | cut -d':' -f2 | xargs)"
echo "RAM: $(free -h | grep Mem | awk '{print $2}')"
echo "Disco: $(df -h / | tail -1 | awk '{print $2}')"
echo ""
echo "=== INTERFACES DE RED ==="
ip -br addr show
echo ""
echo "=== SERVICIOS ACTIVOS ==="
systemctl list-units --type=service --state=running --no-pager | head -20
```

Clasificación de la Información

La información debe clasificarse según su **sensibilidad** para aplicar controles de seguridad apropiados:

Nivel	Clasificación	Descripción	Ejemplos	Controles
4	Confidencial	Información muy sensible. Su divulgación causaría daño grave.	Contraseñas, claves privadas, datos financieros	Cifrado, acceso muy restringido, auditoría completa
3	Restringida	Información interna sensible. Solo para personal autorizado.	Configuraciones de seguridad, informes de auditorías	Acceso controlado, cifrado en tránsito
2	Interna	Información para uso interno. No debe ser pública.	Procedimientos, documentación técnica	Acceso interno, sin exposición pública
1	Pública	Información que puede ser conocida por cualquiera.	Sitio web público, documentación de productos	Protección de integridad

Etiquetado de Archivos en Debian

Usando atributos extendidos para clasificar archivos:

Instalar herramientas de atributos extendidos

```
sudo apt install attr -y
```

Etiquetar un archivo como CONFIDENCIAL

```
setfattr -n user.clasificacion -v "CONFIDENCIAL" /etc/ssl/private/mi-clave.key
```

Verificar la etiqueta

```
getfattr -n user.clasificacion /etc/ssl/private/mi-clave.key
```

Buscar archivos con una clasificación específica

```
getfattr -R -n user.clasificacion /etc/ssl/ 2>/dev/null
```

Permisos recomendados según clasificación:

Archivos CONFIDENCIALES (nivel 4)

Solo lectura para el propietario, ningún acceso para otros

```
chmod 400 archivo_confidencial.key
```

```
chown root:root archivo_confidencial.key
```

Archivos RESTRINGIDOS (nivel 3)

Lectura/escritura propietario, lectura grupo específico

chmod 640 archivo_restringido.conf

chown root:administradores archivo_restringido.conf

Archivos INTERNOS (nivel 2)

Lectura para todos los usuarios del sistema

chmod 644 archivo_interno.txt

Archivos PUBLICOS (nivel 1)

Acceso de lectura global

chmod 644 /var/www/html/index.html

Ubicaciones Críticas en Debian

Directorio	Contenido	Clasificación Típica
/etc/ssl/private/	Claves privadas SSL/TLS	Confidencial
/etc/shadow	Hashes de contrasenas	Confidencial
/etc/ssh/	Configuración SSH, claves de host	Restringida
/etc/nginx/ o /etc/apache2/	Configuración del servidor web	Restringida
/var/log/	Registros del sistema	Interna
/var/www/html/	Contenido web público	Pública

1.6. Gestión de Riesgos y Presupuesto

Que es la Gestión de Riesgos

La **gestión de riesgos** es el proceso sistemático de identificar, evaluar y tratar los riesgos que podrían afectar negativamente los activos informáticos. Un **riesgo** se define como la combinación de la probabilidad de que ocurra una amenaza y el impacto que tendría si ocurre.

Formula del riesgo:

Riesgo = Probabilidad x Impacto

Metodología de Evaluación

Escala de Probabilidad

Valor	Nivel	Descripción	Frecuencia Estimada
5	Muy Alta	Ocurrirá con certeza	Diariamente

Valor	Nivel	Descripcion	Frecuencia Estimada
4	Alta	Es muy probable que ocurra	Semanalmente
3	Media	Podria ocurrir	Mensualmente
2	Baja	Poco probable	Anualmente
1	Muy Baja	Altamente improbable	Menos de una vez al ano

Escala de Impacto

Valor	Nivel	Descripcion	Consecuencias
5	Critico	Dano catastrofico	Perdida total de datos, cierre de operaciones
4	Alto	Dano severo	Interrupcion prolongada, perdida financiera significativa
3	Medio	Dano moderado	Interrupcion parcial, costos de recuperacion
2	Bajo	Dano menor	Inconvenientes menores, rapida recuperacion
1	Minimo	Dano insignificante	Sin impacto real en operaciones

Matriz de Riesgos

	Impacto 1	Impacto 2	Impacto 3	Impacto 4	Impacto 5
Prob 5	5 (Medio)	10 (Alto)	15 (Critico)	20 (Critico)	25 (Critico)
Prob 4	4 (Bajo)	8 (Medio)	12 (Alto)	16 (Critico)	20 (Critico)
Prob 3	3 (Bajo)	6 (Medio)	9 (Medio)	12 (Alto)	15 (Critico)
Prob 2	2 (Bajo)	4 (Bajo)	6 (Medio)	8 (Medio)	10 (Alto)
Prob 1	1 (Bajo)	2 (Bajo)	3 (Bajo)	4 (Bajo)	5 (Medio)

Leyenda de colores: - **Critico (15-25):** Accion inmediata requerida - **Alto (10-14):** Planificar mitigacion a corto plazo - **Medio (5-9):** Monitorear y planificar mitigacion - **Bajo (1-4):** Aceptar o monitorear

Registro de Riesgos Identificados

ID	Amenaza	Activo Afectado	Prob.	Imp.	Riesgo	Tratamiento
R-001	Acceso no autorizado por SSH	Servidores Debian	4	4	16	Mitigar: Deshabilitar acceso root, usar claves SSH
R-002	Vulnerabilidad en servidor web	Servidor web (fisico o virtualizado)	3	4	12	Mitigar: Actualizaciones automaticas

ID	Amenaza	Activo Afectado	Prob.	Imp.	Riesgo	Tratamiento
R-003	Fallo del router principal	Toda la red	2	5	10	Mitigar: Router de respaldo
R-004	Malware en estacion de trabajo	PCs usuarios	3	3	9	Mitigar: ClamAV, restriccion de privilegios
R-005	Perdida de datos por fallo de disco	Servidor BD	2	5	10	Mitigar: RAID, backups diarios
R-006	Denegacion de servicio (DoS)	Servidor web	3	3	9	Mitigar: Rate limiting en servidor web
R-007	Acceso fisico no autorizado	Switch/Router	2	4	8	Mitigar: Rack cerrado con llave
R-008	Configuracion incorrecta de firewall	Red interna	2	4	8	Mitigar: Revision periodica, documentacion

Estrategias de Tratamiento

1. **Mitigar:** Implementar controles para reducir la probabilidad o el impacto.
2. **Transferir:** Trasladar el riesgo a un tercero (seguros, servicios gestionados).
3. **Aceptar:** Reconocer el riesgo y no tomar accion (para riesgos bajos).
4. **Evitar:** Eliminar la actividad que genera el riesgo.

Presupuesto de Seguridad

Categorias de Gasto

Categoria	Descripcion	% Recomendado
Hardware de seguridad	Firewall dedicado, router con funciones avanzadas	20-25%
Software/Licencias	Herramientas de monitoreo, antivirus empresarial	15-20%
Capacitacion	Cursos para administradores, concienciacion usuarios	10-15%
Auditorias externas	Pruebas de penetracion, auditorias de cumplimiento	15-20%
Respuesta a incidentes	Fondo de contingencia para emergencias	10-15%
Mejora continua	Actualizaciones, nuevas	15-20%

Categoria	Descripcion	% Recomendado
	herramientas	

Ejemplo de Presupuesto Anual

Item	Detalle	Costo Estimado
Router empresarial con firewall	Reemplazo cada 5 anos	€200/año
Switch gestionado	Reemplazo cada 5 anos	€100/año
UPS (Sistema de alimentacion ininterrumpida)	Para servidores criticos	€150/año
Certificados SSL/TLS	Let's Encrypt (gratis) o comercial	€0-100/año
Almacenamiento o para backups	Disco externo o NAS	€200/año
Capacitacion del personal	Cursos online, certificaciones	€500/año
Auditoria de seguridad externa	Anual	€1,000/año
Fondo de contingencia	Emergencias	€300/año
TOTAL ESTIMADO		€2,450-2,550/año

Herramientas Gratuitas/Open Source para Debian

Debian ofrece muchas herramientas de seguridad sin costo de licencia:

Herramientas incluidas en repositorios oficiales de Debian

```
sudo apt install \
  ufw \           # Firewall simplificado
  fail2ban \      # Proteccion contra fuerza bruta
  clamav \        # Antivirus
  rkhunter \      # Detector de rootkits
  lynis \         # Auditoria de seguridad
  auditd \        # Sistema de auditoria
  apparmor \      # Control de acceso obligatorio
-y
```

1.7. Procedimiento Basico de Respuesta a Incidentes

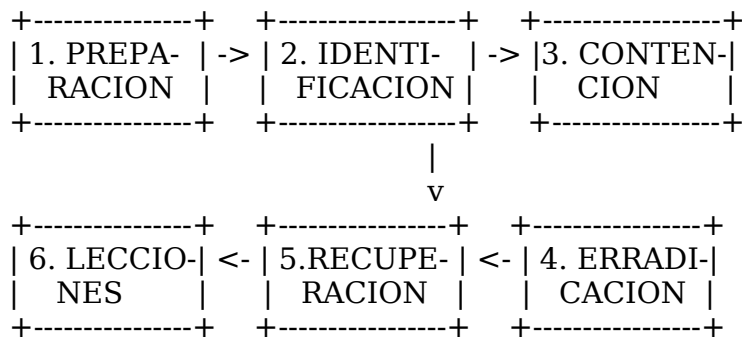
Que es un Incidente de Seguridad

Un **incidente de seguridad** es cualquier evento que compromete o tiene el potencial de comprometer la confidencialidad, integridad o disponibilidad de los sistemas informaticos o la informacion que contienen.

Ejemplos de incidentes: - Acceso no autorizado a un sistema - Infeccion por malware - Fuga de datos sensibles - Ataque de denegacion de servicio (DoS) - Perdida o robo de equipos - Uso indebido de privilegios

Fases de Respuesta a Incidentes

El proceso de respuesta se divide en **seis fases**:



Fase 1: Preparacion

Objetivo: Estar listos para responder antes de que ocurra un incidente.

Acciones: - Formar un equipo de respuesta a incidentes - Documentar procedimientos - Configurar herramientas de monitoreo - Preparar kit de herramientas forenses - Establecer canales de comunicacion de emergencia

Kit de herramientas para Debian:

Crear directorio para herramientas de incidentes
`sudo mkdir -p /opt/incident-response`

Herramientas utiles para analisis

```

sudo apt install \
    tcpdump \           # Captura de trafico de red
    wireshark-common \  # Analisis de paquetes (CLI)
    htop \              # Monitor de procesos
    iotop \             # Monitor de I/O
    lsof \              # Listar archivos abiertos
    strace \            # Trazar llamadas al sistema
    binwalk \           # Analisis de archivos binarios
    -y
    
```

```
# Script de recoleccion rapida de evidencia
cat > /opt/incident-response/recolectar_evidencia.sh << 'EOF'
#!/bin/bash
# Script de recoleccion de evidencia
FECHA=$(date +%Y%m%d_%H%M%S)
DIR_EVIDENCIA="/var/incident_${FECHA}"
mkdir -p ${DIR_EVIDENCIA}

echo "Recolectando evidencia en ${DIR_EVIDENCIA}..."

# Informacion del sistema
uname -a > ${DIR_EVIDENCIA}/sistema.txt
uptime >> ${DIR_EVIDENCIA}/sistema.txt

# Usuarios conectados
who > ${DIR_EVIDENCIA}/usuarios_conectados.txt
last -100 > ${DIR_EVIDENCIA}/ultimos_accesos.txt

# Procesos en ejecucion
ps auxwww > ${DIR_EVIDENCIA}/procesos.txt

# Conexiones de red
ss -tulpn > ${DIR_EVIDENCIA}/conexiones_red.txt
netstat -an >> ${DIR_EVIDENCIA}/conexiones_red.txt

# Archivos abiertos
lsof > ${DIR_EVIDENCIA}/archivos_abiertos.txt 2>/dev/null

# Ultimas modificaciones en /etc
find /etc -mtime -1 -type f > ${DIR_EVIDENCIA}/etc_modificados.txt

# Logs recientes
cp -r /var/log/auth.log* ${DIR_EVIDENCIA}/
cp -r /var/log/syslog* ${DIR_EVIDENCIA}/

echo "Evidencia recolectada en: ${DIR_EVIDENCIA}"
EOF

sudo chmod +x /opt/incident-response/recolectar_evidencia.sh
```

Fase 2: Identificacion

Objetivo: Detectar y confirmar que ha ocurrido un incidente.

Fuentes de deteccion: - Alertas de sistemas de monitoreo - Reportes de usuarios
- Analisis de logs - Deteccion de anomalias

Comandos utiles para identificacion en Debian:

Revisar intentos de acceso fallidos

`sudo grep "Failed password" /var/log/auth.log | tail -20`

Verificar accesos SSH exitosos

`sudo grep "Accepted" /var/log/auth.log | tail -20`

Buscar actividad sospechosa de cron

`sudo grep CRON /var/log/syslog | tail -20`

Verificar procesos con conexiones de red activas

`sudo ss -tulpn`

Buscar archivos modificados recientemente en directorios criticos

`sudo find /etc -mtime -1 -type f -ls`

`sudo find /var/www -mtime -1 -type f -ls`

Verificar integridad de paquetes del sistema

`sudo debsums -s 2>/dev/null`

Clasificacion del incidente:

Severidad	Descripcion	Tiempo de Respuesta
Critica	Compromiso activo de sistemas, fuga de datos en curso	Inmediato (< 1 hora)
Alta	Sistema comprometido, servicio no disponible	< 4 horas
Media	Intento de ataque detectado, vulnerabilidad explotable	< 24 horas
Baja	Anomalia menor, violacion de politica	< 72 horas

Fase 3: Contencion

Objetivo: Limitar el daño y evitar que el incidente se propague.

Contencion a corto plazo (inmediata):

OPCION 1: Aislar el sistema de la red (desconectar cable)

Fisicamente desconectar el cable de red

OPCION 2: Deshabilitar la interfaz de red (mantiene el sistema encendido)

`sudo ip link set eth0 down`

OPCION 3: Bloquear una IP especifica sospechosa

```
sudo iptables -A INPUT -s 203.0.113.100 -j DROP
sudo iptables -A OUTPUT -d 203.0.113.100 -j DROP
```

OPCION 4: Deshabilitar una cuenta comprometida

```
sudo usermod -L usuario_comprometido
sudo pkill -u usuario_comprometido
```

OPCION 5: Detener un servicio comprometido

```
sudo systemctl stop nginx
```

Contencion a largo plazo: - Crear imagen forense del sistema afectado - Activar sistemas de respaldo - Aplicar parches de emergencia

Fase 4: Erradicacion

Objetivo: Eliminar la causa raiz del incidente.

Acciones tipicas:

Eliminar archivos maliciosos identificados

```
sudo rm -f /tmp/archivo_malicioso.sh
```

Eliminar cuentas de usuario no autorizadas

```
sudo userdel -r usuario_malicioso
```

Eliminar tareas cron no autorizadas

```
sudo crontab -r -u usuario_comprometido
```

Limpiar tareas de at

```
sudo atrm $(atq | cut -f1)
```

Reinstalar paquetes potencialmente comprometidos

```
sudo apt install --reinstall nginx
```

Cambiar todas las contraseñas afectadas

```
sudo passwd usuario_afectado
```

Regenerar claves SSH del host

```
sudo rm /etc/ssh/ssh_host_*
sudo dpkg-reconfigure openssh-server
```

Fase 5: Recuperacion

Objetivo: Restaurar los sistemas a su funcionamiento normal.

Pasos de recuperacion:

Restaurar desde backup verificado

```
sudo tar -xzf /var/backups/web/backup_20260210.tar.gz -C /var/www/
```

Verificar integridad de los archivos restaurados

```
find /var/www -type f -exec md5sum {} \; > /tmp/checksums_restaurado.txt
```

Reactivar interfaces de red

```
sudo ip link set eth0 up
```

Reiniciar servicios

```
sudo systemctl start nginx
```

```
sudo systemctl start ssh
```

Verificar funcionamiento

```
curl -I http://localhost
```

```
sudo systemctl status nginx
```

Monitorear intensivamente durante las primeras horas

```
sudo tail -f /var/log/nginx/access.log /var/log/auth.log
```

Fase 6: Lecciones Aprendidas

Objetivo: Documentar el incidente y mejorar los procesos.

Documentacion requerida:

Elemento	Descripcion
Resumen ejecutivo	Descripcion breve del incidente y su impacto
Linea de tiempo	Cronologia detallada de eventos
Sistemas afectados	Lista de activos comprometidos
Vector de ataque	Como ocurrio el incidente
Acciones tomadas	Pasos de respuesta ejecutados
Efectividad	Que funciono y que no
Recomendaciones	Mejoras para prevenir incidentes similares
Metricas	Tiempo de deteccion, contencion, recuperacion

Plantilla de informe post-incidente:

Informe Post-Incidente: [ID-INCIDENTE]

Resumen Ejecutivo

[Descripcion en 2-3 parrafos]

Clasificacion

- Severidad: [Critica/Alta/Media/Baja]

- Tipo: [Malware/Acceso no autorizado/DoS/etc.]

Linea de Tiempo

Fecha/Hora	Evento
2026-02-13 10:00	Primera alerta detectada
2026-02-13 10:15	Incidente confirmado

Sistemas Afectados

- [Lista de sistemas]

Analisis de Causa Raiz

[Descripcion tecnica]

Acciones de Remediacion

1. [Accion 1]

2. [Accion 2]

Recomendaciones

- [Recomendacion 1]

- [Recomendacion 2]

Aprobaciones

- Elaborado por: [Nombre]

- Revisado por: [Nombre]

- Fecha: [Fecha]

Checklist Nivel 0

Instrucciones de Uso

Este checklist esta disenado para verificar la implementacion del Plan de Seguridad Global. Cada item debe ser marcado como: - ☐ Pendiente - ☒ Completado - **[N/A]** No aplicable

Fecha de revision: _____ Responsable de la revision: _____

1. Documentacion y Politicas

#	Item	Estado	Evidencia/Notas
1.1	El plan de seguridad esta documentado y aprobado por la direccion	☐	
1.2	Las politicas	☐	

#	Item	Estado	Evidencia/Notas
	de seguridad están publicadas y accesibles para todo el personal		
1.3	Existe una política de contraseñas documentada	[]	
1.4	Existe una política de acceso remoto documentada	[]	
1.5	Existe una política de copias de seguridad documentada	[]	
1.6	Los roles y responsabilidades están definidos (Matriz RACI)	[]	
1.7	El procedimiento de respuesta a incidentes está documentado	[]	

Comando de verificación:

```
# Verificar existencia de documentación
ls -la /usr/local/share/security-docs/
```

2. Inventario de Activos

#	Item	Estado	Evidencia/Notas
2.1	Existe un inventario actualizado de todos los servidores Debian (físicos o	[]	

#	Item	Estado	Evidencia/Notas
	virtualizados)		
2.2	Existe un inventario de equipos de red (routers, switches)	[]	
2.3	Existe un inventario de servidores web (nginx, Apache, etc.)	[]	
2.4	El cableado de red esta documentado y etiquetado	[]	
2.5	Cada activo tiene un propietario asignado	[]	
2.6	Cada activo tiene una clasificacion de criticidad	[]	
2.7	El inventario se actualiza al menos trimestralmente	[]	

Comando de verificacion:

Generar inventario basico del sistema

hostnamectl && echo "---" && ip addr show && echo "---" && lsblk

3. Clasificacion de la Informacion

#	Item	Estado	Evidencia/Notas
3.1	Existe un esquema de clasificacion de informacion definido	[]	
3.2	Los archivos confidenciales tienen permisos 400	[]	

#	Item	Estado	Evidencia/Notas
3.3	Las claves privadas estan en /etc/ssl/private/ con permisos correctos	[]	
3.4	El archivo /etc/shadow tiene permisos 640	[]	
3.5	Los logs del sistema tienen permisos apropiados	[]	

Comandos de verificacion:

Verificar permisos de archivos sensibles

`ls -la /etc/shadow`

`ls -la /etc/ssl/private/`

`stat -c '%a %n' /etc/ssh/ssh_host_*_key`

4. Gestion de Riesgos

#	Item	Estado	Evidencia/Notas
4.1	Existe un registro de riesgos documentado	[]	
4.2	Los riesgos estan evaluados (probabilidad x impacto)	[]	
4.3	Cada riesgo tiene una estrategia de tratamiento asignada	[]	
4.4	Existe un presupuesto asignado para seguridad	[]	

#	Item	Estado	Evidencia/Notas
4.5	Los riesgos criticos tienen planes de mitigacion activos	[]	
4.6	La evaluacion de riesgos se revisa al menos anualmente	[]	

5. Configuracion Basica de Seguridad en Debian

#	Item	Estado	Evidencia/Notas
5.1	El sistema esta actualizado (sin actualizacion es pendientes de seguridad)	[]	
5.2	Las actualizacion es automaticas estan configuradas	[]	
5.3	El firewall (ufw o nftables) esta activo	[]	
5.4	El acceso SSH por root esta deshabilitado	[]	
5.5	La autenticacion SSH por contrasena esta deshabilitada (solo claves)	[]	
5.6	fail2ban esta instalado y configurado	[]	
5.7	Los servicios innecesarios estan	[]	

#	Item	Estado	Evidencia/Notas
	deshabilitados		

Comandos de verificación:

5.1 - Verificar actualizaciones pendientes

`sudo apt update && apt list --upgradable`

5.2 - Verificar actualizaciones automáticas

`cat /etc/apt/apt.conf.d/20auto-upgrades`

5.3 - Verificar estado del firewall

`sudo ufw status verbose`

0

`sudo nft list ruleset`

5.4 y 5.5 - Verificar configuración SSH

`sudo grep -E "^(PermitRootLogin|PasswordAuthentication)" /etc/ssh/sshd_config`

5.6 - Verificar fail2ban

`sudo systemctl status fail2ban`

`sudo fail2ban-client status`

5.7 - Listar servicios activos

`systemctl list-units --type=service --state=running`

6. Preparación para Respuesta a Incidentes

#	Item	Estado	Evidencia/Notas
6.1	Existe un equipo de respuesta a incidentes designado	[]	
6.2	Los contactos de emergencia están documentados y accesibles	[]	
6.3	Las herramientas de recolección de evidencia están instaladas	[]	

#	Item	Estado	Evidencia/Notas
6.4	Existe un script de recoleccion rapida de evidencia	[]	
6.5	Los logs del sistema se retienen por al menos 90 dias	[]	
6.6	El personal conoce el procedimient o basico de reporte de incidentes	[]	

Comandos de verificacion:

6.3 - Verificar herramientas instaladas
`which tcpdump lsof htop`

6.4 - Verificar script de evidencia
`ls -la /opt/incident-response/`

6.5 - Verificar retencion de logs
`cat /etc/logrotate.conf | grep -A5 "rotate"`

7. Copias de Seguridad

#	Item	Estado	Evidencia/Notas
7.1	Existe un programa de copias de seguridad automatizado	[]	
7.2	Las copias se almacenan en una ubicacion separada	[]	
7.3	Las copias de datos criticos se realizan diariamente	[]	
7.4	Se han realizado pruebas de	[]	

#	Item	Estado	Evidencia/Notas
7.5	restauracion en los ultimos 90 dias Las copias de seguridad estan cifradas	[]	

Comandos de verificacion:

Verificar existencia de backups recientes

`ls -la /var/backups/`

`find /var/backups -type f -mtime -7`

Verificar tareas de cron para backups

`sudo crontab -l | grep -i backup`

8. Seguridad de Red

#	Item	Estado	Evidencia/Notas
8.1	El router tiene configurada una contraseña segura (no la de fabrica)	[]	
8.2	El acceso de administracion al router esta limitado a IPs internas	[]	
8.3	Los switches gestionados tienen contraseñas configuradas	[]	
8.4	Los puertos de switch no utilizados estan deshabilitados	[]	
8.5	El cableado de red esta protegido fisicamente (armarios)	[]	

#	Item	Estado	Evidencia/Notas
8.6	cerrados) Existe documentacion del diagrama de red	[]	

9. Servidores Web (Si Aplica)

#	Item	Estado	Evidencia/Notas
9.1	El servidor web (nginx, Apache, etc.) esta actualizado a la ultima version estable	[]	
9.2	La version del servidor web no se muestra en las respuestas HTTP	[]	
9.3	Las cabeceras de seguridad estan configuradas (X-Frame-Options, etc.)	[]	
9.4	Los archivos de configuracion del servidor web tienen permisos restrictivos	[]	
9.5	SSL/TLS esta configurado correctamente (si aplica)	[]	
9.6	Los logs de acceso y error estan habilitados	[]	

Comandos de verificacion:

9.1 - Version del servidor web (nginx)

```
nginx -v
```

9.1 - Version del servidor web (Apache)

```
apache2 -v
```

9.2 - Verificar que no se muestre la version (nginx)

```
curl -I http://localhost 2>/dev/null | grep -i server
```

9.3 - Verificar cabeceras de seguridad

```
curl -I http://localhost 2>/dev/null | grep -iE "(X-Frame|X-Content|X-XSS)"
```

9.4 - Permisos de configuracion

```
ls -la /etc/nginx/nginx.conf
```

```
# o
```

```
ls -la /etc/apache2/apache2.conf
```

9.6 - Verificar logs

```
ls -la /var/log/nginx/
```

```
# o
```

```
ls -la /var/log/apache2/
```

Resumen de Cumplimiento

Seccion	Total Items	Completados	Porcentaje
1.	7		%
Documentacion y Politicas			
2. Inventario de Activos	7		%
3. Clasificacion de Informacion	5		%
4. Gestion de Riesgos	6		%
5. Configuracion Basica Debian	7		%
6. Respuesta a Incidentes	6		%
7. Copias de Seguridad	5		%
8. Seguridad de Red	6		%
9. nginx	6		%
TOTAL	55		%

Proximos Pasos

Una vez completado este checklist del Nivel 0:

1. **Documentar las brechas:** Registrar todos los items no completados.
 2. **Priorizar:** Ordenar las brechas por criticidad del riesgo asociado.
 3. **Planificar:** Crear un plan de accion con fechas y responsables.
 4. **Implementar:** Ejecutar las acciones de remediacion.
 5. **Verificar:** Realizar una nueva revision del checklist.
 6. **Avanzar:** Proceder al Nivel 1 del manual de seguridad.
-

Glosario de Terminos

Termino	Definicion
Activo informatico	Cualquier recurso de valor relacionado con la informacion (hardware, software, datos).
APT	Advanced Package Tool, sistema de gestion de paquetes de Debian.
CIA (Triada)	Confidencialidad, Integridad y Disponibilidad: los tres pilares de la seguridad.
DoS	Denial of Service (Denegacion de Servicio): ataque que busca inutilizar un servicio.
Firewall	Sistema que controla el trafico de red segun reglas predefinidas.
Incidente de seguridad	Evento que compromete la confidencialidad, integridad o disponibilidad.
Log	Registro cronologico de eventos del sistema.
Malware	Software malicioso disenado para danar o infiltrar sistemas.
Servidor Web	Software que atiende solicitudes HTTP/HTTPS. Ejemplos: nginx, Apache HTTP Server.
RACI	Responsible, Accountable, Consulted, Informed: matriz de asignacion de responsabilidades.
Router	Dispositivo que conecta y dirige trafico entre diferentes redes.
SSH	Secure Shell: protocolo para acceso remoto seguro.
Switch	Dispositivo que conecta multiples dispositivos en una misma red local.
TLS/SSL	Protocolos de cifrado para comunicaciones seguras en red.
Servidor	Maquina (fisica o virtualizada) dedicada a proporcionar servicios en la red.
Vulnerabilidad	Debilidad en un sistema que puede ser explotada.

Nivel 1: Tipos y Clasificación de Seguridad

Manual de Seguridad Informática para Debian 13 “Trixie”

Introducción

Este documento constituye el primer nivel del manual de seguridad informática orientado a sistemas Debian 13 “Trixie”. Su objetivo es proporcionar los fundamentos teóricos y conceptuales necesarios para comprender cómo se estructura, clasifica y gestiona la seguridad en entornos informáticos.

Nota para principiantes: A lo largo de este documento, cada término técnico será definido la primera vez que aparezca, permitiendo una comprensión progresiva de los conceptos.

2.1. Tipos de Seguridad

La seguridad informática se divide en cuatro categorías fundamentales que trabajan de manera complementaria para proteger los activos de una organización.

2.1.1. Seguridad Física

Definición: La **seguridad física** comprende todas las medidas destinadas a proteger el hardware (equipos físicos), las instalaciones y el entorno donde se encuentran los sistemas informáticos contra amenazas tangibles como robos, desastres naturales o accesos no autorizados.

Elementos de protección física en Debian 13:

Elemento	Descripción	Ejemplo en Debian
Control de acceso	Restricción de entrada a salas de servidores	Cerraduras, tarjetas de acceso
Protección ambiental	Resguardo contra factores ambientales	Sistemas de refrigeración, detectores de humo
Redundancia de hardware	Duplicación de componentes críticos	Discos en RAID, fuentes de alimentación redundantes
Respaldo energético	Continuidad eléctrica	UPS (Sistema de Alimentación Ininterrumpida)

Configuración de ejemplo - Monitoreo de temperatura en Debian:

Instalar herramientas de monitoreo de hardware

```
sudo apt install lm-sensors
```

Detectar sensores disponibles

```
sudo sensors-detect
```

Visualizar temperaturas del sistema
sensors

2.1.2. Seguridad Lógica

Definición: La **seguridad lógica** engloba los mecanismos de software y configuración que protegen los datos, aplicaciones y sistemas operativos contra accesos no autorizados, malware (software malicioso) y otras amenazas digitales.

Componentes principales en Debian 13:

7. **Autenticación:** Proceso de verificar la identidad de un usuario o sistema.

Configurar política de contraseñas en Debian
sudo apt install libpam-pwquality
sudo nano /etc/security/pwquality.conf

8. **Autorización:** Determinar qué acciones puede realizar un usuario autenticado.

Gestión de permisos en archivos
chmod 750 /ruta/archivo # rwxr-x---
chown usuario:grupo /ruta/archivo

9. **Cifrado:** Proceso de convertir información legible en código ilegible para protegerla.

Cifrar partición con LUKS (Linux Unified Key Setup)
sudo cryptsetup luksFormat /dev/sdX

10. **Firewall:** Sistema que filtra el tráfico de red según reglas predefinidas.

Configurar nftables (firewall por defecto en Debian 13)
sudo apt install nftables
sudo systemctl enable nftables

2.1.3. Seguridad Organizativa

Definición: La **seguridad organizativa** abarca las políticas, procedimientos, normas y estructuras que una organización implementa para gestionar la seguridad de manera sistemática.

Elementos clave:

Componente	Descripción
Políticas de seguridad	Documentos que establecen las directrices generales de seguridad
Procedimientos operativos	Instrucciones detalladas para ejecutar tareas de seguridad
Gestión de riesgos	Proceso de identificar, analizar y mitigar amenazas

Componente	Descripción
Plan de continuidad	Estrategia para mantener operaciones durante incidentes
Auditorías	Revisiones periódicas del cumplimiento de políticas

Ejemplo de política de contraseñas para Debian:

POLÍTICA DE CONTRASEÑAS - Versión 1.0

1. Longitud mínima: 12 caracteres
2. Complejidad: mayúsculas, minúsculas, números y símbolos
3. Vigencia máxima: 90 días
4. Historial: no reutilizar las últimas 5 contraseñas
5. Bloqueo: después de 5 intentos fallidos

2.1.4. Seguridad Personal

Definición: La **seguridad personal** (o seguridad del personal) se refiere a las medidas relacionadas con el factor humano: formación, concienciación, verificación de antecedentes y gestión del ciclo de vida de los empleados en relación con la seguridad.

Aspectos fundamentales:

- **Concienciación:** Programas de formación sobre amenazas como el **phishing** (técnica de engaño para obtener información confidencial).
- **Principio de mínimo privilegio:** Otorgar solo los permisos estrictamente necesarios.
- **Segregación de funciones:** Dividir tareas críticas entre múltiples personas.
- **Gestión de bajas:** Procedimientos para revocar accesos al cesar la relación laboral.

Implementación en Debian - Gestión de usuarios:

Crear usuario con privilegios limitados

```
sudo adduser --disabled-login --gecos "Usuario Temporal" usuario_temp
```

Asignar a grupo específico

```
sudo usermod -aG grupo_trabajo usuario_temp
```

Establecer fecha de expiración de cuenta

```
sudo chage -E 2026-12-31 usuario_temp
```

Verificar configuración de cuenta

```
sudo chage -l usuario_temp
```

2.2. Modelos Multinivel

Los modelos multinivel proporcionan marcos de referencia para evaluar y estructurar la seguridad de sistemas informáticos.

2.2.1. TCSEC (Trusted Computer System Evaluation Criteria)

Definición: El **TCSEC** (Criterios de Evaluación de Sistemas Informáticos de Confianza), también conocido como **Libro Naranja** (Orange Book), es un estándar del Departamento de Defensa de Estados Unidos publicado en 1983 que define criterios para evaluar la seguridad de sistemas informáticos.

Niveles de seguridad TCSEC:

División	Nivel	Nombre	Descripción
D	D	Protección Mínima	Sin seguridad; sistemas que no cumplen requisitos superiores
C	C1	Protección Discrecional	Control de acceso básico por usuarios
C	C2	Protección de Acceso Controlado	Auditoría de accesos; aislamiento de recursos
B	B1	Protección de Seguridad Etiquetada	Etiquetas de sensibilidad en objetos; MAC básico
B	B2	Protección Estructurada	Modelo formal de política; análisis de canales encubiertos
B	B3	Dominios de Seguridad	Monitor de referencia; recuperación segura
A	A1	Diseño Verificado	Verificación formal matemática del diseño

Glosario: - **MAC (Mandatory Access Control):** Control de acceso obligatorio donde el sistema impone las restricciones, no los usuarios. -

Canales encubiertos: Métodos no previstos para transmitir información eludiendo controles de seguridad. - **Monitor de referencia:** Componente que media todos los accesos a objetos del sistema.

Relevancia para Debian 13:

Aunque TCSEC fue reemplazado por Common Criteria (ISO/IEC 15408), sus conceptos siguen siendo fundamentales:

Debian implementa DAC (Control de Acceso Discrecional) por defecto
`ls -la /etc/passwd` *# Visualizar permisos (nivel C1)*

Para MAC, Debian soporta SELinux y AppArmor (nivel B1)


```
sudo apt install apparmor apparmor-utils
sudo aa-status # Estado de AppArmor
```

2.2.2. Modelo OSI (Open Systems Interconnection)

Definición: El **modelo OSI** (Interconexión de Sistemas Abiertos) es un marco conceptual creado por la ISO (Organización Internacional de Normalización) que divide las comunicaciones de red en siete capas, cada una con funciones específicas.

Las 7 Capas del Modelo OSI:

Capa	Nombre	Función	Amenazas	Protección en Debian
7	Aplicación	Interfaz con el usuario y aplicaciones	Malware, inyección SQL, XSS	WAF, actualizaciones de software
6	Presentación	Formato y cifrado de datos	Ataques a cifrado débil	TLS/SSL correctamente configurado
5	Sesión	Gestión de conexiones	Secuestro de sesión	Tokens seguros, timeouts
4	Transporte	Entrega confiable de datos (TCP/UDP)	SYN flood, escaneo de puertos	nftables, limitación de conexiones
3	Red	Enrutamiento de paquetes (IP)	IP spoofing, ataques ICMP	Filtrado de paquetes, VPN
2	Enlace de datos	Tramas y direcciones MAC	ARP spoofing, MAC flooding	802.1X, segmentación VLAN
1	Física	Transmisión de bits	Interceptación física, jamming	Seguridad física, blindaje

Glosario de términos: - **SQL Injection:** Técnica que inserta código SQL malicioso en consultas de bases de datos. - **XSS (Cross-Site Scripting):** Ataque que inyecta scripts maliciosos en páginas web. - **SYN flood:** Ataque de denegación de servicio que satura con peticiones de conexión. - **IP spoofing:** Falsificación de la dirección IP de origen. - **ARP spoofing:** Ataque que asocia la MAC del atacante con la IP de otro dispositivo.

Protección por capas en Debian 13:

Capa 7 - Aplicación: Actualizar software

```
sudo apt update && sudo apt upgrade
```

Capa 4 - Transporte: Configurar firewall nftables

```
sudo nft add rule inet filter input tcp dport 22 accept
```

```
sudo nft add rule inet filter input tcp dport 80 accept
sudo nft add rule inet filter input drop
```

```
# Capa 3 - Red: Deshabilitar respuestas ICMP redirect
echo "net.ipv4.conf.all.accept_redirects = 0" | sudo tee -a /etc/sysctl.conf
sudo sysctl -p
```

```
# Capa 2 - Enlace: Verificar interfaces de red
ip link show
```

2.3. Clasificación de Información

La clasificación de información es el proceso de categorizar datos según su sensibilidad y el impacto que tendría su divulgación no autorizada.

2.3.1. Niveles de Clasificación

Esquema de cuatro niveles:

Nivel	Etiqueta	Descripción	Ejemplos	Controles en Debian
1	Público	Información de libre acceso	Sitio web corporativo, folletos	Permisos 644, acceso anónimo permitido
2	Interno	Solo para personal interno	Procedimientos operativos, organigramas	Permisos 640, autenticación requerida
3	Confidencial	Acceso limitado por necesidad	Datos financieros, contratos	Permisos 600, cifrado en reposo
4	Restringido	Máxima protección	Claves criptográficas, datos personales sensibles	Permisos 400, cifrado + auditoría

2.3.2. Implementación de clasificación en Debian 13

Estructura de directorios por clasificación:

```
# Crear estructura de directorios clasificados
sudo mkdir -p /datos/{publico,interno,confidencial,restringido}
```

```
# Crear grupos para cada nivel
sudo groupadd nivel_interno
sudo groupadd nivel_confidencial
sudo groupadd nivel_restringido
```

Asignar permisos por nivel

```

sudo chmod 755 /datos/publico      # Lectura para todos
sudo chmod 750 /datos/interno      # Lectura para grupo
sudo chmod 700 /datos/confidencial # Solo propietario
sudo chmod 700 /datos/restringido  # Solo propietario

```

Asignar grupos

```

sudo chgrp nivel_interno /datos/interno
sudo chgrp nivel_confidencial /datos/confidencial
sudo chgrp nivel_restringido /datos/restringido

```

*Etiquetado con atributos extendidos:**# Instalar herramientas de atributos extendidos*

```
sudo apt install attr
```

Etiquetar archivos con clasificación

```

sudo setfattr -n user.clasificacion -v "CONFIDENCIAL"
/datos/confidencial/documento.pdf

```

Verificar etiqueta

```
getfattr -n user.clasificacion /datos/confidencial/documento.pdf
```

2.3.3. Matriz de Controles por Clasificación

Control	Público	Interno	Confidencial	Restringido
Autenticación	No	Sí	Sí + MFA	Sí + MFA
Cifrado en tránsito	Opcional	Recomendado	Obligatorio	Obligatorio
Cifrado en reposo	No	No	Recomendado	Obligatorio
Registro de accesos	No	Básico	Detallado	Completo
Respaldo	Semanal	Diario	Diario cifrado	Tiempo real cifrado
Retención	1 año	3 años	5 años	7 años

MFA (Multi-Factor Authentication): Autenticación que requiere dos o más factores de verificación (algo que sabes, algo que tienes, algo que eres).

2.4. Controles de Seguridad

Los controles de seguridad son salvaguardas o contramedidas implementadas para proteger la confidencialidad, integridad y disponibilidad de la información.

2.4.1. Controles Preventivos

Definición: Los **controles preventivos** son medidas diseñadas para evitar que ocurran incidentes de seguridad antes de que sucedan.

Ejemplos en Debian 13:

Control	Descripción	Implementación
Firewall	Bloquea tráfico no autorizado	nftables, ufw
Cifrado de disco	Protege datos si el equipo es robado	LUKS
Política de contraseñas	Previene contraseñas débiles	PAM + pwquality
Actualizaciones automáticas	Corrige vulnerabilidades conocidas	unattended-upgrades
Hardening del sistema	Reduce superficie de ataque	CIS Benchmarks

Configuración de controles preventivos:

1. Firewall con UFW (Uncomplicated Firewall)

```
sudo apt install ufw
sudo ufw default deny incoming
sudo ufw default allow outgoing
sudo ufw allow ssh
sudo ufw enable
```

2. Actualizaciones automáticas de seguridad

```
sudo apt install unattended-upgrades
sudo dpkg-reconfigure -plow unattended-upgrades
```

3. Política de contraseñas robusta

```
sudo nano /etc/security/pwquality.conf
```

Contenido recomendado:

```
# minlen = 12
# dcredit = -1
# ucredit = -1
# lcredit = -1
# ocredit = -1
```

4. Deshabilitar servicios innecesarios

```
sudo systemctl disable bluetooth.service
sudo systemctl disable cups.service
```

2.4.2. Controles Detectivos

Definición: Los **controles detectivos** son mecanismos que identifican y alertan sobre incidentes de seguridad mientras ocurren o poco después.

Ejemplos en Debian 13:

Control	Descripción	Herramienta
IDS/IPS	Detecta intrusiones en red	Suricata, Snort
Análisis de logs	Identifica patrones sospechosos	rsyslog, journalctl
Monitoreo de integridad	Detecta cambios no autorizados	AIDE, Tripwire
Auditoría del sistema	Registra eventos de seguridad	auditd
Escaneo de vulnerabilidades	Identifica debilidades	OpenVAS, Lynis

IDS (Intrusion Detection System): Sistema que monitorea el tráfico de red buscando actividades sospechosas. **IPS (Intrusion Prevention System):** IDS con capacidad de bloquear amenazas automáticamente.

Configuración de controles detectivos:

1. Sistema de auditoría auditd

```
sudo apt install auditd audispd-plugins
sudo systemctl enable auditd
```

Regla: Monitorear cambios en /etc/passwd

```
sudo auditctl -w /etc/passwd -p wa -k passwd_changes
```

Verificar logs de auditoría

```
sudo ausearch -k passwd_changes
```

2. Monitoreo de integridad con AIDE

```
sudo apt install aide
sudo aideinit
sudo cp /var/lib/aide/aide.db.new /var/lib/aide/aide.db
```

Verificar integridad

```
sudo aide --check
```

3. Análisis de logs con journalctl

Ver intentos de autenticación fallidos

```
sudo journalctl -u ssh --since "1 hour ago" | grep "Failed"
```

4. Escaneo de seguridad con Lynis

```
sudo apt install lynis
sudo lynis audit system
```

2.4.3. Controles Correctivos

Definición: Los **controles correctivos** son acciones que se ejecutan después de un incidente para restaurar sistemas, minimizar daños y prevenir recurrencias.

Ejemplos en Debian 13:

Control	Descripción	Implementación
Respaldos	Restauración de datos perdidos	rsync, Bacula, BorgBackup
Plan de recuperación	Procedimientos post-incidente	Documentación + scripts
Parches de emergencia	Corrección urgente de vulnerabilidades	apt upgrade específico
Aislamiento de red	Contención de amenazas	nftables, desconexión
Forense digital	Análisis post-incidente	Sleuth Kit, Autopsy

*Configuración de controles correctivos:**# 1. Sistema de respaldos con BorgBackup*

```
sudo apt install borgbackup
```

Inicializar repositorio de respaldos

```
borg init --encryption=repokey /ruta/respaldos
```

Crear respaldo

```
borg create /ruta/respaldos::backup-{now} /datos/confidencial
```

Restaurar respaldo

```
borg extract /ruta/respaldos::backup-2026-02-13
```

2. Script de aislamiento de emergencia

```
cat << 'EOF' | sudo tee /usr/local/bin/aislamiento_emergencia.sh
```

```
#!/bin/bash
```

```
# Script de aislamiento de red de emergencia
```

```
echo "ALERTA: Iniciando aislamiento de red..."
```

```
nft flush ruleset
```

```
nft add table inet emergency
```

```
nft add chain inet emergency input { type filter hook input priority 0 \; policy drop \; }
```

```
nft add chain inet emergency output { type filter hook output priority 0 \; policy drop \; }
```

```
nft add rule inet emergency input iif lo accept
```

```
nft add rule inet emergency output oif lo accept
```

```
echo "Sistema aislado. Solo conexiones locales permitidas."
```

```
EOF
```

```
sudo chmod 700 /usr/local/bin/aislamiento_emergencia.sh
```

3. Procedimiento de parche de emergencia

```
sudo apt update
```

```
sudo apt install --only-upgrade paquete_vulnerable
```

```
sudo systemctl restart servicio_afectado
```

2.4.4. Comparativa de Controles

Aspecto	Preventivo	Detectivo	Correctivo
Momento	Antes del incidente	Durante/después	Después del incidente
Objetivo	Evitar amenazas	Identificar amenazas	Recuperarse de amenazas
Costo	Medio-Alto inicial	Medio continuo	Variable (según daño)
Ejemplo físico	Cerradura	Alarma	Seguro de robo
Ejemplo lógico	Firewall	IDS	Respaldo

2.5. Marco Normativo

El marco normativo proporciona las directrices, estándares y mejores prácticas reconocidas internacionalmente para implementar seguridad informática.

2.5.1. ISO/IEC 27001

Definición: ISO/IEC 27001 es un estándar internacional publicado por la Organización Internacional de Normalización (ISO) y la Comisión Electrotécnica Internacional (IEC) que especifica los requisitos para establecer, implementar, mantener y mejorar continuamente un Sistema de Gestión de Seguridad de la Información (SGSI).

Estructura de ISO 27001:2022

Cláusula	Título	Descripción
4	Contexto de la organización	Entender la organización y sus necesidades
5	Liderazgo	Compromiso de la alta dirección
6	Planificación	Gestión de riesgos y oportunidades
7	Apoyo	Recursos, competencia, comunicación
8	Operación	Implementación de controles
9	Evaluación del desempeño	Monitoreo y auditorías internas
10	Mejora	Acciones correctivas y mejora continua

Anexo A - Controles de Seguridad (93 controles en 4 temas):

Tema	Controles	Ejemplos
Organizativos	37	Políticas, roles, gestión de activos
Personas	8	Selección, formación, terminación
Físicos	14	Perímetros, equipos, servicios

Tema	Controles	Ejemplos
Tecnológicos	34	Acceso, criptografía, redes

Aplicación a Debian 13:

Control A.8.9 - Gestión de la configuración

Documentar configuración del sistema

sudo apt install debsums

debsums --all --changed # Verificar integridad de paquetes

Control A.8.15 - Registro de actividades

Configurar logging centralizado

sudo apt install rsyslog

sudo systemctl enable rsyslog

Control A.8.24 - Uso de criptografía

Verificar soporte de cifrado

openssl version

cat /proc/crypto | grep -E "^name"

2.5.2. CIS Benchmarks para Debian

Definición: Los **CIS Benchmarks** (Puntos de Referencia del Centro para la Seguridad de Internet) son guías de configuración segura desarrolladas por consenso de expertos en seguridad. Proporcionan recomendaciones específicas y detalladas para fortificar sistemas operativos.

Niveles de CIS Benchmarks:

Nivel	Descripción	Uso recomendado
Nivel 1	Configuraciones básicas de seguridad con mínimo impacto operativo	Todos los sistemas
Nivel 2	Configuraciones avanzadas que pueden afectar funcionalidad	Sistemas de alta seguridad

Categorías del CIS Benchmark para Debian 12/13:

Sección	Área	Ejemplos de controles
1	Configuración inicial	Particiones, actualizaciones, bootloader
2	Servicios	Deshabilitar servicios innecesarios
3	Configuración de red	Parámetros de kernel, firewall
4	Auditoría y logging	auditd, rsyslog, journald

Sección	Área	Ejemplos de controles
5	Acceso y autenticación	SSH, PAM, sudo
6	Mantenimiento del sistema	Permisos de archivos, integridad

Implementación de controles CIS en Debian 13:

=== SECCIÓN 1: Configuración Inicial ===

1.1.1 - Verificar partición separada para /tmp
mount | grep " /tmp "

1.4.1 - Proteger bootloader GRUB con contraseña

sudo grub-mkpasswd-pbkdf2

Agregar hash a /etc/grub.d/40_custom

=== SECCIÓN 2: Servicios ===

2.1.1 - Deshabilitar servicios innecesarios

sudo systemctl --now disable avahi-daemon

sudo systemctl --now disable cups

=== SECCIÓN 3: Configuración de Red ===

3.1.1 - Deshabilitar IPv6 si no se usa

echo "net.ipv6.conf.all.disable_ipv6 = 1" | sudo tee -a /etc/sysctl.d/99-sysctl.conf

sudo sysctl -p /etc/sysctl.d/99-sysctl.conf

3.2.1 - Deshabilitar reenvío de paquetes

echo "net.ipv4.ip_forward = 0" | sudo tee -a /etc/sysctl.d/99-sysctl.conf

=== SECCIÓN 4: Auditoría ===

4.1.1 - Instalar y habilitar auditd

sudo apt install auditd

sudo systemctl enable auditd

=== SECCIÓN 5: Acceso y Autenticación ===

5.2.1 - Configurar SSH seguro

sudo nano /etc/ssh/sshd_config

Configuraciones recomendadas:

PermitRootLogin no

MaxAuthTries 4

ClientAliveInterval 300

ClientAliveCountMax 0

AllowUsers usuario_autorizado

5.3.1 - Configurar política de contraseñas

```
sudo apt install libpam-pwquality
```

=== SECCIÓN 6: Mantenimiento ===

6.1.1 - Verificar permisos de archivos críticos

```
sudo chmod 644 /etc/passwd
```

```
sudo chmod 600 /etc/shadow
```

```
sudo chmod 644 /etc/group
```

```
sudo chmod 600 /etc/gshadow
```

Herramientas de auditoría CIS:

Lynis - Auditoría de seguridad open source

```
sudo apt install lynis
```

```
sudo lynis audit system --quick
```

Ver resultados

```
sudo cat /var/log/lynis-report.dat | grep warning
```

```
sudo cat /var/log/lynis-report.dat | grep suggestion
```

2.5.3. Comparativa de Marcos Normativos

Aspecto	ISO 27001	CIS Benchmarks
Tipo	Estándar internacional certificable	Guía técnica de mejores prácticas
Enfoque	Gestión de seguridad (qué hacer)	Configuración técnica (cómo hacerlo)
Alcance	Organización completa	Sistema operativo específico
Actualización	Cada 5-7 años	Continua (con cada versión de SO)
Certificación	Sí (auditoría externa)	No (autoevaluación)
Costo	Alto (certificación)	Gratuito
Complementariedad	Marco de gestión	Implementación técnica

Recomendación: ISO 27001 y CIS Benchmarks son complementarios. ISO 27001 define QUÉ controles implementar; CIS Benchmarks especifica CÓMO implementarlos técnicamente en Debian.

Checklist Nivel 1: Tipos y Clasificación de Seguridad

Este checklist permite verificar la comprensión e implementación de los conceptos del Nivel 1. Marque cada elemento completado.

Seguridad Física

- ☐ Verificar que el servidor/equipo está en ubicación segura con acceso restringido
- ☐ Comprobar existencia de sistema de alimentación ininterrumpida (UPS)

3. ☐ Instalar sensores de temperatura: `sudo apt install lm-sensors && sensors-detect`
4. ☐ Documentar ubicación física y controles de acceso al hardware
5. ☐ Verificar redundancia de almacenamiento (RAID si aplica)

Seguridad Lógica

- ☐ Configurar firewall nftables o UFW:

`sudo apt install ufw`
`sudo ufw enable`
`sudo ufw default deny incoming`
- ☐ Habilitar cifrado de disco LUKS en particiones sensibles
- ☐ Configurar política de contraseñas robusta en `/etc/security/pwquality.conf`
- ☐ Instalar y configurar AppArmor:

`sudo apt install apparmor apparmor-utils`
`sudo aa-status`
- ☐ Deshabilitar inicio de sesión root por SSH

Seguridad Organizativa

- ☐ Documentar política de seguridad de la información
- ☐ Crear procedimiento de gestión de incidentes
- ☐ Definir roles y responsabilidades de seguridad
- ☐ Establecer proceso de gestión de cambios
- ☐ Planificar auditorías de seguridad periódicas

Seguridad Personal

- ☐ Implementar principio de mínimo privilegio en cuentas de usuario
- ☐ Crear usuarios con permisos específicos (no usar root):

`sudo adduser --disabled-password usuario_limitado`
- ☐ Configurar expiración de cuentas temporales: `sudo chage -E YYYY-MM-DD usuario`
- ☐ Documentar procedimiento de baja de usuarios
- ☐ Planificar formación en concienciación de seguridad

Clasificación de Información

- ☐ Crear estructura de directorios por nivel de clasificación:

`sudo mkdir -p /datos/{publico,interno,confidencial,restringido}`
- ☐ Asignar permisos según clasificación (755/750/700/700)

- ☐ Crear grupos para cada nivel de clasificación
- ☐ Implementar etiquetado de archivos con atributos extendidos
- ☐ Documentar matriz de controles por nivel de clasificación

Controles Preventivos

- ☐ Configurar actualizaciones automáticas de seguridad:
`sudo apt install unattended-upgrades`
`sudo dpkg-reconfigure -plow unattended-upgrades`
- ☐ Deshabilitar servicios innecesarios: `sudo systemctl disable servicio`
- ☐ Aplicar hardening básico según CIS Benchmarks Nivel 1
- ☐ Configurar banners de advertencia legal en `/etc/issue` y `/etc/issue.net`
- ☐ Verificar configuración segura de SSH

Controles Detectivos

- ☐ Instalar y configurar auditd:
`sudo apt install auditd`
`sudo systemctl enable auditd`
- ☐ Configurar monitoreo de integridad con AIDE:
`sudo apt install aide`
`sudo aideinit`
- ☐ Configurar análisis de logs con journalctl
- ☐ Ejecutar escaneo de seguridad con Lynis: `sudo lynis audit system`
- ☐ Revisar logs de autenticación periódicamente

Controles Correctivos

1. ☐ Configurar sistema de respaldos automatizado:
`sudo apt install borgbackup`
2. ☐ Documentar procedimiento de restauración de respaldos
3. ☐ Crear script de aislamiento de emergencia
4. ☐ Documentar plan de respuesta a incidentes
5. ☐ Probar restauración de respaldos periódicamente

Marco Normativo

- ☐ Revisar requisitos aplicables de ISO 27001

- ☐ Descargar CIS Benchmark para Debian 12/13 desde [cisecurity.org](https://www.cisecurity.org)
- ☐ Ejecutar auditoría CIS con herramienta automatizada
- ☐ Documentar desviaciones justificadas de los benchmarks
- ☐ Planificar revisiones periódicas de cumplimiento

Verificación Final del Nivel 1

Script de verificación rápida del Nivel 1

```
echo "=== Verificación Nivel 1 - Debian 13 Trixie ==="
```

```
echo -e "\n[1] Estado del firewall:"
sudo ufw status || sudo nft list ruleset | head -10
```

```
echo -e "\n[2] Estado de AppArmor:"
sudo aa-status | head -5
```

```
echo -e "\n[3] Política de contraseñas:"
grep -E "^(minlen|dcredit|ucredit)" /etc/security/pwquality.conf 2>/dev/null || echo
"No configurado"
```

```
echo -e "\n[4] Servicios en escucha:"
ss -tlnp | grep LISTEN
```

```
echo -e "\n[5] Usuarios con shell de login:"
grep -v "nologin\\false" /etc/passwd | cut -d: -f1
```

```
echo -e "\n[6] Estado de auditd:"
sudo systemctl is-active auditd
```

```
echo -e "\n[7] Última actualización del sistema:"
stat /var/cache/apt/pkgcache.bin | grep Modify
```

```
echo -e "\n=== Fin de verificación ==="
```

Resumen del Nivel 1

Este nivel ha cubierto los fundamentos conceptuales de la seguridad informática aplicados a Debian 13 “Trixie”:

Tema	Conceptos Clave
Tipos de seguridad	Física, lógica, organizativa, personal
Modelos multinivel	TCSEC (D a A1), OSI (7 capas)
Clasificación	Público, interno, confidencial, restringido
Controles	Preventivos, detectivos, correctivos
Marco normativo	ISO 27001 (gestión), CIS

Tema	Conceptos Clave
	Benchmarks (técnico)

Próximos pasos

Con los fundamentos del Nivel 1 comprendidos, el siguiente nivel abordará la **implementación práctica de controles de acceso y autenticación** en Debian 13, incluyendo:

1. Configuración avanzada de PAM
2. Gestión de usuarios y grupos
3. Implementación de sudo y políticas de privilegios
4. Autenticación multifactor (MFA)

Glosario Completo

Término	Definición
AppArmor	Sistema de control de acceso obligatorio (MAC) para Linux
ARP spoofing	Ataque que asocia la MAC del atacante con la IP de otro dispositivo
Auditoría	Revisión sistemática del cumplimiento de políticas y controles
Autenticación	Proceso de verificar la identidad de un usuario o sistema
Autorización	Determinar qué acciones puede realizar un usuario autenticado
Benchmark	Punto de referencia o estándar de comparación
Canal encubierto	Método no previsto para transmitir información eludiendo controles
Cifrado	Proceso de convertir información legible en código ilegible
CIS	Center for Internet Security (Centro para la Seguridad de Internet)
Control correctivo	Acción que restaura sistemas después de un incidente
Control detectivo	Mecanismo que identifica incidentes de seguridad
Control preventivo	Medida que evita incidentes antes de que ocurran
DAC	Control de Acceso Discrecional (permisos gestionados por usuarios)
Firewall	Sistema que filtra el tráfico de red según reglas predefinidas
Hardening	Proceso de fortificar un sistema reduciendo su superficie de ataque
IDS	Sistema de Detección de Intrusiones

Término	Definición
IPS	Sistema de Prevención de Intrusiones
ISO	Organización Internacional de Normalización
LUKS	Estándar de cifrado de disco en Linux
MAC	Control de Acceso Obligatorio (permisos impuestos por el sistema)
Malware	Software malicioso (virus, ransomware, troyanos, etc.)
MFA	Autenticación Multifactor
Monitor de referencia	Componente que media todos los accesos a objetos del sistema
OSI	Modelo de Interconexión de Sistemas Abiertos
PAM	Módulos de Autenticación Conectables
Phishing	Técnica de engaño para obtener información confidencial
RAID	Conjunto Redundante de Discos Independientes
SGSI	Sistema de Gestión de Seguridad de la Información
SQL Injection	Técnica que inserta código SQL malicioso
SYN flood	Ataque de denegación de servicio con peticiones de conexión
TCSEC	Criterios de Evaluación de Sistemas Informáticos de Confianza
UPS	Sistema de Alimentación Ininterrumpida
XSS	Cross-Site Scripting (inyección de scripts maliciosos)

Nivel 2: Seguridad Física y Arranque Confiable

Manual de Seguridad Informática para Debian 13 "Trixie"

Introducción

La seguridad informática no comienza con el software: comienza con el acceso físico al hardware. Un atacante con acceso físico a un equipo puede eludir la mayoría de las protecciones lógicas. Este nivel del manual aborda las medidas de seguridad física y los mecanismos de arranque confiable necesarios para proteger sistemas Debian 13 "Trixie".

3.1. Protección de Instalaciones (CCTV, Cerraduras, Alarmas)

Conceptos Fundamentales

La primera línea de defensa en seguridad informática es la **seguridad perimetral**: las barreras físicas que impiden el acceso no autorizado a las instalaciones donde se encuentran los equipos informáticos.

Componentes Clave

CCTV (Circuito Cerrado de Televisión)

El **CCTV** es un sistema de videovigilancia que transmite señales a un conjunto específico de monitores, a diferencia de la televisión convencional que transmite abiertamente. Sus características esenciales incluyen:

11. **Cobertura 24/7**: Grabación continua de áreas críticas
12. **Almacenamiento seguro**: Los registros deben guardarse en ubicaciones separadas del área vigilada
13. **Retención mínima**: Se recomienda conservar grabaciones durante al menos 30 días
14. **Resolución adecuada**: Mínimo 1080p para identificación facial

Cerraduras de Alta Seguridad

Tipo	Descripción	Nivel de Seguridad
Mecánicas de alta seguridad	Cilindros con protección anti-ganzúa	Medio
Electrónicas con código	Requieren PIN numérico	Medio-Alto

Tipo	Descripción	Nivel de Seguridad
Electromagnéticas	Controladas por sistema centralizado	Alto
Biométricas	Requieren huella dactilar o reconocimiento facial	Muy Alto

Sistemas de Alarma

Los sistemas de alarma deben incluir:

- **Sensores de movimiento PIR** (Infrarrojo Pasivo)
- **Sensores de apertura** en puertas y ventanas
- **Sirenas audibles** y alertas silenciosas
- **Conexión con central de monitoreo**
- **Respaldo de energía** (UPS o baterías)

Recomendaciones para Centros de Datos

Zona de Servidores:

- Acceso restringido únicamente a personal autorizado
- Registro de entrada/salida obligatorio
- Cámaras en todos los puntos de acceso
- Sensores de temperatura y humedad
- Detección y extinción de incendios

3.2. Control Físico de Acceso (Tarjetas RFID, Biometría)

Definiciones Técnicas

RFID (Radio Frequency Identification): Tecnología de identificación por radiofrecuencia que permite la lectura de información almacenada en etiquetas o tarjetas mediante ondas de radio, sin necesidad de contacto físico ni línea de visión directa.

Biometría: Método de identificación basado en características físicas únicas de una persona, como huellas dactilares, iris, rostro o voz.

Sistemas de Control de Acceso

Tarjetas RFID

Las tarjetas RFID funcionan mediante un chip que almacena un identificador único. Cuando se acerca al lector, este emite una señal de radio que energiza el chip y lee su información.

Tipos de tarjetas RFID:

Frecuencia	Rango	Uso Común	Seguridad
125 kHz (LF)	< 10 cm	Acceso básico	Baja (clonables)
13.56 MHz (HF)	< 1 m	Tarjetas inteligentes	Media-Alta
860-960 MHz (UHF)	Hasta 12 m	Logística	Variable

Recomendación: Utilizar tarjetas HF con cifrado (como MIFARE DESFire EV3) para control de acceso a áreas con equipos informáticos.

Sistemas Biométricos

Métodos Biométricos por Nivel de Seguridad:

Muy Alto	Reconocimiento de iris Reconocimiento vascular
Alto	Huella dactilar multiespectral Reconocimiento facial 3D
Medio	Huella dactilar capacitiva Reconocimiento facial 2D
Básico	Reconocimiento de voz Geometría de la mano

Autenticación Multifactor Física

La mejor práctica es combinar múltiples factores:

6. **Algo que tienes:** Tarjeta RFID o token físico
7. **Algo que sabes:** PIN o contraseña
8. **Algo que eres:** Característica biométrica

Ejemplo de configuración recomendada:

Acceso a sala de servidores:

Factor 1: Tarjeta RFID cifrada

Factor 2: PIN de 6 dígitos

Factor 3: Huella dactilar

→ Los tres factores deben validarse en menos de 30 segundos

3.3. Seguridad del Hardware y Firmware

Definición de Firmware

Firmware: Software permanente programado en la memoria de solo lectura del hardware. Es el código de bajo nivel que controla el funcionamiento básico de un dispositivo, actuando como intermediario entre el hardware y el sistema operativo.

3.3.1. BIOS/UEFI: Password, Secure Boot y TPM 2.0

Definiciones Fundamentales

BIOS (Basic Input/Output System): Sistema básico de entrada/salida. Es el firmware tradicional que inicializa el hardware del computador y carga el sistema operativo. Funciona en modo de 16 bits y tiene limitaciones significativas en sistemas modernos.

UEFI (Unified Extensible Firmware Interface): Interfaz de firmware extensible unificada. Es el sucesor moderno del BIOS que ofrece una interfaz gráfica, soporte para discos grandes (más de 2 TB), arranque más rápido y características de seguridad avanzadas como Secure Boot.

Secure Boot: Característica de seguridad de UEFI que verifica que cada componente de software cargado durante el arranque esté firmado digitalmente por un editor de confianza. Esto previene la ejecución de código malicioso antes de que el sistema operativo tome control.

TPM (Trusted Platform Module): Módulo de plataforma confiable. Es un chip criptográfico dedicado, soldado a la placa base, que proporciona funciones de seguridad basadas en hardware como generación de claves, cifrado y medición de integridad del sistema.

Configuración de Contraseña UEFI

La contraseña de UEFI/BIOS previene cambios no autorizados en la configuración del firmware:

- **Contraseña de administrador:** Protege el acceso a la configuración UEFI
- **Contraseña de usuario:** Requerida para arrancar el sistema
- **Contraseña de disco duro:** Cifra el acceso al almacenamiento (ATA Security)

Acceso a la configuración UEFI en Debian:

Reiniciar y acceder a UEFI (generalmente F2, F12, DEL o ESC durante el arranque)

`sudo systemctl reboot --firmware-setup`

Habilitación de Secure Boot en Debian 13

Debian 13 "Trixie" soporta Secure Boot de forma nativa. Para verificar y gestionar su estado:

```
# Verificar si Secure Boot está activo
mokutil --sb-state
```

```
# Salida esperada si está habilitado:
# SecureBoot enabled
```

```
# Verificar el modo de arranque (UEFI o BIOS)
[ -d /sys/firmware/efi ] && echo "UEFI" || echo "BIOS/Legacy"
```

```
# Listar las claves de Secure Boot enrolladas
mokutil --list-enrolled
```

Gestión de Secure Boot con sbctl:

sbctl es una herramienta para gestionar Secure Boot con claves propias:

```
# Instalar sbctl
sudo apt update
sudo apt install sbctl
```

```
# Verificar estado de Secure Boot
sudo sbctl status
```

```
# Crear claves propias (requiere Secure Boot en modo Setup)
sudo sbctl create-keys
```

```
# Enrollar claves en el firmware
sudo sbctl enroll-keys
```

```
# Firmar el kernel y bootloader
sudo sbctl sign -s /boot/efi/EFI/debian/shimx64.efi
sudo sbctl sign -s /boot/efi/EFI/debian/grubx64.efi
sudo sbctl sign -s /boot/vmlinuz-$(uname -r)
```

```
# Verificar archivos firmados
sudo sbctl verify
```

Configuración y Uso de TPM 2.0

El TPM 2.0 almacena mediciones criptográficas del proceso de arranque en registros llamados **PCRs (Platform Configuration Registers)**:

```
# Instalar herramientas de TPM
sudo apt install tpm2-tools
```

Verificar presencia del TPM

`sudo tpm2_getcap properties-fixed | grep -i "TPM2_PT_FAMILY_INDICATOR"`

Verificar que el TPM está operativo

`sudo systemctl status tpm2-abrmd`

Mostrar información del TPM

`sudo tpm2_getcap properties-variable`

Analizar los PCRs (Registros de Configuración de Plataforma)

`sudo systemd-analyze pcrs`

Significado de los PCRs principales:

PCR	Contenido Medido
PCR 0	Código del firmware (UEFI)
PCR 1	Configuración del firmware
PCR 2	Código de terceros (Option ROMs)
PCR 4	Gestor de arranque (bootloader)
PCR 5	Configuración del gestor de arranque
PCR 7	Estado de Secure Boot
PCR 8-9	Línea de comandos del kernel
PCR 11	Mediciones de systemd-stub

Ver valores actuales de todos los PCRs

`sudo tpm2_pcrread`

Ver PCRs específicos (0, 4 y 7)

`sudo tpm2_pcrread sha256:0,4,7`

Usar systemd para análisis detallado de PCRs

`sudo systemd-analyze pcrs`

3.3.2. Control de Puertos USB (USBGuard) y BadUSB

Definición de BadUSB

BadUSB: Ataque que explota vulnerabilidades en el firmware de dispositivos USB. Un dispositivo aparentemente inofensivo (como una memoria USB) puede reprogramarse para actuar como un teclado que ejecuta comandos maliciosos automáticamente, eludiendo antivirus y otras protecciones de software.

Ejemplos de ataques BadUSB: - Rubber Ducky: Dispositivo que emula un teclado y escribe comandos predefinidos - Cable O.MG: Cable de carga con

hardware malicioso oculto - USBNinja: Dispositivos que combinan funcionalidad legítima con payloads maliciosos

Instalación y Configuración de USBGuard

USBGuard es un framework de autorización de dispositivos USB que implementa listas blancas/negras:

Instalar USBGuard

```
sudo apt update  
sudo apt install usbguard
```

Habilitar el servicio

```
sudo systemctl enable usbguard  
sudo systemctl start usbguard
```

Verificar estado

```
sudo systemctl status usbguard
```

Generación de política inicial:

Generar política basada en dispositivos actualmente conectados

IMPORTANTE: Conectar solo dispositivos de confianza antes de ejecutar

```
sudo usbguard generate-policy > /etc/usbguard/rules.conf
```

Reiniciar el servicio para aplicar la política

```
sudo systemctl restart usbguard
```

Gestión de dispositivos USB:

Listar dispositivos USB actuales

```
sudo usbguard list-devices
```

Ejemplo de salida:

```
# 1: allow id 1d6b:0002 serial "0000:00:14.0" name "xHCI Host Controller"
```

```
# 5: block id 0951:1666 serial "E0D55EA573E5B0A159A90256" name  
"DataTraveler 3.0"
```

Permitir un dispositivo bloqueado (temporal, hasta reinicio)

```
sudo usbguard allow-device 5
```

Permitir un dispositivo permanentemente

```
sudo usbguard allow-device 5 --permanent
```

Bloquear un dispositivo específico

```
sudo usbguard block-device 5
```

Rechazar (desconectar eléctricamente) un dispositivo

```
sudo usbguard reject-device 5
```

Archivo de reglas (/etc/usbguard/rules.conf):

```
# Sintaxis básica de reglas USBGuard
# allow|block|reject [condiciones]

# Permitir todos los hubs USB del sistema
allow with-interface equals { 09:*:* }

# Permitir teclados y ratones HID
allow with-interface one-of { 03:00:01 03:01:01 03:01:02 }

# Permitir un dispositivo específico por ID de fabricante/producto
allow id 0781:5567 # SanDisk Cruzer Blade

# Bloquear dispositivos que se presentan como múltiples interfaces
block with-interface count-of > 2

# Bloquear dispositivos que emulan teclados excepto los permitidos
block with-interface equals { 03:01:01 }

# Política por defecto: bloquear todo lo no especificado
block
```

Configuración avanzada (/etc/usbguard/usbguard-daemon.conf):

```
# Ver configuración actual
sudo cat /etc/usbguard/usbguard-daemon.conf

# Opciones importantes:
# RuleFile=/etc/usbguard/rules.conf
# ImplicitPolicyTarget=block # Acción por defecto
# PresentDevicePolicy=apply-policy
# InsertedDevicePolicy=apply-policy
# AuthorizedDefault=none # Ningún dispositivo autorizado por defecto
# IPCAllowedUsers=root
# IPCAllowedGroups=wheel
```

Monitoreo de eventos USB:

```
# Ver log de eventos USB en tiempo real
sudo journalctl -f -u usbguard

# Auditar dispositivos conectados recientemente
sudo usbguard list-rules

# Exportar reglas actuales
sudo usbguard export-rules > reglas_backup.conf
```

3.3.3. Protección contra Manipulación de Arranque (Evil Maid)

Definición de Evil Maid Attack

Evil Maid Attack (Ataque de la Criada Malvada): Ataque que requiere acceso físico temporal al equipo objetivo. El atacante modifica el gestor de arranque o instala un keylogger de hardware para capturar contraseñas de cifrado de disco. El nombre proviene del escenario donde un atacante (como personal de limpieza de hotel) tiene acceso breve al equipo desatendido.

Escenario típico: 1. El atacante accede físicamente al equipo apagado 2. Arranca desde un medio externo 3. Modifica GRUB o instala malware en el sector de arranque 4. El usuario arranca normalmente e ingresa su contraseña 5. El malware captura la contraseña y la almacena o transmite

Medidas de Protección

1. Secure Boot (Primera línea de defensa):

Ya configurado en la sección anterior, previene la carga de bootloaders no firmados.

2. Cifrado completo de disco con LUKS:

Verificar que el disco está cifrado con LUKS
`sudo cryptsetup status /dev/mapper/debian-root`

Ver información de la partición cifrada
`sudo cryptsetup luksDump /dev/sda3` *# Ajustar partición según sistema*

3. Medición de arranque con TPM:

Vincular el descifrado de LUKS al estado del TPM:

Instalar systemd-cryptenroll para gestión de LUKS con TPM
`sudo apt install systemd-cryptsetup`

Vincular LUKS al TPM (PCRs 0,4,7)
`sudo systemd-cryptenroll --tpm2-device=auto --tpm2-pcrs=0+4+7 /dev/sda3`

Esto permite que el disco solo se descifre si:
- El firmware no ha sido modificado (PCR 0)
- El bootloader no ha sido modificado (PCR 4)
- Secure Boot mantiene su configuración (PCR 7)

4. Verificación de integridad del bootloader:

Verificar firma del bootloader GRUB
`sudo sbctl verify /boot/efi/EFI/debian/grubx64.efi`

Verificar integridad de archivos críticos de arranque


```
sha256sum /boot/vmlinuz-$(uname -r) > /root/kernel_hash.txt
sha256sum /boot/initrd.img-$(uname -r) >> /root/kernel_hash.txt
sha256sum /boot/efi/EFI/debian/grubx64.efi >> /root/kernel_hash.txt
```

Comparar en cada arranque (puede automatizarse)

```
sha256sum -c /root/kernel_hash.txt
```

5. Sellos de seguridad físicos:

Aplicar sellos de seguridad **tamper-evident** en: - Tornillos de la carcasa del equipo - Puertos USB no utilizados - Bahías de disco duro

6. Contraseña de GRUB:

Generar hash de contraseña para GRUB

```
grub-mkpasswd-pbkdf2
```

Agregar a /etc/grub.d/40_custom:

```
cat << 'EOF' | sudo tee -a /etc/grub.d/40_custom
```

```
set superusers="admin"
```

```
password_pbkdf2 admin grub.pbkdf2.sha512.10000.HASH_GENERADO
```

```
EOF
```

Actualizar GRUB

```
sudo update-grub
```

3.4. Cableado Estructurado Seguro

Principios de Seguridad en Cableado

El cableado de red representa un punto vulnerable frecuentemente ignorado. Un atacante con acceso al cableado puede interceptar tráfico o inyectar datos maliciosos.

Medidas de Protección

Clasificación de Zonas de Cableado

Zona de Seguridad Alta (Restringida):

- └─ Cableado de backbone entre centros de datos
- └─ Conexiones a servidores críticos
- └─ Enlaces de fibra óptica principales
- └─ Cableado de sistemas de seguridad

Zona de Seguridad Media (Controlada):

- └─ Cableado horizontal a estaciones de trabajo
- └─ Conexiones a switches de acceso
- └─ Cableado de VoIP

Zona de Seguridad Baja (General):

- └─ Puntos de red en áreas públicas
- └─ Conexiones para invitados
- └─ Cableado temporal

Recomendaciones Técnicas

Aspecto	Recomendación
Tipo de cable	Cat6A o superior para nuevas instalaciones
Fibra óptica	Preferida para enlaces entre edificios (inmune a interferencias)
Bandejas y canaletas	Cerradas con tapa y cerradura
Patch panels	En armarios con llave
Etiquetado	Sistema estandarizado (TIA-606)
Documentación	Planos actualizados y acceso restringido

Protección contra Interceptación

1. Detección de intrusiones en el cableado:

Monitorear cambios en la topología de red

```
sudo apt install lldpd
sudo systemctl enable lldpd
sudo systemctl start lldpd
```

Ver dispositivos descubiertos

```
lldpcli show neighbors
```

Detectar cambios inesperados en direcciones MAC

```
sudo apt install arpwatc
sudo systemctl enable arpwatc
sudo systemctl start arpwatc
```

Ver alertas de arpwatc

```
sudo journalctl -u arpwatc -f
```

2. Cifrado de capa 2 (MACsec):

MACsec proporciona cifrado punto a punto en Ethernet

Verificar soporte de hardware

```
ethtool -k eth0 | grep macsec
```

Configurar MACsec con NetworkManager

```
nmcli connection add type macsec \
  con-name macsec-link \
  ifname macsec0 \
  connection.autoconnect yes \
  macsec.parent eth0 \
```

```
macsec.mode psk \  
macsec.mka-cak 1234567890abcdef1234567890abcdef \  
macsec.mka-ckn 00112233445566778899aabbccddeeff
```

3.5. Routers/Switches: Tamper-evident y Etiquetado

Definición de Tamper-evident

Tamper-evident (Evidencia de manipulación): Característica de un dispositivo o embalaje que revela visualmente cualquier intento de acceso no autorizado. Incluye sellos, etiquetas holográficas, tornillos especiales o materiales que se rompen o decoloran al ser manipulados.

Seguridad Física de Equipos de Red

Ubicación y Montaje

Requisitos de Instalación Segura:

RACK/GABINETE
- Puertas con cerradura (frontal y trasera) - Paneles laterales fijos con tornillos de seguridad - Anclaje al piso o pared - Ventilación adecuada con filtros - Sensor de apertura de puerta

Sistema de Etiquetado

Estándar TIA-606-C para etiquetado:

Formato de etiqueta: EDIFICIO-PISO-RACK-PUERTO

Ejemplo: ED1-P3-RK02-SW01-P24

Código de colores recomendado:

- Azul: Conexiones de datos estándar
- Naranja: Fibra óptica multimodo
- Amarillo: Fibra óptica monomodo
- Verde: Conexiones de red demilitarizada (DMZ)
- Rojo: Conexiones críticas/seguridad
- Blanco: Conexiones de telefonía

Sellos Tamper-evident

Tipos de sellos recomendados:

Tipo	Aplicación	Características
Etiquetas holográficas	Tornillos de acceso	Se destruyen al despegar
Sellos numerados	Puertas de rack	Secuencia verificable
Cinta de seguridad	Cables y conectores	Deja residuo "VOID"
Esmalte de uñas/pintura	Tornillos	Método económico, visible

Verificación y Auditoría

Crear inventario de equipos de red

```
cat << 'EOF' > /root/inventario_red.sh
#!/bin/bash
echo "=== Inventario de Equipos de Red ==="
echo "Fecha: $(date)"
echo ""
echo "--- Interfaces de Red ---"
ip link show
echo ""
echo "--- Vecinos LLDP ---"
lldpcli show neighbors
echo ""
echo "--- Tabla ARP ---"
ip neigh show
echo ""
echo "--- Rutas ---"
ip route show
EOF
```

```
chmod +x /root/inventario_red.sh
```

Verificar cambios en la configuración de interfaces

```
sudo apt install ethtool
for iface in $(ls /sys/class/net/); do
    echo "=== $iface ==="
    ethtool $iface 2>/dev/null | grep -E "Speed|Duplex|Link detected"
done
```

Registro de verificación de sellos:

Crear registro de verificación de sellos de seguridad

```
cat << 'EOF' > /root/verificacion_sellos.sh
#!/bin/bash
FECHA=$(date +%Y-%m-%d)
HORA=$(date +%H:%M)
ARCHIVO="/var/log/sellos_${FECHA}.log"
```

```
echo "Verificación de Sellos - $FECHA $HORA" >> $ARCHIVO
echo "Inspector: $1" >> $ARCHIVO
echo "---" >> $ARCHIVO

# Lista de equipos a verificar
EQUIPOS=("SW-CORE-01" "SW-ACC-01" "SW-ACC-02" "RT-BORDE-01")

for equipo in "${EQUIPOS[@]"; do
    read -p "Estado del sello $equipo (OK/ROTO/FALTANTE): " estado
    echo "$equipo: $estado" >> $ARCHIVO
done

echo "---" >> $ARCHIVO
echo "Verificación completada" >> $ARCHIVO
EOF

chmod +x /root/verificacion_sellos.sh
```

Checklist Nivel 2: Seguridad Física y Arranque Confiable

Lista de Verificación con Comandos

Ejecute los siguientes comandos para verificar el estado de seguridad de su sistema Debian 13:

A. Verificación de Firmware y Arranque

1. Verificar modo de arranque (UEFI requerido)

```
[ -d /sys/firmware/efi ] && echo "[OK] Sistema arrancado en modo UEFI" || echo "[FALLO] Sistema en modo BIOS Legacy"
```

2. Verificar estado de Secure Boot

```
mokutil --sb-state
```

3. Verificar estado con sbctl

```
sudo sbctl status
```

4. Verificar archivos de arranque firmados

```
sudo sbctl verify
```

5. Verificar presencia y estado del TPM

```
sudo systemctl status tpm2-abrmd
```

6. Analizar PCRs del TPM

```
sudo systemd-analyze pcrs
```

7. Leer valores de PCRs críticos

```
sudo tpm2_pcrread sha256:0,4,7
```

8. Verificar cifrado de disco LUKS

```
sudo cryptsetup status /dev/mapper/*-root 2>/dev/null || echo "[INFO] Verificar  
partición cifrada manualmente"
```

B. Verificación de Control USB (USBGuard)

9. Verificar que USBGuard está instalado y activo

```
sudo systemctl is-active usbguard && echo "[OK] USBGuard activo" || echo  
"[FALLO] USBGuard no activo"
```

10. Verificar política de dispositivos USB

```
sudo usbguard list-rules | head -20
```

11. Listar dispositivos USB actuales y su estado

```
sudo usbguard list-devices
```

12. Verificar configuración de USBGuard

```
grep -E "^(ImplicitPolicyTarget|PresentDevicePolicy)" /etc/usbguard/usbguard-  
daemon.conf
```

C. Verificación de Red y Monitoreo

13. Verificar servicio LLDP para detección de topología

```
sudo systemctl is-active lldpd && echo "[OK] LLDP activo" || echo  
"[ADVERTENCIA] LLDP no instalado"
```

14. Verificar servicio arpwatch para detección de cambios MAC

```
sudo systemctl is-active arpwatch 2>/dev/null && echo "[OK] arpwatch activo" ||  
echo "[ADVERTENCIA] arpwatch no instalado"
```

15. Listar interfaces de red

```
ip link show | grep -E "[0-9]+:" | awk '{print $2}'
```

D. Script Completo de Auditoría

```
#!/bin/bash
```

```
# Script de auditoría de seguridad física - Nivel 2
```

```
# Guardar como: /root/auditoria_nivel2.sh
```

```
echo "===== "  
echo " Auditoría de Seguridad Física - Nivel 2 "  
echo " Debian 13 Trixie "  
echo " Fecha: $(date) "  
echo "===== "  
echo ""
```

Función para mostrar resultado

```
check() {
```

```

    if [ $? -eq 0 ]; then
        echo "[OK] $1"
    else
        echo "[FALLO] $1"
    fi
}

echo "=== A. FIRMWARE Y ARRANQUE ==="
echo ""

# UEFI
echo -n "1. Modo UEFI: "
[ -d /sys/firmware/efi ] && echo "OK" || echo "FALLO (BIOS Legacy)"

# Secure Boot
echo -n "2. Secure Boot: "
mokutil --sb-state 2>/dev/null | grep -q "enabled" && echo "OK (Habilitado)" ||
echo "ADVERTENCIA (Deshabilitado)"

# sbctl status
echo "3. Estado sbctl:"
sudo sbctl status 2>/dev/null || echo " sbctl no instalado"

# TPM
echo -n "4. TPM 2.0: "
sudo systemctl is-active tpm2-abrmd &>/dev/null && echo "OK (Activo)" || echo
"ADVERTENCIA (No activo)"

# PCRs
echo "5. Análisis de PCRs:"
sudo systemd-analyze pcrcs 2>/dev/null | head -15 || echo " No se pueden leer
PCRs"

echo ""
echo "=== B. CONTROL USB ==="
echo ""

# USBGuard
echo -n "6. USBGuard: "
sudo systemctl is-active usbguard &>/dev/null && echo "OK (Activo)" || echo
"FALLO (No activo)"

# Política USB
echo "7. Reglas USBGuard activas:"
sudo usbguard list-rules 2>/dev/null | wc -l | xargs echo " Número de reglas:"

# Dispositivos bloqueados

```

```
echo "8. Dispositivos USB bloqueados:"
sudo usbguard list-devices 2>/dev/null | grep -c "block" | xargs echo " Cantidad:"
```

```
echo ""
echo "=== C. INTEGRIDAD DE ARRANQUE ==="
echo ""
```

Verificar firmas

```
echo "9. Verificación de firmas sbctl:"
sudo sbctl verify 2>/dev/null || echo " sbctl no configurado"
```

LUKS

```
echo "10. Cifrado de disco:"
sudo cryptsetup status $(ls /dev/mapper/ | grep -v control | head -1) 2>/dev/null |
grep -q "LUKS" && echo " OK (LUKS activo)" || echo " ADVERTENCIA (Verificar
manualmente)"
```

```
echo ""
echo "=== D. MONITOREO DE RED ==="
echo ""
```

LLDP

```
echo -n "11. LLDP: "
sudo systemctl is-active lldpd &>/dev/null && echo "OK" || echo "No instalado"
```

arptwatch

```
echo -n "12. arptwatch: "
sudo systemctl is-active arptwatch &>/dev/null && echo "OK" || echo "No
instalado"
```

```
echo ""
echo "===== "
echo " Auditoría completada"
echo "===== "
```

Resumen de Comandos de Verificación Rápida

Verificación	Comando
Modo UEFI	[-d /sys/firmware/efi] && echo UEFI
Secure Boot	mokutil --sb-state
Estado sbctl	sudo sbctl status
Verificar firmas	sudo sbctl verify
Estado TPM	sudo systemctl status tpm2-abrmd
Analizar PCRs	sudo systemd-analyze pcrs

Verificación	Comando
Leer PCRs	<code>sudo tpm2_pcrread sha256:0,4,7</code>
USBGuard activo	<code>sudo systemctl is-active usbguard</code>
Listar USB	<code>sudo usbguard list-devices</code>
Reglas USB	<code>sudo usbguard list-rules</code>
LUKS status	<code>sudo cryptsetup status /dev/mapper/root</code>

Acciones Correctivas

Si alguna verificación falla, consulte la sección correspondiente de este manual:

- **UEFI no detectado** → Reinstalar Debian en modo UEFI
- **Secure Boot deshabilitado** → Sección 3.3.1
- **TPM no funcional** → Verificar en UEFI que esté habilitado
- **USBGuard inactivo** → Sección 3.3.2
- **Firmas no válidas** → Ejecutar `sudo sbctl sign-all`

Referencias

- Debian Security Manual: <https://www.debian.org/doc/manuals/securing-debian-manual/>
- USBGuard Documentation: <https://usbguard.github.io/>
- TPM 2.0 Tools: <https://github.com/tpm2-software/tpm2-tools>
- UEFI Specification: <https://uefi.org/specifications>
- TIA-606 Labeling Standard: <https://www.tiaonline.org/>

Nivel 3: Seguridad Lógica - Sistema Base Debian 13

Introducción

Este nivel aborda la configuración segura del hardware de red y la instalación y endurecimiento de Debian 13 "Trixie" como sistema operativo base. Aquí se establecen los cimientos de seguridad sobre los cuales se construirán las capas superiores de protección.

Glosario de Términos Técnicos

Antes de comenzar, es importante familiarizarse con los conceptos clave que se utilizarán a lo largo de este documento:

Término	Definición
VLAN	<i>Virtual Local Area Network</i> (Red de Área Local Virtual). Tecnología que permite dividir una red física en múltiples redes lógicas aisladas, mejorando la seguridad y el rendimiento al separar el tráfico de diferentes grupos de dispositivos.
MAC Spoofing	Técnica de ataque donde un atacante falsifica la dirección MAC (identificador único del hardware de red) de su dispositivo para hacerse pasar por otro equipo autorizado en la red.
Hardening	Proceso de endurecimiento o fortificación de un sistema mediante la reducción de su superficie de ataque, eliminando servicios innecesarios y aplicando configuraciones de seguridad estrictas.
LUKS2	<i>Linux Unified Key Setup versión 2</i> . Estándar de cifrado de disco completo en Linux que protege los datos almacenados mediante algoritmos criptográficos robustos.
tmpfs	Sistema de archivos temporal que reside completamente en la memoria RAM, ofreciendo alta velocidad y la garantía de que los datos se eliminan al reiniciar.
netinst	<i>Network Installation</i> . Imagen de instalación mínima de Debian que descarga únicamente los paquetes necesarios desde Internet,

Término	Definición
Bootloader	reduciendo la superficie de ataque inicial. Programa que se ejecuta al encender el ordenador, responsable de cargar el sistema operativo. Es el primer software que se ejecuta y, por tanto, un punto crítico de seguridad.
GRUB	<i>GRand Unified Bootloader</i> . El bootloader más utilizado en sistemas Linux, capaz de arrancar múltiples sistemas operativos.
UKI	<i>Unified Kernel Image</i> (Imagen de Kernel Unificada). Archivo único que combina el kernel de Linux, el initramfs y los parámetros de arranque, facilitando el arranque seguro.
PAM	<i>Pluggable Authentication Modules</i> . Sistema modular de autenticación en Linux que permite configurar diferentes métodos de verificación de identidad.
2FA	<i>Two-Factor Authentication</i> (Autenticación de Dos Factores). Método de seguridad que requiere dos formas distintas de verificación: algo que sabes (contraseña) y algo que tienes (código temporal).
setuid	Permiso especial en sistemas Unix que permite ejecutar un programa con los privilegios del propietario del archivo, generalmente root. Potencialmente peligroso si se abusa de él.
AppArmor	Sistema de control de acceso obligatorio (MAC) que restringe las capacidades de los programas mediante perfiles de seguridad predefinidos.
nftables	Framework moderno de filtrado de paquetes en el kernel de Linux que reemplaza a iptables. Proporciona un firewall más eficiente y con sintaxis unificada.
UFW	<i>Uncomplicated Firewall</i> . Interfaz simplificada para gestionar reglas de firewall, ideal para usuarios que prefieren comandos más intuitivos.

4.1. Hardware de Red Seguro

La seguridad comienza en la capa física y de enlace de datos. Un hardware de red mal configurado puede invalidar todas las medidas de seguridad implementadas en capas superiores.

4.1.1. Configuración de Switches (VLANs, Port Security)

Los switches gestionables permiten implementar controles de seguridad a nivel de red local.

Creación de VLANs para Segmentación de Red

La segmentación mediante VLANs aísla diferentes tipos de tráfico y limita la propagación de amenazas.

Esquema de VLANs recomendado:

VLAN ID	Nombre	Propósito	Rango IP
1	Default	No usar (deshabilitar)	-
10	Gestión	Administración de equipos	192.168.10.0/24
20	Servidores	Servicios internos	192.168.20.0/24
30	Usuarios	Estaciones de trabajo	192.168.30.0/24
40	IoT	Dispositivos inteligentes	192.168.40.0/24
99	Cuarentena	Dispositivos no confiables	192.168.99.0/24

Configuración en switch gestionable (ejemplo con Open vSwitch en Debian 13):

Instalar Open vSwitch

```
sudo apt update
sudo apt install -y openvswitch-switch
```

Crear bridge virtual

```
sudo ovs-vsctl add-br br0
```

Añadir puerto físico al bridge

```
sudo ovs-vsctl add-port br0 enp1s0
```

Crear VLANs

```
sudo ovs-vsctl set port enp1s0 tag=10
```

```
# Puerto de acceso VLAN 10
```

```
sudo ovs-vsctl set port enp2s0 trunks=10,20,30 # Puerto trunk
```

```
# Verificar configuración
```

```
sudo ovs-vsctl show
```

Port Security (Seguridad de Puerto)

Limita qué dispositivos pueden conectarse a cada puerto del switch.

```
# Configuración de port security con Open vSwitch
```

```
# Limitar a una MAC por puerto
```

```
sudo ovs-vsctl set port enp1s0 other_config:max-mac-addresses=1
```

```
# Habilitar aprendizaje de MAC seguro
```

```
sudo ovs-vsctl set bridge br0 other-config:mac-aging-time=300
```

Archivo de configuración persistente /etc/network/interfaces.d/vlans:

```
# Interfaz VLAN 10 - Gestión
```

```
auto enp1s0.10
```

```
iface enp1s0.10 inet static
```

```
    address 192.168.10.1
```

```
    netmask 255.255.255.0
```

```
    vlan-raw-device enp1s0
```

```
# Interfaz VLAN 20 - Servidores
```

```
auto enp1s0.20
```

```
iface enp1s0.20 inet static
```

```
    address 192.168.20.1
```

```
    netmask 255.255.255.0
```

```
    vlan-raw-device enp1s0
```

4.1.2. Router Hardening (OpenWRT/Debian)

El router es el punto de entrada a la red y debe configurarse con especial cuidado.

Opción A: Debian 13 como Router

Habilitar enrutamiento IP:

```
# Habilitar IP forwarding de forma persistente
```

```
echo 'net.ipv4.ip_forward = 1' | sudo tee /etc/sysctl.d/90-router.conf
```

```
echo 'net.ipv6.conf.all.forwarding = 1' | sudo tee -a /etc/sysctl.d/90-router.conf
```

```
# Aplicar cambios
```

```
sudo sysctl --system
```

Deshabilitar servicios innecesarios:

Listar servicios activos

```
systemctl list-units --type=service --state=running
```

Deshabilitar servicios no esenciales para un router

```
sudo systemctl disable --now avahi-daemon.service
```

```
sudo systemctl disable --now cups.service
```

```
sudo systemctl disable --now bluetooth.service
```

```
sudo systemctl mask avahi-daemon.service cups.service bluetooth.service
```

Configuración de seguridad del kernel (/etc/sysctl.d/91-hardening.conf):

Protección contra ataques de red

```
net.ipv4.conf.all.rp_filter = 1
```

```
net.ipv4.conf.default.rp_filter = 1
```

```
net.ipv4.conf.all.accept_redirects = 0
```

```
net.ipv4.conf.default.accept_redirects = 0
```

```
net.ipv4.conf.all.send_redirects = 0
```

```
net.ipv4.conf.default.send_redirects = 0
```

```
net.ipv4.conf.all.accept_source_route = 0
```

```
net.ipv4.conf.default.accept_source_route = 0
```

```
net.ipv4.icmp_echo_ignore_broadcasts = 1
```

```
net.ipv4.icmp_ignore_bogus_error_responses = 1
```

```
net.ipv4.tcp_syncookies = 1
```

Protección IPv6

```
net.ipv6.conf.all.accept_redirects = 0
```

```
net.ipv6.conf.default.accept_redirects = 0
```

```
net.ipv6.conf.all.accept_source_route = 0
```

```
net.ipv6.conf.default.accept_source_route = 0
```

Opción B: OpenWRT Hardening

Para routers dedicados con OpenWRT:

Acceder via SSH (cambiar contraseña por defecto primero)

```
ssh root@192.168.1.1
```

Cambiar contraseña de root

```
passwd
```

Deshabilitar acceso HTTP (usar solo HTTPS)

```
uci set uhttpd.main.listen_http=""
```

```
uci set uhttpd.main.listen_https='0.0.0.0:443'
```

```
uci commit uhttpd
```

```
/etc/init.d/uhttpd restart
```

Cambiar puerto SSH y deshabilitar acceso desde WAN

```
uci set dropbear.@dropbear[0].Port='2222'
```

```
uci set dropbear.@dropbear[0].Interface='lan'
```

```
uci commit dropbear  
/etc/init.d/dropbear restart
```

```
# Habilitar actualizaciones automáticas de listas de bloqueo  
opkg update  
opkg install banip
```

4.1.3. Control de MAC Spoofing

El MAC spoofing permite a atacantes evadir controles basados en direcciones MAC.

Detección de MAC spoofing con arpwatcH:

```
# Instalar arpwatcH  
sudo apt install -y arpwatcH  
  
# Configurar para cada interfaz  
sudo nano /etc/default/arpwatcH
```

Contenido de /etc/default/arpwatcH:

```
# Interfaces a monitorizar  
INTERFACES="enp1s0 enp2s0"  
  
# Opciones adicionales  
ARGS="-N -p"
```

Habilitar y verificar:

```
# Iniciar servicio  
sudo systemctl enable --now arpwatcH  
  
# Verificar detecciones  
sudo tail -f /var/log/syslog | grep arpwatcH
```

Implementar 802.1X para autenticación de puerto:

```
# Instalar hostapd para autenticación 802.1X  
sudo apt install -y hostapd freeradius  
  
# Configuración básica de hostapd para 802.1X cableado  
sudo nano /etc/hostapd/wired.conf
```

Contenido de /etc/hostapd/wired.conf:

```
interface=enp1s0  
driver=wired  
ieee8021x=1  
eap_server=0  
auth_server_addr=127.0.0.1
```

```
auth_server_port=1812
auth_server_shared_secret=ClaveSecretaRADIUS
```

4.2. Instalación Segura de Debian 13 “Trixie”

Debian 13 “Trixie” introduce mejoras significativas en seguridad. Una instalación cuidadosa establece las bases para un sistema robusto.

4.2.1. Particionado Moderno: LUKS2 con systemd-cryptenroll

LUKS2 proporciona cifrado de disco con mejoras respecto a su predecesor, incluyendo mejor resistencia a ataques de fuerza bruta y soporte para tokens de seguridad.

Esquema de Particionado Recomendado

Partición	Tamaño	Sistema de Archivos	Punto de Montaje	Cifrado
EFI	512 MB	FAT32	/boot/efi	No
Boot	1 GB	ext4	/boot	No*
Sistema	Resto	LUKS2 + LVM	/	Sí

*Con UKI, /boot puede integrarse en la partición EFI.

Creación de Particiones Cifradas Durante la Instalación

Durante el instalador de Debian:

15. Seleccionar “Particionado manual”
16. Crear partición EFI (512 MB, FAT32)
17. Crear partición boot (1 GB, ext4)
18. Crear partición para LUKS con el resto del espacio

Configuración Post-Instalación con systemd-cryptenroll

systemd-cryptenroll permite agregar métodos adicionales de desbloqueo.

Verificar estado actual de LUKS

```
sudo cryptsetup luksDump /dev/sda3
```

Agregar clave de recuperación imprimible

```
sudo systemd-cryptenroll --recovery-key /dev/sda3
```

IMPORTANTE: Guardar la clave de recuperación en lugar seguro

Agregar desbloqueo mediante TPM 2.0 (si está disponible):

Verificar disponibilidad de TPM

```
sudo systemd-cryptenroll --tpm2-device=list
```



```
# Registrar TPM para desbloqueo automático
sudo systemd-cryptenroll --tpm2-device=auto \
    --tpm2-pcrs=0+7 \
    /dev/sda3
```

Agregar token FIDO2 (llave de seguridad USB):

```
# Instalar soporte FIDO2
sudo apt install -y libfido2-1
```

```
# Registrar token FIDO2
sudo systemd-cryptenroll --fido2-device=auto /dev/sda3
```

Configuración de /etc/crypttab para arranque:

```
# Ejemplo de crypttab con múltiples métodos
sda3_crypt UUID=xxxx-xxxx-xxxx none luks,discard,tpm2-device=auto,fido2-
device=auto
```

4.2.2. Sistema de Archivos: /tmp en tmpfs (RAM)

Montar /tmp en memoria RAM tiene múltiples beneficios de seguridad: - Los archivos temporales se eliminan automáticamente al reiniciar - Mayor velocidad de acceso - Reduce el desgaste de discos SSD - Los archivos temporales nunca tocan el disco (importantes para datos sensibles)

Configurar /tmp en tmpfs:

```
# Verificar si ya está configurado
mount | grep /tmp
```

```
# Añadir a /etc/fstab
echo 'tmpfs /tmp tmpfs defaults,noatime,nosuid,nodev,noexec,mode=1777,size=2G
0 0' | sudo tee -a /etc/fstab
```

```
# Montar inmediatamente
sudo systemctl daemon-reload
sudo mount /tmp
```

Opciones de seguridad explicadas:

Opción	Función
nosuid	Ignora bits setuid/setgid (previene escalada de privilegios)
nodev	No permite dispositivos especiales (previene ataques mediante dispositivos)
noexec	Prohíbe ejecución de binarios (previene ejecución de malware en /tmp)

Opción	Función
mode=1777	Sticky bit activo (usuarios solo pueden borrar sus propios archivos)
size=2G	Limita el tamaño máximo en RAM

Configurar también /var/tmp:

```
# /var/tmp también debe ser seguro (pero persiste entre reinicios por diseño)
echo 'tmpfs /var/tmp tmpfs
defaults,noatime,nosuid,nodev,noexec,mode=1777,size=1G 0 0' | sudo tee -a
/etc/fstab
```

4.2.3. Selección Mínima de Paquetes (netinst)

Una instalación mínima reduce drásticamente la superficie de ataque.

Durante la Instalación

- Descargar imagen netinst de Debian 13 desde <https://www.debian.org/devel/debian-installer/>
- En la selección de software, desmarcar **todas** las opciones excepto:
 1. “Utilidades del sistema estándar”
- NO seleccionar ningún entorno de escritorio

Post-Instalación: Instalar Solo lo Necesario

Actualizar sistema base

```
sudo apt update && sudo apt upgrade -y
```

Instalar herramientas esenciales de seguridad únicamente

```
sudo apt install -y \
sudo \
openssh-server \
ufw \
fail2ban \
unattended-upgrades \
apt-listbugs \
apt-listchanges \
needrestart \
debsums
```

Configurar actualizaciones automáticas de seguridad

```
sudo dpkg-reconfigure -plow unattended-upgrades
```

Verificar paquetes instalados:

Listar paquetes instalados manualmente

```
apt-mark showmanual
```

```
# Contar paquetes totales (objetivo: menos de 300)
dpkg -l | grep '^ii' | wc -l
```

Eliminar paquetes innecesarios:

```
# Identificar paquetes huérfanos
sudo apt install -y deborphan
deborphan
```

```
# Eliminar paquetes huérfanos
sudo apt autoremove --purge $(deborphan)
```

```
# Limpiar caché
sudo apt clean
```

4.3. Hardening del Bootloader

El bootloader es el primer código que se ejecuta y un objetivo prioritario para atacantes que buscan persistencia.

4.3.1. Transición a systemd-boot o GRUB Firmado

Debian 13 soporta Secure Boot con GRUB firmado y ofrece systemd-boot como alternativa más simple.

Opción A: GRUB con Secure Boot (Recomendado para la mayoría)

```
# Verificar estado de Secure Boot
mokutil --sb-state
```

```
# Instalar GRUB firmado para UEFI
sudo apt install -y grub-efi-amd64-signed shim-signed
```

```
# Reinstalar GRUB con firma
sudo grub-install --target=x86_64-efi --efi-directory=/boot/efi --bootloader-id=debian --recheck
```

```
# Actualizar configuración
sudo update-grub
```

Proteger GRUB con contraseña:

```
# Generar hash de contraseña
grub-mkpasswd-pbkdf2
```

```
# Añadir a /etc/grub.d/40_custom
sudo nano /etc/grub.d/40_custom
```

Contenido de /etc/grub.d/40_custom:

```
#!/bin/sh
set superusers="admin"
password_pbkdf2 admin grub.pbkdf2.sha512.10000.HASH_GENERADO_AQUI

# Aplicar cambios
sudo chmod 755 /etc/grub.d/40_custom
sudo update-grub
```

Opción B: systemd-boot (Más simple, requiere UKI)

```
# Instalar systemd-boot
sudo apt install -y systemd-boot
```

```
# Instalar en la partición EFI
sudo bootctl install --path=/boot/efi
```

```
# Verificar instalación
sudo bootctl status
```

Configuración de /boot/efi/loader/loader.conf:

```
default debian.efi
timeout 3
console-mode max
editor no
```

4.3.2. Imágenes de Kernel Unificadas (UKI)

Las UKI combinan kernel, initramfs y cmdline en un único archivo firmado, mejorando la seguridad del arranque.

Instalar herramientas necesarias:

```
# Instalar ukify (herramienta para crear UKI)
sudo apt install -y systemd-ukify systemd-boot-efi
```

```
# Instalar herramientas de firma (opcional, para Secure Boot)
sudo apt install -y sbsigntool
```

Crear UKI manualmente:

```
# Crear directorio para UKI
sudo mkdir -p /boot/efi/EFI/Linux
```

```
# Generar UKI con ukify
sudo ukify build \
  --linux=/boot/vmlinuz-$(uname -r) \
  --initrd=/boot/initrd.img-$(uname -r) \
  --cmdline="root=UUID=$(blkid -s UUID -o value /dev/mapper/sda3_crypt) ro
quiet" \
  --output=/boot/efi/EFI/Linux/debian-$(uname -r).efi
```

Automatizar generación de UKI (/etc/kernel/postinst.d/zz-ukify):

```
#!/bin/bash
# Script para generar UKI automáticamente tras actualización de kernel

KERNEL_VERSION=$1
KERNEL_IMAGE=$2

if [ -z "$KERNEL_VERSION" ]; then
    exit 0
fi

ukify build \
    --linux=/boot/vmlinuz-${KERNEL_VERSION} \
    --initrd=/boot/initrd.img-${KERNEL_VERSION} \
    --cmdline="$(cat /etc/kernel/cmdline)" \
    --output=/boot/efi/EFI/Linux/debian-${KERNEL_VERSION}.efi

# Eliminar UKI de kernels antiguos (mantener últimos 2)
ls -t /boot/efi/EFI/Linux/debian-*.efi | tail -n +3 | xargs -r rm

# Hacer ejecutable
sudo chmod +x /etc/kernel/postinst.d/zz-ukify

# Crear archivo de cmdline
echo "root=UUID=$(blkid -s UUID -o value /dev/mapper/sda3_crypt) ro quiet" |
sudo tee /etc/kernel/cmdline
.
```

4.4. Hardening Básico del Sistema

Esta sección cubre las configuraciones fundamentales de seguridad del sistema operativo.

4.4.1. Usuarios y PAM: Desactivar Root, 2FA

Desactivar Acceso Directo como Root

Bloquear contraseña de root (fuerza uso de sudo)

```
sudo passwd -l root
```

Deshabilitar shell de root

```
sudo usermod -s /usr/sbin/nologin root
```

Verificar

```
sudo grep root /etc/passwd
```

Debe mostrar: root:x:0:0:root:/root:/usr/sbin/nologin

Configurar sudo correctamente:

Crear usuario administrativo si no existe

```
sudo adduser adminuser
```

```
sudo usermod -aG sudo adminuser
```

Configurar sudo con restricciones

```
sudo visudo
```

Añadir a /etc/sudoers (via visudo):

Requerir contraseña siempre

```
Defaults    timestamp_timeout=5
```

```
Defaults    passwd_tries=3
```

```
Defaults    logfile="/var/log/sudo.log"
```

```
Defaults    log_input, log_output
```

```
Defaults    requiretty
```

Usuarios administrativos

```
adminuser  ALL=(ALL:ALL) ALL
```

Implementar 2FA con Google Authenticator

Instalar módulo PAM de Google Authenticator

```
sudo apt install -y libpam-google-authenticator
```

Configurar para cada usuario (ejecutar como usuario, no root)

```
google-authenticator
```

Responder a las preguntas de configuración: - Do you want authentication

tokens to be time-based? → **y** - Do you want me to update your

"~/.google_authenticator" file? → **y** - Do you want to disallow multiple uses...? → **y** -

Do you want to enable rate-limiting? → **y**

Configurar PAM para requerir 2FA (/etc/pam.d/sshd):

Añadir al inicio del archivo (después de @include common-auth)

```
auth required pam_google_authenticator.so nullok
```

nullok permite acceso sin 2FA si el usuario no lo ha configurado

Eliminar nullok después de que todos los usuarios lo configuren

Configurar SSH para usar 2FA (/etc/ssh/sshd_config):

Habilitar autenticación interactiva

```
KbdInteractiveAuthentication yes
```

```
ChallengeResponseAuthentication yes
```

Requerir tanto clave como 2FA

```
AuthenticationMethods publickey,keyboard-interactive
```

Reiniciar SSH

```
sudo systemctl restart sshd
```

4.4.2. Permisos Críticos y Limpieza de setuid

Los binarios con bit setuid ejecutan con privilegios elevados y son vectores de ataque comunes.

Identificar binarios setuid/setgid:*# Buscar todos los binarios setuid*

```
sudo find / -type f -perm -4000 -ls 2>/dev/null
```

Buscar todos los binarios setgid

```
sudo find / -type f -perm -2000 -ls 2>/dev/null
```

Guardar lista para auditoría

```
sudo find / -type f \( -perm -4000 -o -perm -2000 \) -ls 2>/dev/null > /root/setuid_audit.txt
```

Binarios setuid que generalmente pueden eliminarse:*# Eliminar setuid de binarios no esenciales*

```
sudo chmod u-s /usr/bin/newgrp
sudo chmod u-s /usr/bin/chsh
sudo chmod u-s /usr/bin/chfn
sudo chmod u-s /usr/bin/gpasswd
sudo chmod u-s /usr/bin/wall
sudo chmod u-s /usr/bin/write
```

Si no se usa mount por usuarios normales

```
sudo chmod u-s /usr/bin/mount
sudo chmod u-s /usr/bin/umount
```

Binarios setuid esenciales (NO eliminar):

Binario	Función
/usr/bin/sudo	Elevación de privilegios controlada
/usr/bin/passwd	Cambio de contraseñas
/usr/bin/su	Cambio de usuario (puede eliminarse si solo se usa sudo)
/usr/lib/openssh/ssh-keysign	Autenticación SSH basada en host

Monitorear cambios en binarios setuid:*# Instalar AIDE para detección de intrusiones*

```
sudo apt install -y aide
```

Inicializar base de datos

```
sudo aideinit
```

```
# Programar verificación diaria
```

```
echo '0 5 * * * root /usr/bin/aide --check' | sudo tee /etc/cron.d/aide-check
```

4.4.3. AppArmor: Auditoría y Perfiles en Modo 'Enforce'

AppArmor restringe lo que cada programa puede hacer, limitando el daño de vulnerabilidades.

Verificar estado de AppArmor:

```
# Instalar herramientas de AppArmor
```

```
sudo apt install -y apparmor apparmor-utils apparmor-profiles apparmor-profiles-extra
```

```
# Verificar estado
```

```
sudo aa-status
```

Estados de los perfiles: - **enforce:** El perfil se aplica activamente (objetivo de seguridad) - **complain:** Solo registra violaciones sin bloquear (modo auditoría) - **unconfined:** Sin restricciones (inseguro)

Habilitar AppArmor si no está activo:

```
# Añadir a parámetros del kernel
```

```
sudo nano /etc/default/grub
```

```
# Añadir a GRUB_CMDLINE_LINUX: apparmor=1 security=apparmor
```

```
# Actualizar GRUB
```

```
sudo update-grub
```

```
sudo reboot
```

Poner perfiles en modo enforce:

```
# Ver perfiles en modo complain
```

```
sudo aa-complain --list
```

```
# Pasar todos los perfiles a modo enforce
```

```
sudo aa-enforce /etc/apparmor.d/*
```

```
# Pasar perfil específico a enforce
```

```
sudo aa-enforce /etc/apparmor.d/usr.sbin.sshd
```

Crear perfil básico para una aplicación:

```
# Generar perfil automáticamente (modo aprendizaje)
```

```
sudo aa-genprof /usr/bin/mi_aplicacion
```

```
# En otra terminal, ejecutar la aplicación con todos sus usos normales
```


Volver a la terminal de aa-genprof y pulsar 'S' para escanear logs

Después de ajustar, pasar a enforce

sudo aa-enforce /etc/apparmor.d/usr.bin.mi_aplicacion

Ejemplo de perfil personalizado (/etc/apparmor.d/usr.local.bin.miapp):

#include <tunables/global>

```
/usr/local/bin/miapp {  
  #include <abstractions/base>  
  #include <abstractions/nameservice>
```

Permitir lectura de su propio binario
/usr/local/bin/miapp mr,

Permitir lectura de configuración
/etc/miapp/** r,

Permitir escritura en directorio de logs
/var/log/miapp/** w,

Permitir acceso a red
network inet stream,
network inet6 stream,

Denegar todo lo demás implícitamente
}

Cargar perfil

sudo apparmor_parser -r /etc/apparmor.d/usr.local.bin.miapp

4.5. Firewall Base (nftables/UFW)

Un firewall correctamente configurado es esencial para controlar el tráfico de red.

Opción A: UFW (Recomendado para Principiantes)

UFW proporciona una interfaz simplificada sobre nftables.

Configuración básica:

Instalar UFW

sudo apt install -y ufw

Configurar políticas por defecto

sudo ufw default deny incoming

sudo ufw default allow outgoing

Permitir SSH (¡IMPORTANTE antes de activar!)

```
sudo ufw allow ssh
```

Activar firewall

```
sudo ufw enable
```

Verificar estado

```
sudo ufw status verbose
```

Reglas comunes:

Permitir SSH solo desde red local

```
sudo ufw allow from 192.168.1.0/24 to any port 22
```

Permitir HTTP/HTTPS

```
sudo ufw allow 80/tcp
```

```
sudo ufw allow 443/tcp
```

Permitir puerto específico desde IP específica

```
sudo ufw allow from 192.168.1.100 to any port 5432
```

Denegar IP específica

```
sudo ufw deny from 10.0.0.5
```

Limitar conexiones (protección contra fuerza bruta)

```
sudo ufw limit ssh
```

Ver reglas numeradas (para eliminar)

```
sudo ufw status numbered
```

Eliminar regla

```
sudo ufw delete 3
```

Opción B: nftables Directamente (Mayor Control)

Para usuarios avanzados que necesitan mayor flexibilidad.

Configuración base (/etc/nftables.conf):

```
#!/usr/sbin/nft -f
```

Limpiar reglas existentes

```
flush ruleset
```

Definir tabla para filtrado IPv4/IPv6

```
table inet filter {
```

```
    # Conjunto de IPs bloqueadas
```

```
    set blocklist {
```

```
type ipv4_addr
flags interval
elements = { 10.0.0.0/8, 172.16.0.0/12 }
}

# Conjunto de puertos permitidos
set tcp_allowed {
    type inet_service
    elements = { 22, 80, 443 }
}

# Cadena de entrada
chain input {
    type filter hook input priority 0; policy drop;

    # Permitir tráfico de loopback
    iif "lo" accept

    # Permitir conexiones establecidas y relacionadas
    ct state established,related accept

    # Descartar inválidos
    ct state invalid drop

    # Bloquear IPs del blocklist
    ip saddr @blocklist drop

    # Protección contra escaneo de puertos
    tcp flags syn tcp dport != @tcp_allowed \
        limit rate 1/second burst 5 packets \
        log prefix "PORTSCAN: " drop

    # Permitir ICMP (ping) con límite
    ip protocol icmp icmp type echo-request \
        limit rate 5/second accept

    # Permitir puertos TCP configurados
    tcp dport @tcp_allowed accept

    # Registrar y descartar el resto
    log prefix "INPUT DROP: " drop
}

# Cadena de salida
chain output {
    type filter hook output priority 0; policy accept;
```

```
# Permitir todo el tráfico saliente
# Añadir restricciones según necesidades
}

# Cadena de reenvío (para routers)
chain forward {
    type filter hook forward priority 0; policy drop;

    # Permitir tráfico establecido
    ct state established,related accept

    # Añadir reglas de reenvío entre VLANs aquí
}

# Tabla para NAT (si actúa como router)
table ip nat {
    chain postrouting {
        type nat hook postrouting priority 100;

        # Masquerade para red interna
        ip saddr 192.168.0.0/16 oifname "enp1s0" masquerade
    }
}
```

Aplicar y habilitar:

```
# Verificar sintaxis
sudo nft -c -f /etc/nftables.conf

# Aplicar reglas
sudo nft -f /etc/nftables.conf

# Habilitar servicio
sudo systemctl enable --now nftables

# Verificar reglas activas
sudo nft list ruleset
```

Comandos útiles de nftables:

```
# Añadir IP al blocklist dinámicamente
sudo nft add element inet filter blocklist { 203.0.113.50 }

# Eliminar IP del blocklist
sudo nft delete element inet filter blocklist { 203.0.113.50 }

# Ver contadores de reglas
```

```
sudo nft list ruleset -a
```

```
# Monitorear en tiempo real
```

```
sudo nft monitor
```

Checklist Nivel 3: Verificación de Seguridad del Sistema Base

Este checklist permite verificar que todas las medidas de seguridad han sido implementadas correctamente.

4.1 Hardware de Red

#	Verificación	Comando de Comprobación	Estado
4.1 .1a	VLANs configuradas y separadas	sudo ovs-vsctl show o cat /proc/net/vlan/config	[]
4.1 .1b	VLAN 1 (default) no en uso	sudo ovs-vsctl list-ports br0	[]
4.1 .1c	Port security habilitado	sudo ovs-vsctl get port enp1s0 other_config	[]
4.1 .2a	IP forwarding habilitado (router)	sysctl net.ipv4.ip_forward	[]
4.1 .2b	Protecciones de red en sysctl	sysctl -a \ grep rp_filter	[]
4.1 .2c	Servicios innecesarios deshabilitados	systemctl list-units --state=running	[]
4.1 .3a	arpwatch instalado y activo	systemctl status arpwatch	[]
4.1 .3b	Logs de MAC spoofing monitoreados	grep -i flip /var/log/syslog	[]

4.2 Instalación Debian 13

#	Verificación	Comando de Comprobación	Estado
4.2 .1a	Partición raíz cifrada con LUKS2	sudo cryptsetup luksDump /dev/sda3 \ grep Version	[]
4.2 .1b	systemd-cryptenroll configurado	sudo systemd-cryptenroll /dev/sda3 --list	[]
4.2 .1c	Clave de recuperación generada y guardada	Verificar manualmente	[]
4.2 .2a	/tmp montado en tmpfs	mount \ grep "tmpfs on /tmp"	[]
4.2 .2b	/tmp con opciones nosuid,nodev,noexec	mount \ grep /tmp	[]
4.2 .3a	Instalación mínima (<300 paquetes)	dpkg -l \ grep '^ii' \ wc -l	[]

#	Verificación	Comando de Comprobación	Estado
4.2 .3b	unattended-upgrades configurado	systemctl status unattended-upgrades	[]
4.2 .3c	No hay paquetes huérfanos	deborphan \ wc -l (debe ser 0)	[]

4.3 Bootloader

#	Verificación	Comando de Comprobación	Estado
4.3 .1a	Secure Boot habilitado	mokutil --sb-state	[]
4.3 .1b	GRUB firmado instalado	dpkg -l \ grep grub-efi-amd64-signed	[]
4.3 .1c	GRUB protegido con contraseña	grep superusers /boot/grub/grub.cfg	[]
4.3 .2a	UKI generada (si aplica)	ls /boot/efi/EFI/Linux/*.efi	[]
4.3 .2b	Script de auto-generación UKI	test -x /etc/kernel/postinst.d/zz-ukify	[]

4.4 Hardening del Sistema

#	Verificación	Comando de Comprobación	Estado
4.4 .1a	Root bloqueado	sudo passwd -S root (debe mostrar 'L')	[]
4.4 .1b	Shell de root deshabilitado	grep root /etc/passwd \ grep nologin	[]
4.4 .1c	sudo con logging habilitado	grep logfile /etc/sudoers	[]
4.4 .1d	2FA configurado para SSH	grep pam_google_authenticator /etc/pam.d/sshd	[]
4.4 .1e	SSH requiere clave + 2FA	grep AuthenticationMethods /etc/ssh/sshd_config	[]
4.4 .2a	Auditoría setuid realizada	test -f /root/setuid_audit.txt	[]
4.4 .2b	Binarios setuid innecesarios eliminados	find /usr -perm -4000 2>/dev/null \ wc -l (<10)	[]
4.4 .2c	AIDE instalado y configurado	systemctl status aide-check.timer	[]
4.4 .3a	AppArmor habilitado	sudo aa-status \ grep "apparmor module is loaded"	[]
4.4 .3b	Perfiles en modo enforce	sudo aa-status \ grep -c enforce (>20)	[]
4.4 .3c	Sin perfiles en modo complain	sudo aa-status \ grep -c complain (=0)	[]

4.5 Firewall

#	Verificación	Comando de Comprobación	Estado
4.5a	Firewall activo	sudo ufw status o sudo nft list ruleset	[]
4.5b	Política por defecto: deny incoming	sudo ufw status verbose \ grep "Default: deny"	[]
4.5c	Solo puertos necesarios abiertos	sudo ss -tlnp	[]
4.5d	SSH limitado por IP o rate-limit	sudo ufw status \ grep -E "22.*LIMIT\ Anywhere"	[]
4.5e	Logging de firewall habilitado	sudo ufw logging status o revisar nftables log	[]

Script de Verificación Automática

Guardar como /usr/local/bin/check-nivel3.sh:

```
#!/bin/bash
# Script de verificación de seguridad - Nivel 3
# Debian 13 "Trixie"

RED='\033[0;31m'
GREEN='\033[0;32m'
YELLOW='\033[1;33m'
NC='\033[0m'

PASS=0
FAIL=0
WARN=0

check() {
    local description="$1"
    local command="$2"
    local expected="$3"

    result=$(eval "$command" 2>/dev/null)

    if [[ "$result" == *"$expected"* ]]; then
        echo -e "${GREEN}[PASS]${NC} $description"
        ((PASS++))
    else
        echo -e "${RED}[FAIL]${NC} $description"
        ((FAIL++))
    fi
}
```

```

}

warn_check() {
    local description="$1"
    local command="$2"

    if eval "$command" &>/dev/null; then
        echo -e "${GREEN}[PASS]${NC} $description"
        ((PASS++))
    else
        echo -e "${YELLOW}[WARN]${NC} $description"
        ((WARN++))
    fi
}

echo "=====
echo " Verificación de Seguridad - Nivel 3"
echo " Debian 13 Sistema Base"
echo "=====
echo ""

echo "--- 4.2 Instalación Segura ---"
check "LUKS2 cifrado activo" "cryptsetup luksDump /dev/sda3 2>/dev/null | grep
Version" "2"
check "/tmp en tmpfs" "mount | grep 'tmpfs on /tmp'" "tmpfs"
check "/tmp con noexec" "mount | grep /tmp" "noexec"
check "Instalación mínima" "test $(dpkg -l | grep '^ii' | wc -l) -lt 400 && echo 'ok'"
"ok"

echo ""
echo "--- 4.3 Bootloader ---"
warn_check "Secure Boot habilitado" "mokutil --sb-state 2>/dev/null | grep -q
'enabled'"
check "GRUB firmado" "dpkg -l | grep grub-efi-amd64-signed" "ii"

echo ""
echo "--- 4.4 Hardening Sistema ---"
check "Root bloqueado" "passwd -S root" "L"
check "AppArmor cargado" "aa-status 2>/dev/null | head -1" "apparmor module is
loaded"
check "Perfiles enforce > 15" "test $(aa-status 2>/dev/null | grep -c enforce) -gt 15
&& echo 'ok'" "ok"

echo ""
echo "--- 4.5 Firewall ---"
check "UFW/nftables activo" "ufw status 2>/dev/null | head -1 || nft list ruleset |
head -1" "active|table"

```



```
check "Política deny incoming" "ufw status verbose 2>/dev/null | grep Default"
"deny"

echo ""
echo "=====
echo -e "Resultados: ${GREEN}$PASS PASS${NC}, ${RED}$FAIL FAIL${NC}, $
{YELLOW}$WARN WARN${NC}"
echo "=====

if [ $FAIL -gt 0 ]; then
    exit 1
fi
exit 0

# Hacer ejecutable
sudo chmod +x /usr/local/bin/check-nivel3.sh

# Ejecutar verificación
sudo /usr/local/bin/check-nivel3.sh
,
```

Resumen

Este nivel ha cubierto los fundamentos de seguridad para un sistema Debian 13:

- **Hardware de red:** Segmentación con VLANs, protección de puertos y detección de MAC spoofing
- **Instalación segura:** Cifrado LUKS2, tmpfs para datos temporales e instalación mínima
- **Bootloader:** Secure Boot, GRUB protegido y UKI para arranque verificable
- **Hardening:** Desactivación de root, 2FA, limpieza de setuid y AppArmor en modo enforce
- **Firewall:** Configuración restrictiva con UFW o nftables

El siguiente nivel abordará la seguridad de servicios y aplicaciones que se ejecutan sobre esta base segura.

Nivel 4: Red, Servicios y Mantenimiento Avanzado

Manual de Seguridad Informática para Debian 13 “Trixie”

Glosario de Términos Esenciales

Antes de profundizar en los contenidos técnicos, es fundamental comprender los siguientes términos que aparecerán a lo largo de este nivel:

Término	Definición
DMZ (Zona Desmilitarizada)	Subred aislada que actúa como zona intermedia entre la red interna (privada) y la red externa (Internet). Aloja servicios públicos como servidores web, minimizando el riesgo si son comprometidos.
DNS (Sistema de Nombres de Dominio)	Protocolo que traduce nombres de dominio legibles (ejemplo.com) a direcciones IP numéricas (192.168.1.1).
DNSSEC (Extensiones de Seguridad DNS)	Conjunto de extensiones que añaden firmas criptográficas a las respuestas DNS, permitiendo verificar su autenticidad y prevenir manipulaciones.
DoT (DNS over TLS)	Protocolo que cifra las consultas DNS mediante TLS, protegiendo la privacidad de las consultas contra espionaje en la red.
VPN (Red Privada Virtual)	Tecnología que crea un túnel cifrado entre dispositivos a través de Internet, permitiendo comunicaciones seguras como si estuvieran en la misma red local.
WireGuard	Protocolo VPN moderno, ligero y de alto rendimiento, integrado en el kernel de Linux desde la versión 5.6.
SSH (Secure Shell)	Protocolo de red que permite acceso remoto seguro a sistemas mediante comunicaciones cifradas.
DSA (Algoritmo de Firma Digital)	Algoritmo criptográfico antiguo para firmas digitales, actualmente considerado obsoleto e inseguro.
Ed25519	Algoritmo de firma digital moderno basado en curvas elípticas, ofrece alta seguridad con claves pequeñas y rendimiento excelente.
FIDO2/U2F	Estándares abiertos de autenticación que permiten usar dispositivos físicos (llaves de seguridad) como segundo factor de autenticación.
CSP (Política de Seguridad de	Cabecera HTTP que define qué recursos puede

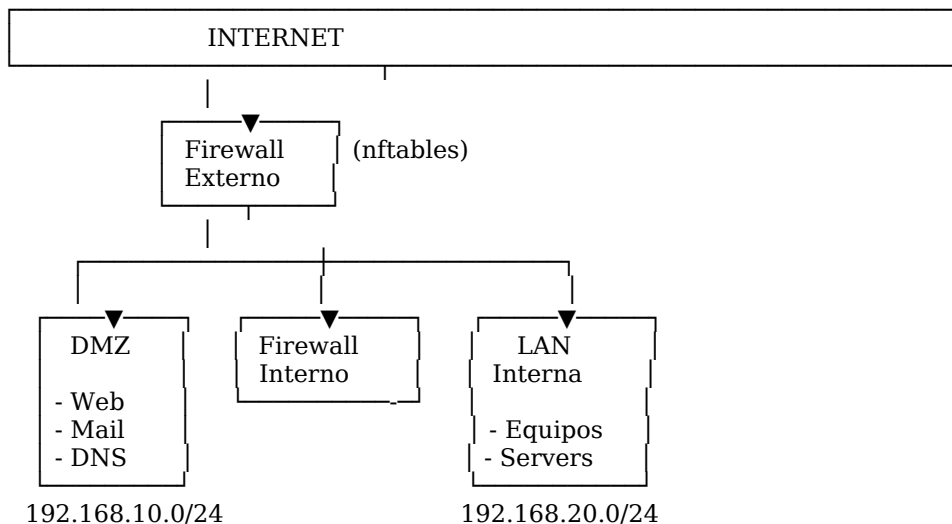
Término	Definición
Contenido)	cargar una página web, previniendo ataques de inyección de código (XSS).
HSTS (Seguridad de Transporte Estricta HTTP)	Cabecera HTTP que obliga a los navegadores a usar exclusivamente conexiones HTTPS con el servidor.
TLS (Seguridad de Capa de Transporte)	Protocolo criptográfico que proporciona comunicaciones seguras en Internet. TLS 1.3 es la versión más reciente y segura.
deb822	Formato moderno de configuración para repositorios APT en Debian, más estructurado y legible que el formato tradicional de una línea.
journald	Servicio de systemd que recopila y gestiona logs del sistema de forma estructurada, con soporte para firmado criptográfico.
auditd	Demonio de auditoría del kernel de Linux que registra eventos de seguridad del sistema según reglas configurables.
Estrategia 3-2-1	Metodología de copias de seguridad: mantener 3 copias de los datos, en 2 tipos de medios diferentes, con 1 copia fuera del sitio.

5.1. Arquitectura de Red y DNS

5.1.1. Segmentación y DMZ

La **segmentación de red** consiste en dividir una red en subredes más pequeñas y aisladas. Esto limita el movimiento lateral de un atacante que comprometa un sistema, conteniendo el daño potencial.

Arquitectura Recomendada



Configuración de nftables para DMZ

Cree el archivo /etc/nftables.conf:

```
#!/usr/sbin/nft -f
```

```
flush ruleset
```

```
# Definición de variables
```

```
define LAN_NET = 192.168.20.0/24
```

```
define DMZ_NET = 192.168.10.0/24
```

```
define WAN_IF = "eth0"
```

```
define LAN_IF = "eth1"
```

```
define DMZ_IF = "eth2"
```

```
table inet filter {
```

```
  # Cadena de entrada: tráfico hacia el firewall
```

```
  chain input {
```

```
    type filter hook input priority 0; policy drop;
```

```
    # Permitir tráfico establecido y relacionado
```

```
    ct state established,related accept
```

```
    # Permitir loopback
```

```
    iif "lo" accept
```

```
    # Permitir ICMP (ping) limitado
```

```
    ip protocol icmp icmp type echo-request limit rate 5/second accept
```

```
    # Permitir SSH solo desde LAN
```

```
    iifname $LAN_IF tcp dport 22 accept
```

```
    # Registrar y descartar el resto
```

```
    log prefix "INPUT DROP: " drop
```

```
  }
```

```
  # Cadena de reenvío: tráfico entre interfaces
```

```
  chain forward {
```

```
    type filter hook forward priority 0; policy drop;
```

```
    # Permitir tráfico establecido
```

```
    ct state established,related accept
```

```
    # LAN puede acceder a DMZ e Internet
```

```
    iifname $LAN_IF oifname $DMZ_IF accept
```

```
    iifname $LAN_IF oifname $WAN_IF accept
```

```
    # DMZ puede salir a Internet (para actualizaciones)
```

```
iifname $DMZ_IF oifname $WAN_IF accept

# DMZ NO puede acceder a LAN (regla crítica de seguridad)
iifname $DMZ_IF oifname $LAN_IF drop

# Permitir acceso desde Internet a servicios DMZ
iifname $WAN_IF oifname $DMZ_IF tcp dport { 80, 443 } accept

log prefix "FORWARD DROP: " drop
}

# Cadena de salida: tráfico desde el firewall
chain output {
    type filter hook output priority 0; policy accept;
}

# Tabla NAT para traducción de direcciones
table inet nat {
    chain prerouting {
        type nat hook prerouting priority -100;

        # Redirigir tráfico web a servidor DMZ
        iifname $WAN_IF tcp dport { 80, 443 } dnat to 192.168.10.10
    }

    chain postrouting {
        type nat hook postrouting priority 100;

        # Enmascarar tráfico saliente
        oifname $WAN_IF masquerade
    }
}

Comandos de Verificación
# Verificar reglas cargadas
sudo nft list ruleset

# Habilitar reenvío de paquetes IP
sudo sysctl -w net.ipv4.ip_forward=1

# Hacer permanente el reenvío
echo "net.ipv4.ip_forward = 1" | sudo tee /etc/sysctl.d/99-ip-forward.conf

# Verificar estado del reenvío
cat /proc/sys/net/ipv4/ip_forward

# Habilitar y arrancar nftables
```

```
sudo systemctl enable nftables
sudo systemctl start nftables
```

```
# Verificar estado
sudo systemctl status nftables
```

5.1.2. DNS Seguro: DNSSEC y DoT con systemd-resolved

systemd-resolved es el servicio de resolución DNS integrado en systemd, que soporta tanto DNSSEC como DoT de forma nativa.

Configuración de systemd-resolved

Edite el archivo `/etc/systemd/resolved.conf`:

```
# /etc/systemd/resolved.conf
# Configuración de DNS seguro para Debian 13 Trixie
```

[Resolve]

```
# Servidores DNS primarios con soporte DoT
# Formato: IP#nombre_servidor (el nombre se usa para verificar TLS)
DNS=9.9.9.9#dns.quad9.net 149.112.112.112#dns.quad9.net
FallbackDNS=1.1.1.1#cloudflare-dns.com 8.8.8.8#dns.google
```

```
# Dominios de búsqueda (opcional)
#Domains=ejemplo.local
```

```
# Habilitar DNSSEC
# yes: rechaza respuestas no firmadas
# allow-downgrade: acepta si el dominio no soporta DNSSEC
DNSSEC=yes
```

```
# Habilitar DNS over TLS
# yes: solo conexiones cifradas
# opportunistic: intenta cifrar, pero acepta sin cifrar si falla
DNSOverTLS=yes
```

```
# Modo de caché
Cache=yes
```

```
# No usar DNS del archivo /etc/resolv.conf tradicional
ReadEtcHosts=yes
```

```
# Multicast DNS (mDNS) deshabilitado por seguridad
MulticastDNS=no
```

Link-Local Multicast Name Resolution deshabilitado

LLMNR=no

Configuración del enlace simbólico de resolv.conf

Enlazar resolv.conf a systemd-resolved

sudo ln -sf /run/systemd/resolve/stub-resolv.conf /etc/resolv.conf

Reiniciar el servicio

sudo systemctl restart systemd-resolved

Habilitar en el arranque

sudo systemctl enable systemd-resolved

Comandos de Verificación DNS

Verificar estado del servicio

sudo systemctl status systemd-resolved

Ver configuración actual de DNS

resolvectl status

Verificar que DoT está funcionando

resolvectl query ejemplo.com

Comprobar DNSSEC de un dominio

resolvectl query --type=DNSKEY ejemplo.com

Estadísticas de caché y consultas

resolvectl statistics

Verificar conectividad DNS con DoT

sudo tcpdump -i eth0 port 853 -c 5

Prueba de resolución con DNSSEC

dig +dnssec ejemplo.com

Verificar que no hay fugas DNS (debe mostrar los DNS configurados)

resolvectl dns

5.1.3. VPN con WireGuard

WireGuard es un protocolo VPN que destaca por su simplicidad, velocidad y seguridad. Utiliza criptografía moderna (Curve25519, ChaCha20, Poly1305) y tiene una base de código muy reducida.

Instalación

Instalar WireGuard

```
sudo apt update  
sudo apt install wireguard wireguard-tools
```

Verificar que el módulo del kernel está cargado

```
lsmod | grep wireguard
```

Generación de Claves

Crear directorio con permisos restrictivos

```
sudo mkdir -p /etc/wireguard  
sudo chmod 700 /etc/wireguard
```

Generar par de claves para el servidor

```
wg genkey | sudo tee /etc/wireguard/server_private.key | wg pubkey | sudo tee  
/etc/wireguard/server_public.key
```

Establecer permisos seguros para la clave privada

```
sudo chmod 600 /etc/wireguard/server_private.key
```

Generar claves para un cliente

```
wg genkey | tee client_private.key | wg pubkey > client_public.key
```

Configuración del Servidor

Cree el archivo /etc/wireguard/wg0.conf:

```
# /etc/wireguard/wg0.conf  
# Configuración del servidor WireGuard
```

[Interface]

Dirección IP del servidor en la VPN

```
Address = 10.200.200.1/24
```

Puerto de escucha UDP

```
ListenPort = 51820
```

Clave privada del servidor (insertar contenido de server_private.key)

```
PrivateKey = CLAVE_PRIVADA_SERVIDOR_AQUI
```

Reglas de firewall automáticas (opcional)

```
PostUp = nft add rule inet filter input udp dport 51820 accept
```

```
PostUp = nft add rule inet filter forward iifname wg0 accept
```

```
PostDown = nft delete rule inet filter input udp dport 51820 accept
```

```
PostDown = nft delete rule inet filter forward iifname wg0 accept
```

Guardar configuración al apagar

```
SaveConfig = false
```


--- Clientes (Peers) ---

[Peer]

Nombre descriptivo: Cliente-Laptop

Clave pública del cliente

PublicKey = CLAVE_PUBLICA_CLIENTE_AQUI

IP permitida para este cliente

AllowedIPs = 10.200.200.2/32

Mantener conexión activa (útil para NAT)

PersistentKeepalive = 25

Configuración del Cliente

Cree el archivo en el dispositivo cliente:

/etc/wireguard/wg0.conf (cliente)

Configuración del cliente WireGuard

[Interface]

Dirección IP del cliente en la VPN

Address = 10.200.200.2/24

Clave privada del cliente

PrivateKey = CLAVE_PRIVADA_CLIENTE_AQUI

DNS a usar cuando la VPN está activa

DNS = 10.200.200.1

[Peer]

Clave pública del servidor

PublicKey = CLAVE_PUBLICA_SERVIDOR_AQUI

Dirección pública del servidor

Endpoint = servidor.ejemplo.com:51820

Rutas a enviar por la VPN

0.0.0.0/0 = todo el tráfico (full tunnel)

10.200.200.0/24 = solo tráfico de la VPN (split tunnel)

AllowedIPs = 10.200.200.0/24, 192.168.20.0/24

Mantener conexión activa

PersistentKeepalive = 25

Comandos de Gestión y Verificación

Iniciar la interfaz VPN

```
sudo wg-quick up wg0
```

Detener la interfaz VPN

```
sudo wg-quick down wg0
```

Habilitar inicio automático

```
sudo systemctl enable wg-quick@wg0
```

Ver estado de la conexión

```
sudo wg show
```

Ver estado detallado

```
sudo wg show wg0
```

Verificar que la interfaz está activa

```
ip addr show wg0
```

Probar conectividad

```
ping 10.200.200.1
```

Ver tráfico en tiempo real

```
sudo wg show wg0 transfer
```

Verificar rutas

```
ip route | grep wg0
```

5.2. SSH Hardening Moderno (OpenSSH 10+)

5.2.1. Eliminación de Claves DSA, Uso de Ed25519

Las claves **DSA** fueron deshabilitadas por defecto en OpenSSH 7.0 y eliminadas completamente en versiones recientes debido a vulnerabilidades conocidas.

Ed25519 es el algoritmo recomendado actualmente por ofrecer:

19. Mayor seguridad con claves más pequeñas (256 bits vs 3072+ bits de RSA)
20. Rendimiento superior en generación y verificación
21. Resistencia a varios tipos de ataques de canal lateral

Generación de Claves Ed25519

Generar nueva clave Ed25519 con comentario descriptivo

```
ssh-keygen -t ed25519 -C "usuario@hostname-$(date +%Y%m%d)" -f ~/.ssh/id_ed25519
```

Opción con más rondas de derivación (más seguro, más lento)

```
ssh-keygen -t ed25519 -a 100 -C "usuario@servidor-produccion" -f  
~/.ssh/id_ed25519_produccion
```

```
# Verificar la clave generada  
ssh-keygen -l -f ~/.ssh/id_ed25519.pub
```

```
# Si necesita compatibilidad con sistemas antiguos, RSA 4096 es aceptable  
ssh-keygen -t rsa -b 4096 -C "usuario@legacy-$(date +%Y%m%d)" -f  
~/.ssh/id_rsa_legacy
```

Migración desde Claves Antiguas

```
# Listar claves existentes
```

```
ls -la ~/.ssh/
```

```
# Identificar y eliminar claves DSA (inseguras)  
rm ~/.ssh/id_dsa ~/.ssh/id_dsa.pub 2>/dev/null
```

```
# Verificar algoritmos de claves existentes
```

```
for key in ~/.ssh/id_*; do  
    [[ -f "$key" && ! "$key" =~ \.pub$ ]] && ssh-keygen -l -f "$key"  
done
```

```
# Copiar nueva clave pública al servidor  
ssh-copy-id -i ~/.ssh/id_ed25519.pub usuario@servidor
```

Configuración del Servidor SSH

Edite el archivo /etc/ssh/sshd_config:

```
# /etc/ssh/sshd_config  
# Configuración de seguridad SSH para Debian 13 Trixie  
# OpenSSH 10+
```

```
# === CONFIGURACIÓN DE RED ===
```

```
Port 22
```

```
AddressFamily inet
```

```
ListenAddress 0.0.0.0
```

```
# Para IPv6 también: ListenAddress ::
```

```
# === ALGORITMOS Y CLAVES ===
```

```
# Solo claves de host seguras (eliminar DSA y ECDSA débil)
```

```
HostKey /etc/ssh/ssh_host_ed25519_key
```

```
HostKey /etc/ssh/ssh_host_rsa_key
```

```
# Algoritmos de intercambio de claves (solo seguros)
```

```
KexAlgorithms sntrup761x25519-sha512@openssh.com,curve25519-  
sha256,curve25519-sha256@libssh.org
```

Cifrados permitidos (solo AEAD)

Ciphers chacha20-poly1305@openssh.com,aes256-gcm@openssh.com,aes128-gcm@openssh.com

Algoritmos MAC (solo encrypt-then-mac)

MACs hmac-sha2-512-etm@openssh.com,hmac-sha2-256-etm@openssh.com

Algoritmos de clave pública aceptados

PubkeyAcceptedAlgorithms ssh-ed25519,ssh-ed25519-cert-v01@openssh.com,rsa-sha2-512,rsa-sha2-256

Algoritmos de firma de CA

CASignatureAlgorithms ssh-ed25519,rsa-sha2-512,rsa-sha2-256

=== AUTENTICACIÓN ===

Deshabilitar autenticación por contraseña

PasswordAuthentication no

PermitEmptyPasswords no

Habilitar autenticación por clave pública

PubkeyAuthentication yes

AuthorizedKeysFile .ssh/authorized_keys

Deshabilitar métodos inseguros

ChallengeResponseAuthentication no

KerberosAuthentication no

GSSAPIAuthentication no

HostbasedAuthentication no

=== RESTRICCIONES DE ACCESO ===

Deshabilitar acceso root directo

PermitRootLogin no

Limitar usuarios permitidos (ajustar según necesidad)

AllowUsers admin operador

AllowGroups ssh-users

Número máximo de intentos de autenticación

MaxAuthTries 3

Tiempo máximo para autenticación

LoginGraceTime 30

Máximo de sesiones simultáneas

MaxSessions 3

Máximo de conexiones no autenticadas

MaxStartups 3:50:10

```
# === SEGURIDAD ADICIONAL ===
# Deshabilitar reenvío de agente SSH
AllowAgentForwarding no

# Deshabilitar reenvío TCP (habilitar solo si necesario)
AllowTcpForwarding no

# Deshabilitar reenvío X11
X11Forwarding no

# Deshabilitar túneles
PermitTunnel no

# No permitir variables de entorno del cliente
PermitUserEnvironment no

# Usar separación de privilegios
UsePrivilegeSeparation sandbox

# Mostrar banner antes de autenticación
Banner /etc/ssh/banner.txt

# Mensaje post-autenticación
PrintMotd no
PrintLastLog yes

# Verificar permisos de archivos del usuario
StrictModes yes

# Intervalo de verificación de conexión activa
ClientAliveInterval 300
ClientAliveCountMax 2

# === LOGGING ===
SyslogFacility AUTH
LogLevel VERBOSE

# === SFTP ===
Subsystem sftp internal-sftp -l INFO

# Configuración específica para grupo sftp-only
Match Group sftp-only
    ForceCommand internal-sftp -l INFO
    ChrootDirectory /home/%u
    AllowTcpForwarding no
    X11Forwarding no
    PermitTunnel no
```

Regenerar Claves de Host

Eliminar claves de host antiguas

```
sudo rm /etc/ssh/ssh_host_*
```

Regenerar solo claves seguras

```
sudo ssh-keygen -t ed25519 -f /etc/ssh/ssh_host_ed25519_key -N ""
```

```
sudo ssh-keygen -t rsa -b 4096 -f /etc/ssh/ssh_host_rsa_key -N ""
```

Ajustar permisos

```
sudo chmod 600 /etc/ssh/ssh_host_*_key
```

```
sudo chmod 644 /etc/ssh/ssh_host_*_key.pub
```

Comandos de Verificación SSH

Verificar sintaxis de configuración

```
sudo sshd -t
```

Ver configuración efectiva

```
sudo sshd -T
```

Reiniciar servicio (mantener sesión actual abierta como respaldo)

```
sudo systemctl restart sshd
```

Verificar estado

```
sudo systemctl status sshd
```

Ver conexiones activas

```
ss -tnlp | grep ssh
```

Probar conexión con verbosidad

```
ssh -v usuario@servidor
```

Verificar algoritmos soportados por el servidor

```
ssh -Q cipher
```

```
ssh -Q mac
```

```
ssh -Q kex
```

```
ssh -Q key
```

Auditar configuración SSH

```
sudo ssh-audit localhost
```

5.2.2. Configuración FIDO2/U2F para Acceso SSH

FIDO2 y **U2F** permiten usar llaves de seguridad físicas (como YubiKey, SoloKey, o Nitrokey) para autenticación SSH, proporcionando un segundo factor imposible de robar remotamente.

Instalación de Dependencias

Instalar librerías necesarias

```
sudo apt install libfido2-1 libfido2-dev libfido2-tools
```

Verificar que el dispositivo es detectado

```
fido2-token -L
```

Configurar Reglas udev

Cree el archivo /etc/udev/rules.d/70-fido2.rules:

/etc/udev/rules.d/70-fido2.rules

Reglas udev para dispositivos FIDO2

YubiKey

```
KERNEL=="hidraw*", SUBSYSTEM=="hidraw", ATTRS{idVendor}=="1050",  
MODE="0660", GROUP="plugdev", TAG+="uaccess"
```

SoloKey

```
KERNEL=="hidraw*", SUBSYSTEM=="hidraw", ATTRS{idVendor}=="0483",  
MODE="0660", GROUP="plugdev", TAG+="uaccess"
```

Nitrokey

```
KERNEL=="hidraw*", SUBSYSTEM=="hidraw", ATTRS{idVendor}=="20a0",  
MODE="0660", GROUP="plugdev", TAG+="uaccess"
```

Google Titan

```
KERNEL=="hidraw*", SUBSYSTEM=="hidraw", ATTRS{idVendor}=="18d1",  
MODE="0660", GROUP="plugdev", TAG+="uaccess"
```

Aplicar reglas:

Recargar reglas udev

```
sudo udevadm control --reload-rules
```

```
sudo udevadm trigger
```

Añadir usuario al grupo plugdev

```
sudo usermod -aG plugdev $USER
```

Generar Clave SSH Respaldada por FIDO2

Clave residente (almacenada en el dispositivo)

Requiere PIN y toque físico

```
ssh-keygen -t ed25519-sk -O resident -O verify-required -C "fido2-$(date +  
%Y%m%d)" -f ~/.ssh/id_ed25519_sk
```

Clave no residente (handle almacenado en disco)

Solo requiere toque físico

```
ssh-keygen -t ed25519-sk -O no-touch-required -C "fido2-notoque" -f  
~/.ssh/id_ed25519_sk_notoque
```

Para compatibilidad con dispositivos más antiguos (ECDSA)

```
ssh-keygen -t ecdsa-sk -C "fido2-ecdsa-$(date +%Y%m%d)" -f ~/.ssh/id_ecdsa_sk
```

Las opciones disponibles son: - -O resident: Almacena la clave en el dispositivo (recuperable) - -O verify-required: Requiere PIN del dispositivo - -O no-touch-required: No requiere toque físico (menos seguro)

Exportar Clave Residente

Listar claves residentes en el dispositivo

```
ssh-keygen -K
```

Esto crea archivos id_ed25519_sk_rk en el directorio actual*

Configuración del Servidor para FIDO2

Añadir al archivo /etc/ssh/sshd_config:

Añadir soporte para claves FIDO2

```
PubkeyAcceptedAlgorithms ssh-ed25519,sk-ssh-ed25519@openssh.com,sk-ecdsa-sha2-nistp256@openssh.com
```

Configuración del Cliente SSH

Edite ~/.ssh/config:

~/.ssh/config

Configuración del cliente SSH con FIDO2

Host servidor-seguro

HostName servidor.ejemplo.com

User admin

IdentityFile ~/.ssh/id_ed25519_sk

IdentitiesOnly yes

Requerir confirmación de usuario para el agente

AddKeysToAgent confirm

Host *

Preferir claves Ed25519

IdentityFile ~/.ssh/id_ed25519

IdentityFile ~/.ssh/id_ed25519_sk

Verificar huella del servidor

VisualHostKey yes

Algoritmos preferidos

KexAlgorithms curve25519-sha256,curve25519-sha256@libssh.org

Ciphers chacha20-poly1305@openssh.com,aes256-gcm@openssh.com

MACs hmac-sha2-512-etm@openssh.com,hmac-sha2-256-etm@openssh.com

Verificación de FIDO2

Verificar que el dispositivo está conectado

fido2-token -L

Ver información del dispositivo

fido2-token -I /dev/hidraw0

Listar credenciales residentes

fido2-token -L -r /dev/hidraw0

Probar conexión SSH con FIDO2

ssh -v -i ~/.ssh/id_ed25519_sk usuario@servidor

El sistema solicitará tocar la llave física

5.3. Servicios Web Seguros (nginx/Apache)

5.3.1. Cabeceras de Seguridad (CSP, HSTS)

Las **cabeceras de seguridad HTTP** instruyen a los navegadores sobre cómo manejar el contenido de forma segura. Son una capa de defensa esencial contra ataques como XSS, clickjacking y MIME-sniffing.

Configuración Principal de nginx

Cree el archivo /etc/nginx/conf.d/security-headers.conf:

```
# /etc/nginx/conf.d/security-headers.conf
```

```
# Cabeceras de seguridad globales para nginx
```

```
# HSTS - Forzar HTTPS durante 1 año
```

```
# includeSubDomains: aplica a todos los subdominios
```

```
# preload: permite inclusión en listas de precarga de navegadores
```

```
add_header Strict-Transport-Security "max-age=31536000; includeSubDomains; preload" always;
```

```
# CSP - Política de Seguridad de Contenido
```

```
# Esta es una política estricta; ajustar según necesidades de la aplicación
```

```
add_header Content-Security-Policy "
```

```
default-src 'self';
```

```
script-src 'self' 'unsafe-inline' https://cdn.ejemplo.com;
```

```
style-src 'self' 'unsafe-inline' https://fonts.googleapis.com;
```

```
img-src 'self' data: https;;
```

```
font-src 'self' https://fonts.gstatic.com;
```

```
connect-src 'self' https://api.ejemplo.com;
```

```
frame-ancestors 'self';
```

```
form-action 'self';
```

```

    base-uri 'self';
    upgrade-insecure-requests;
" always;

# X-Content-Type-Options - Prevenir MIME-sniffing
add_header X-Content-Type-Options "nosniff" always;

# X-Frame-Options - Prevenir clickjacking (redundante con CSP frame-ancestors)
add_header X-Frame-Options "SAMEORIGIN" always;

# X-XSS-Protection - Filtro XSS del navegador (legacy, deshabilitado en
navegadores modernos)
add_header X-XSS-Protection "0" always;

# Referrer-Policy - Controlar información de referencia enviada
add_header Referrer-Policy "strict-origin-when-cross-origin" always;

# Permissions-Policy - Controlar APIs del navegador
add_header Permissions-Policy "
    accelerometer=(),
    camera=(),
    geolocation=(),
    gyroscope=(),
    magnetometer=(),
    microphone=(),
    payment=(),
    usb=()
" always;

# Cross-Origin-Embedder-Policy
add_header Cross-Origin-Embedder-Policy "require-corp" always;

# Cross-Origin-Opener-Policy
add_header Cross-Origin-Opener-Policy "same-origin" always;

# Cross-Origin-Resource-Policy
add_header Cross-Origin-Resource-Policy "same-origin" always;

```

Explicación de Directivas CSP Principales

Directiva	Propósito
default-src	Política por defecto para todos los recursos
script-src	Orígenes permitidos para JavaScript
style-src	Orígenes permitidos para CSS
img-src	Orígenes permitidos para imágenes
connect-src	Orígenes para conexiones (fetch, WebSocket)

Directiva	Propósito
frame-ancestors	Quién puede incrustar la página en iframe
form-action	Destinos permitidos para formularios
upgrade-insecure-requests	Convertir HTTP a HTTPS automáticamente

5.3.2. TLS 1.3 y Ocultación de Versión

TLS 1.3 es la versión más reciente del protocolo, que elimina algoritmos obsoletos y reduce la latencia de conexión.

Configuración Completa del Servidor nginx

Cree el archivo `/etc/nginx/sites-available/ejemplo-seguro`:

```
# /etc/nginx/sites-available/ejemplo-seguro
# Configuración de servidor nginx con TLS 1.3

# Redirigir HTTP a HTTPS
server {
    listen 80;
    listen [::]:80;
    server_name ejemplo.com www.ejemplo.com;

    # Redirigir todo el tráfico a HTTPS
    return 301 https://$host$request_uri;
}

# Servidor HTTPS principal
server {
    listen 443 ssl;
    listen [::]:443 ssl;
    http2 on;

    server_name ejemplo.com www.ejemplo.com;

    # === OCULTAR INFORMACIÓN DEL SERVIDOR ===
    server_tokens off;

    # Eliminar cabecera Server (requiere módulo headers-more)
    # more_clear_headers Server;

    # === CERTIFICADOS TLS ===
    ssl_certificate /etc/letsencrypt/live/ejemplo.com/fullchain.pem;
    ssl_certificate_key /etc/letsencrypt/live/ejemplo.com/privkey.pem;
```

```
# Certificado de cadena para OCSP Stapling
ssl_trusted_certificate /etc/letsencrypt/live/ejemplo.com/chain.pem;

# === CONFIGURACIÓN TLS ===
# Solo TLS 1.3 (o incluir 1.2 para compatibilidad)
ssl_protocols TLSv1.3 TLSv1.2;

# Preferir cifrados del servidor
ssl_prefer_server_ciphers off;

# Cifrados para TLS 1.2 (TLS 1.3 usa sus propios)
ssl_ciphers ECDHE-ECDSA-AES128-GCM-SHA256:ECDHE-RSA-AES128-GCM-
SHA256:ECDHE-ECDSA-AES256-GCM-SHA384:ECDHE-RSA-AES256-GCM-
SHA384:ECDHE-ECDSA-CHACHA20-POLY1305:ECDHE-RSA-CHACHA20-
POLY1305;

# Curvas elípticas permitidas
ssl_ecdh_curve X25519:secp384r1;

# Parámetros DH personalizados (generar con: openssl dhparam -out
/etc/nginx/dhparam.pem 4096)
ssl_dhparam /etc/nginx/dhparam.pem;

# === SESIONES TLS ===
# Caché de sesiones compartida
ssl_session_cache shared:TLS:10m;
ssl_session_timeout 1d;

# Deshabilitar tickets de sesión (mejor seguridad)
ssl_session_tickets off;

# === OCSP STAPLING ===
ssl_stapling on;
ssl_stapling_verify on;
resolver 9.9.9.9 149.112.112.112 valid=300s;
resolver_timeout 5s;

# === CABECERAS DE SEGURIDAD ===
include /etc/nginx/conf.d/security-headers.conf;

# === RAÍZ Y LOGS ===
root /var/www/ejemplo.com/html;
index index.html index.htm;

access_log /var/log/nginx/ejemplo.com.access.log;
error_log /var/log/nginx/ejemplo.com.error.log warn;
```

```
# === UBICACIONES ===
location / {
    try_files $uri $uri/ =404;
}

# Bloquear archivos ocultos
location ~ /\. {
    deny all;
    access_log off;
    log_not_found off;
}

# Seguridad para archivos sensibles
location ~* \.(git|env|htaccess|htpasswd|ini|log|sh|sql|conf)$ {
    deny all;
}
}
```

Generar Parámetros DH Seguros

```
# Generar parámetros Diffie-Hellman (puede tardar varios minutos)
sudo openssl dhparam -out /etc/nginx/dhparam.pem 4096
sudo chmod 600 /etc/nginx/dhparam.pem
```

Comandos de Verificación nginx

```
# Verificar sintaxis de configuración
sudo nginx -t
```

```
# Recargar configuración
sudo systemctl reload nginx
```

```
# Ver estado del servicio
sudo systemctl status nginx
```

```
# Verificar módulos compilados
nginx -V 2>&1 | grep -o 'with-[^ ]*' | sort
```

```
# Verificar puertos en escucha
ss -tlnp | grep nginx
```

```
# Probar configuración TLS con OpenSSL
openssl s_client -connect ejemplo.com:443 -tls1_3
```

```
# Verificar certificado
openssl s_client -connect ejemplo.com:443 -servername ejemplo.com 2>/dev/null |
openssl x509 -noout -dates
```

```
# Verificar OCSP Stapling
openssl s_client -connect ejemplo.com:443 -status 2>/dev/null | grep -A 10 "OCSP"
```

Response"

```
# Escanear con testssl.sh (herramienta externa)
# git clone --depth 1 https://github.com/drwetter/testssl.sh.git
# ./testssl.sh/testssl.sh https://ejemplo.com
```

```
# Verificar cabeceras de seguridad
curl -I https://ejemplo.com
```

```
# Comprobar HSTS
curl -sI https://ejemplo.com | grep -i strict
```

```
# Ver logs de acceso en tiempo real
sudo tail -f /var/log/nginx/ejemplo.com.access.log
```

5.3.3. Apache HTTPd - Cabeceras de Seguridad y TLS 1.3

Apache HTTPd es el servidor web más popular y ampliamente utilizado. Ofrece capacidades equivalentes a nginx para seguridad, con una configuración y modularidad diferentes.

Instalación

```
# Instalar Apache y módulos necesarios
sudo apt install apache2 apache2-utils openssl
```

```
# Habilitar módulos esenciales
```

```
sudo a2enmod ssl
sudo a2enmod headers
sudo a2enmod rewrite
sudo a2enmod remoteip
```

Configuración de Cabeceras de Seguridad en Apache

Cree el archivo /etc/apache2/mods-available/security-headers.conf:

```
# /etc/apache2/mods-available/security-headers.conf
# Cabeceras de seguridad globales para Apache
```

```
# HSTS - Forzar HTTPS durante 1 año
```

```
Header always set Strict-Transport-Security "max-age=31536000;
includeSubDomains; preload"
```

```
# CSP - Política de Seguridad de Contenido
```

```
Header always set Content-Security-Policy "default-src 'self'; script-src 'self'
'unsafe-inline' https://cdn.ejemplo.com; style-src 'self' 'unsafe-inline'
https://fonts.googleapis.com; img-src 'self' data: https:; font-src 'self'
https://fonts.gstatic.com; connect-src 'self' https://api.ejemplo.com; frame-
```

```
ancestors 'self'; form-action 'self'; base-uri 'self'; upgrade-insecure-requests;"
```

```
# X-Content-Type-Options - Prevenir MIME-sniffing
```

```
Header always set X-Content-Type-Options "nosniff"
```

```
# X-Frame-Options - Prevenir clickjacking
```

```
Header always set X-Frame-Options "SAMEORIGIN"
```

```
# X-XSS-Protection - Filtro XSS del navegador (legacy)
```

```
Header always set X-XSS-Protection "0"
```

```
# Referrer-Policy - Controlar información de referencia
```

```
Header always set Referrer-Policy "strict-origin-when-cross-origin"
```

```
# Permissions-Policy - Controlar APIs del navegador
```

```
Header always set Permissions-Policy "accelerometer=(), camera=(),  
geolocation=(), gyroscope=(), magnetometer=(), microphone=(), payment=(),  
usb=()"
```

```
# Cross-Origin policies
```

```
Header always set Cross-Origin-Embedder-Policy "require-corp"
```

```
Header always set Cross-Origin-Opener-Policy "same-origin"
```

```
Header always set Cross-Origin-Resource-Policy "same-origin"
```

```
# Ocultar versión del servidor
```

```
Header always set Server "Apache"
```

```
Header always set X-Powered-By "Apache"
```

Habilitar el módulo:

```
# Crear enlace simbólico
```

```
sudo ln -s /etc/apache2/mods-available/security-headers.conf /etc/apache2/mods-  
enabled/
```

```
# O incluirlo en la configuración del sitio
```

```
# Include /etc/apache2/mods-available/security-headers.conf
```

Configuración Completa del Virtual Host con TLS 1.3

Cree el archivo /etc/apache2/sites-available/ejemplo-seguro-ssl.conf:

```
# /etc/apache2/sites-available/ejemplo-seguro-ssl.conf
```

```
# Configuración de Apache con TLS 1.3
```

```
# Redirigir HTTP a HTTPS
```

```
<VirtualHost *:80>
```

```
    ServerName ejemplo.com
```

```
    ServerAlias www.ejemplo.com
```

```
    DocumentRoot /var/www/ejemplo.com/html
```

```
# Redirigir todo a HTTPS
RewriteEngine On
RewriteCond %{HTTPS} off
RewriteRule ^(.*)$ https://%{HTTP_HOST}%{REQUEST_URI} [L,R=301]

ErrorLog ${APACHE_LOG_DIR}/ejemplo.com.error.log
CustomLog ${APACHE_LOG_DIR}/ejemplo.com.access.log combined
</VirtualHost>

# Servidor HTTPS principal
<VirtualHost *:443>
    ServerName ejemplo.com
    ServerAlias www.ejemplo.com
    DocumentRoot /var/www/ejemplo.com/html

    # === OCULTAR INFORMACIÓN DEL SERVIDOR ===
    Header always set Server "Apache"
    Header always edit Set-Cookie ^(.*)$ "$1; HttpOnly; Secure; SameSite=Strict"

    # Eliminar cabeceras que revelan información
    Header always unset X-Powered-By
    Header always unset X-Runtime
    Header always unset X-Aspnet-Version

    # === SSL/TLS CONFIGURATION ===
    SSLEngine on
    SSLCertificateFile /etc/letsencrypt/live/ejemplo.com/fullchain.pem
    SSLCertificateKeyFile /etc/letsencrypt/live/ejemplo.com/privkey.key
    SSLCertificateChainFile /etc/letsencrypt/live/ejemplo.com/chain.pem

    # === PROTOCOLOS TLS ===
    # Solo TLS 1.3 (o incluir 1.2 para compatibilidad)
    SSLProtocol -all +TLSv1.3 +TLSv1.2

    # Preferir cifrados del servidor
    SSLHonorCipherOrder off

    # Cifrados para TLS 1.2 (TLS 1.3 usa sus propios)
    SSLCipherSuite ECDHE-ECDSA-AES128-GCM-SHA256:ECDHE-RSA-AES128-
GCM-SHA256:ECDHE-ECDSA-AES256-GCM-SHA384:ECDHE-RSA-AES256-GCM-
SHA384:ECDHE-ECDSA-CHACHA20-POLY1305:ECDHE-RSA-CHACHA20-
POLY1305

    # Curvas elípticas permitidas
    SSLECDHCurve X25519:secp384r1
```


Parámetros DH personalizados

SSLOpenSSLConfCmd DHParameters "/etc/apache2/dhparam.pem"

=== SESIONES TLS ===

SSLSessionCache shmcb:/var/cache/apache2/ssl_scache(512000)

SSLSessionCacheTimeout 300

=== STAPLING OCSP ===

SSLUseStapling On

SSLStaplingCache shmcb:/var/cache/apache2/ssl_stapling(32768)

SSLStaplingResponderTimeout 5

SSLStaplingReturnResponderErrors off

=== HTTP/2 ===

Protocols h2 http/1.1

=== CABECERAS DE SEGURIDAD ===

Header always set Strict-Transport-Security "max-age=31536000;
includeSubDomains; preload"

Header always set Content-Security-Policy "default-src 'self'; script-src 'self'
'unsafe-inline' https://cdn.ejemplo.com; style-src 'self' 'unsafe-inline'
https://fonts.googleapis.com; img-src 'self' data: https;; font-src 'self'
https://fonts.gstatic.com; connect-src 'self' https://api.ejemplo.com; frame-
ancestors 'self'; form-action 'self'; base-uri 'self'; upgrade-insecure-requests;"

Header always set X-Content-Type-Options "nosniff"

Header always set X-Frame-Options "SAMEORIGIN"

Header always set X-XSS-Protection "0"

Header always set Referrer-Policy "strict-origin-when-cross-origin"

Header always set Permissions-Policy "accelerometer=(), camera=(),
geolocation=(), gyroscope=(), magnetometer=(), microphone=(), payment=(),
usb=()"

Header always set Cross-Origin-Embedder-Policy "require-corp"

Header always set Cross-Origin-Opener-Policy "same-origin"

Header always set Cross-Origin-Resource-Policy "same-origin"

=== LOGS ===

ErrorLog \${APACHE_LOG_DIR}/ejemplo.com.error.log

CustomLog \${APACHE_LOG_DIR}/ejemplo.com.access.log combined

=== UBICACIONES ===

<Directory /var/www/ejemplo.com/html>

Options **-Indexes +FollowSymLinks**

AllowOverride **None**

Require all granted

</Directory>

Bloquear acceso a archivos ocultos

```
<FilesMatch "^\\.\\.env$|\\.git|\\.htaccess|\\.htpasswd">
    Require all denied
</FilesMatch>

# Seguridad para archivos sensibles
<FilesMatch "\\.(ini|log|sh|sql|conf)$">
    Require all denied
</FilesMatch>
</VirtualHost>
```

Generar Parámetros DH para Apache
Generar parámetros Diffie-Hellman (puede tardar varios minutos)
sudo openssl dhparam -out /etc/apache2/dhparam.pem 4096
sudo chmod 644 /etc/apache2/dhparam.pem

Habilitar el Sitio y Configuración
Crear enlace simbólico del sitio
sudo a2ensite ejemplo-seguro-ssl

Deshabilitar sitio default si es necesario
sudo a2dissite 000-default

Verificar sintaxis
sudo apache2ctl configtest
Debe mostrar: Syntax OK

Comandos de Verificación Apache
Verificar sintaxis de configuración
sudo apache2ctl configtest

Verificar sintaxis con más detalle
sudo apache2ctl -t -D DUMP_CONFIG

Recargar configuración
sudo systemctl reload apache2

Ver estado del servicio
sudo systemctl status apache2

Verificar módulos habilitados
apache2ctl -M

Verificar módulos SSL específicamente
apache2ctl -M | grep ssl

Verificar puertos en escucha
ss -tlnp | grep apache

Probar configuración TLS con OpenSSL

```
openssl s_client -connect ejemplo.com:443 -tls1_3
```

Verificar protocolo y cifrados

```
openssl s_client -connect ejemplo.com:443 -tls1_3 </dev/null 2>/dev/null | grep -E "Protocol|Cipher"
```

Verificar certificado

```
openssl s_client -connect ejemplo.com:443 -servername ejemplo.com 2>/dev/null | openssl x509 -noout -dates
```

Verificar OCSP Stapling

```
openssl s_client -connect ejemplo.com:443 -status 2>/dev/null | grep -A 5 "OCSP"
```

Verificar cabeceras de seguridad

```
curl -I https://ejemplo.com
```

Comprobar HSTS

```
curl -sI https://ejemplo.com | grep -i strict
```

Comprobar CSP

```
curl -sI https://ejemplo.com | grep -i content-security
```

Ver versión de Apache

```
apache2 -v
```

Ver configuración cargada del módulo SSL

```
openssl ciphers -v | head -20
```

Ver logs en tiempo real

```
sudo tail -f /var/log/apache2/ejemplo.com.access.log
```

Buscar errores en logs

```
sudo grep "error|warn" /var/log/apache2/ejemplo.com.error.log | tail -20
```

Probar con herramientas de escaneo

testssl.sh

```
./testssl.sh/testssl.sh https://ejemplo.com
```

Comparación nginx vs Apache

Aspecto	nginx	Apache
Cabeceras HSTS	add_header en bloque server	Header always set en VirtualHost
CSP	add_header Content-Security-Policy	Header always set Content-Security-Policy
Ocultación versión	server_tokens off;	Header always unset Server;

Aspecto	nginx	Apache
TLS 1.3	ssl_protocols TLSv1.3 TLSv1.2;	SSLProtocol +TLSv1.3 +TLSv1.2;
Cifrados	ssl_ciphers ...;	SSLCipherSuite ...
Curvas elípticas	ssl_ecdh_curve ...;	SSLECDHCurve ...
OCSP Stapling	ssl_stapling on;	SSLUseStapling On;
Verificación sintaxis	nginx -t	apache2ctl configtest
Recarga	systemctl reload nginx	systemctl reload apache2
Módulos	Compilados en binario	a2enmod / a2dismod

5.4. Gestión de Paquetes y Vulnerabilidades

5.4.1. Formato deb822 (/etc/apt/sources.list.d/debian.sources)

El formato **deb822** es el nuevo estándar en Debian para configurar repositorios APT. Ofrece una sintaxis más clara y estructurada que el formato tradicional de una línea.

Migración al Formato deb822

Respalda configuración actual

```
sudo cp /etc/apt/sources.list /etc/apt/sources.list.backup
```

Crear nuevo archivo de fuentes

```
sudo touch /etc/apt/sources.list.d/debian.sources
```

Vaciar el archivo antiguo (opcional, después de verificar)

```
# sudo truncate -s 0 /etc/apt/sources.list
```

Configuración de Repositorios

Cree el archivo /etc/apt/sources.list.d/debian.sources:

```
# /etc/apt/sources.list.d/debian.sources
```

Configuración de repositorios Debian 13 Trixie en formato deb822

Repositorio principal de Debian

Types: **deb deb-src**

URIs: **https://deb.debian.org/debian**

Suites: **trixie trixie-updates**

Components: **main contrib non-free non-free-firmware**

Signed-By: **/usr/share/keyrings/debian-archive-keyring.gpg**

Architectures: **amd64**

Actualizaciones de seguridad

Types: **deb deb-src**

URIs: **https://security.debian.org/debian-security**

Suites: **trixie-security**
 Components: **main contrib non-free non-free-firmware**
 Signed-By: **/usr/share/keyrings/debian-archive-keyring.gpg**
 Architectures: **amd64**

Repositorio Backports (versiones más recientes de software)

Types: **deb deb-src**
 URIs: **https://deb.debian.org/debian**
 Suites: **trixie-backports**
 Components: **main contrib non-free non-free-firmware**
 Signed-By: **/usr/share/keyrings/debian-archive-keyring.gpg**
 Architectures: **amd64**

Campos del Formato deb822

Campo	Descripción
Types	Tipo de paquetes: deb (binarios) y/o deb-src (fuentes)
URIs	URLs de los repositorios (puede incluir múltiples)
Suites	Nombres de las distribuciones o ramas
Components	Secciones del repositorio (main, contrib, non-free)
Signed-By	Ruta al archivo de clave GPG para verificación
Architectures	Arquitecturas de CPU (amd64, arm64, i386)
Enabled	yes o no para activar/desactivar el repositorio

Añadir Repositorio de Terceros (Ejemplo)

/etc/apt/sources.list.d/docker.sources
Repositorio oficial de Docker

Types: **deb**
 URIs: **https://download.docker.com/linux/debian**
 Suites: **trixie**
 Components: **stable**
 Signed-By: **/usr/share/keyrings/docker-archive-keyring.gpg**
 Architectures: **amd64**

Descargar la clave GPG:

Descargar y almacenar clave GPG
curl -fsSL https://download.docker.com/linux/debian/gpg | sudo gpg --dearmor -o /usr/share/keyrings/docker-archive-keyring.gpg

Verificar permisos

```
sudo chmod 644 /usr/share/keyrings/docker-archive-keyring.gpg
```

Verificación de Repositorios

Actualizar lista de paquetes

```
sudo apt update
```

Verificar que no hay errores

```
apt-cache policy
```

Ver repositorios configurados

```
apt-cache policy | grep -E '^\s*(release|origin)'
```

Listar paquetes de un repositorio específico

```
apt list --upgradable 2>/dev/null
```

Verificar firmas de paquetes

```
apt-key list 2>/dev/null # Método legacy
```

```
gpg --list-keys --keyring /usr/share/keyrings/debian-archive-keyring.gpg
```

5.4.2. unattended-upgrades y apt-listbugs

unattended-upgrades automatiza la instalación de actualizaciones de seguridad, mientras que **apt-listbugs** advierte sobre bugs conocidos antes de instalar paquetes.

Instalación

Instalar herramientas

```
sudo apt install unattended-upgrades apt-listbugs apt-listchanges needrestart
```

Configurar automáticamente

```
sudo dpkg-reconfigure -plow unattended-upgrades
```

Configuración de unattended-upgrades

Edite el archivo /etc/apt/apt.conf.d/50unattended-upgrades:

```
# /etc/apt/apt.conf.d/50unattended-upgrades
```

```
# Configuración de actualizaciones automáticas
```

```
// Orígenes permitidos para actualizaciones automáticas
```

```
Unattended-Upgrade::Origins-Pattern {
```

```
    // Actualizaciones de seguridad de Debian
```

```
    "origin=Debian,codename=${distro_codename}-security,label=Debian-Security";
```

```
    // Actualizaciones estables de Debian
```

```
"origin=Debian,codename=${distro_codename},label=Debian";

// Actualizaciones del sistema
"origin=Debian,codename=${distro_codename}-updates,label=Debian";
};

// Paquetes a NO actualizar automáticamente (lista negra)
Unattended-Upgrade::Package-Blacklist {
    // "linux-image";
    // "linux-headers";
    // "nginx";
};

// Opciones de correo electrónico
Unattended-Upgrade::Mail "admin@ejemplo.com";
Unattended-Upgrade::MailReport "on-change";

// Eliminar dependencias no utilizadas automáticamente
Unattended-Upgrade::Remove-Unused-Dependencies "true";

// Eliminar kernels no utilizados
Unattended-Upgrade::Remove-Unused-Kernel-Packages "true";

// Reiniciar automáticamente si es necesario (usar con precaución)
Unattended-Upgrade::Automatic-Reboot "false";

// Si se habilita reinicio automático, hora preferida
Unattended-Upgrade::Automatic-Reboot-Time "03:00";

// Limitar ancho de banda de descarga (KB/s, 0 = ilimitado)
Acquire::http::Dl-Limit "0";

// Registro detallado
Unattended-Upgrade::Verbose "true";

// Modo de prueba (solo simular, no instalar)
// Unattended-Upgrade::Dry-Run "true";

// No interrumpir aplicaciones dpkg en ejecución
Unattended-Upgrade::InstallOnShutdown "false";

// Solo actualizar si hay suficiente espacio en disco (MB)
Unattended-Upgrade::MinimalSteps "true";
```

Configuración de Periodicidad

Edite el archivo `/etc/apt/apt.conf.d/20auto-upgrades`:

```
# /etc/apt/apt.conf.d/20auto-upgrades
# Periodicidad de actualizaciones automáticas

// Actualizar lista de paquetes diariamente
APT::Periodic::Update-Package-Lists "1";

// Descargar paquetes actualizables diariamente
APT::Periodic::Download-Upgradeable-Packages "1";

// Ejecutar actualizaciones automáticas diariamente
APT::Periodic::Unattended-Upgrade "1";

// Limpiar caché de paquetes cada 7 días
APT::Periodic::AutocleanInterval "7";
```

Configuración de apt-listbugs

Edite el archivo /etc/apt/apt.conf.d/10apt-listbugs:

```
# /etc/apt/apt.conf.d/10apt-listbugs
# Configuración de apt-listbugs

// Severidad mínima para mostrar bugs
Apt::Listbugs::Severities "critical,grave,serious";

// Acción al encontrar bugs (ask, ignore, both)
Apt::Listbugs::Action "ask";
```

Comandos de Verificación y Gestión

```
# Verificar configuración de actualizaciones automáticas
apt-config dump | grep -i unattended
```

```
# Ejecutar actualización manual en modo prueba
sudo unattended-upgrade --dry-run --debug
```

```
# Ejecutar actualización real
sudo unattended-upgrade -v
```

```
# Ver logs de actualizaciones automáticas
sudo cat /var/log/unattended-upgrades/unattended-upgrades.log
```

```
# Verificar estado del servicio
sudo systemctl status unattended-upgrades
```

```
# Ver bugs conocidos para un paquete
apt-listbugs list nombre-paquete
```

```
# Verificar si hay reinicios pendientes
```



```
sudo needrestart -r l
```

```
# Listar servicios que necesitan reinicio
```

```
sudo needrestart -b
```

```
# Ver actualizaciones pendientes
```

```
apt list --upgradable
```

5.5. Monitorización y Logs

5.5.1. journald (Logs Firmados) y auditd

Configuración de journald con Logs Firmados

journald puede firmar criptográficamente los logs para detectar manipulaciones. Esta característica utiliza FSS (Forward Secure Sealing).

Edite el archivo /etc/systemd/journald.conf:

```
# /etc/systemd/journald.conf
```

```
# Configuración de journald con logs firmados
```

[Journal]

```
# Almacenamiento persistente de logs
```

```
Storage=persistent
```

```
# Comprimir logs
```

```
Compress=yes
```

```
# Habilitar sellado criptográfico (Forward Secure Sealing)
```

```
Seal=yes
```

```
# Dividir logs por usuario
```

```
SplitMode=uid
```

```
# Sincronizar al disco cada 5 minutos
```

```
SyncIntervalSec=5m
```

```
# Límites de tamaño
```

```
SystemMaxUse=2G
```

```
SystemKeepFree=1G
```

```
SystemMaxFileSize=128M
```

```
SystemMaxFiles=100
```

```
# Límites para logs de usuario
```

```
RuntimeMaxUse=256M
```

```
RuntimeKeepFree=128M
```

RuntimeMaxFileSize=32M

Nivel mínimo de log

MaxLevelStore=debug

MaxLevelSyslog=debug

MaxLevelKMsg=warning

MaxLevelConsole=info

Tasa de limitación de mensajes

RateLimitIntervalSec=30s

RateLimitBurst=10000

Reenviar a syslog (si se usa rsyslog también)

ForwardToSyslog=no

ForwardToKMsg=no

ForwardToConsole=no

ForwardToWall=yes

Configurar Sellado FSS

Generar claves de sellado FSS

sudo journalctl --setup-keys

Esto genera:

- Una clave de verificación (guardar fuera del sistema)

- Las claves de sellado se rotan automáticamente

Verificar integridad de logs

sudo journalctl --verify

Ver clave de verificación

sudo journalctl --verify-key

Configuración de auditd

Instalar y configurar auditd:

Instalar auditd

sudo apt install auditd audispd-plugins

Habilitar servicio

sudo systemctl enable auditd

sudo systemctl start auditd

Edite el archivo /etc/audit/auditd.conf:

/etc/audit/auditd.conf

Configuración principal de auditd

Archivo de log

```
log_file = /var/log/audit/audit.log
log_group = adm
log_format = ENRICHED
```

```
# Rotación de logs
```

```
max_log_file = 50
num_logs = 5
max_log_file_action = ROTATE
```

```
# Comportamiento cuando el disco está lleno
```

```
space_left = 100
space_left_action = SYSLOG
admin_space_left = 50
admin_space_left_action = SUSPEND
disk_full_action = SUSPEND
disk_error_action = SUSPEND
```

```
# Flush de escritura
```

```
flush = INCREMENTAL_ASYNC
freq = 50
```

```
# Prioridad del demonio
```

```
priority_boost = 4
```

```
# Tamaño del buffer
```

```
name_format = HOSTNAME
name = debian-server
```

Reglas de Auditoría

Cree el archivo /etc/audit/rules.d/security.rules:

```
# /etc/audit/rules.d/security.rules
# Reglas de auditoría de seguridad para Debian 13
```

```
# Eliminar reglas anteriores
-D
```

```
# Aumentar buffer de auditoría
-b 8192
```

```
# Hacer reglas inmutables (requiere reinicio para cambiar)
# Descomentar en producción:
# -e 2
```

```
# === MONITOREO DE ARCHIVOS CRÍTICOS ===
```

```
# Cambios en passwd, shadow, group
```

```
-w /etc/passwd -p wa -k identity  
-w /etc/shadow -p wa -k identity  
-w /etc/group -p wa -k identity  
-w /etc/gshadow -p wa -k identity
```

Cambios en sudoers

```
-w /etc/sudoers -p wa -k sudoers  
-w /etc/sudoers.d/ -p wa -k sudoers
```

Cambios en configuración SSH

```
-w /etc/ssh/sshd_config -p wa -k sshd_config  
-w /etc/ssh/ssh_config -p wa -k ssh_config
```

Cambios en PAM

```
-w /etc/pam.d/ -p wa -k pam
```

Cambios en cron

```
-w /etc/crontab -p wa -k cron  
-w /etc/cron.d/ -p wa -k cron  
-w /var/spool/cron/ -p wa -k cron
```

Cambios en systemd

```
-w /etc/systemd/ -p wa -k systemd
```

Cambios en configuración de red

```
-w /etc/hosts -p wa -k network  
-w /etc/network/ -p wa -k network  
-w /etc/netplan/ -p wa -k network
```

=== LLAMADAS AL SISTEMA CRÍTICAS ===

Cambios de tiempo del sistema

```
-a always,exit -F arch=b64 -S adjtimex -S settimeofday -S clock_settime -k  
time_change
```

Creación y eliminación de usuarios/grupos

```
-a always,exit -F arch=b64 -S setuid -S setgid -S setreuid -S setregid -k  
identity_change
```

Montaje de sistemas de archivos

```
-a always,exit -F arch=b64 -S mount -S umount2 -k mounts
```

Eliminación de archivos

```
-a always,exit -F arch=b64 -S unlink -S unlinkat -S rename -S renameat -k  
file_deletion
```

Modificación de atributos de archivos

```
-a always,exit -F arch=b64 -S chmod -S fchmod -S fchmodat -k file_permission
```

```
-a always,exit -F arch=b64 -S chown -S fchown -S fchownat -S lchown -k  
file_ownership
```

Uso de privilegios

```
-a always,exit -F arch=b64 -S execve -C uid!=euid -F key=privilege_escalation
```

```
# === ACCESO A ARCHIVOS SENSIBLES ===
```

Acceso a claves privadas SSH

```
-w /etc/ssh/ -p r -k ssh_keys
```

Acceso a certificados

```
-w /etc/ssl/private/ -p r -k ssl_keys
```

```
# === COMANDOS ESPECÍFICOS ===
```

Uso de comandos de administración

```
-w /usr/bin/passwd -p x -k passwd_cmd  
-w /usr/bin/sudo -p x -k sudo_cmd  
-w /usr/bin/su -p x -k su_cmd  
-w /usr/sbin/useradd -p x -k user_management  
-w /usr/sbin/userdel -p x -k user_management  
-w /usr/sbin/usermod -p x -k user_management  
-w /usr/sbin/groupadd -p x -k group_management  
-w /usr/sbin/groupdel -p x -k group_management  
-w /usr/sbin/groupmod -p x -k group_management
```

Herramientas de red

```
-w /usr/bin/ssh -p x -k network_tools  
-w /usr/bin/scp -p x -k network_tools  
-w /usr/bin/wget -p x -k network_tools  
-w /usr/bin/curl -p x -k network_tools
```

Cargar las reglas:

Cargar reglas

```
sudo augenrules --load
```

Verificar reglas cargadas

```
sudo auditctl -l
```

Ver estado de auditd

```
sudo auditctl -s
```

Comandos de Verificación de Logs

```
# === JOURNALD ===
```

Ver logs del sistema

```
sudo journalctl -xe
```

Ver logs de un servicio específico

```
sudo journalctl -u nginx.service
```

Ver logs desde el último arranque

```
sudo journalctl -b
```

Ver logs en tiempo real

```
sudo journalctl -f
```

Filtrar por prioridad (0-7, 0=emergencia)

```
sudo journalctl -p err
```

Filtrar por fecha

```
sudo journalctl --since "2024-01-01" --until "2024-01-02"
```

Verificar integridad de logs firmados

```
sudo journalctl --verify
```

Ver espacio usado por logs

```
sudo journalctl --disk-usage
```

Limpiar logs antiguos

```
sudo journalctl --vacuum-time=30d
```

```
sudo journalctl --vacuum-size=1G
```

```
# === AUDITD ===
```

Ver eventos de auditoría recientes

```
sudo ausearch -ts recent
```

Buscar eventos por clave

```
sudo ausearch -k identity
```

Buscar eventos de un usuario

```
sudo ausearch -ua root
```

Buscar eventos de un comando

```
sudo ausearch -c sudo
```

Generar reporte de auditoría

```
sudo aureport
```

Reporte de autenticaciones

```
sudo aureport -au
```

Reporte de comandos ejecutados

```
sudo aureport -x
```

```
# Reporte de anomalías
```

```
sudo aureport --anomaly
```

```
# Ver estado del servicio
```

```
sudo systemctl status auditd
```

5.5.2. Alertas Básicas

Sistema de Alertas con Logwatch

```
# Instalar logwatch
```

```
sudo apt install logwatch
```

```
# Configurar logwatch
```

```
sudo cp /usr/share/logwatch/default.conf/logwatch.conf /etc/logwatch/conf/
```

```
# Editar configuración
```

```
sudo nano /etc/logwatch/conf/logwatch.conf
```

Configuración de /etc/logwatch/conf/logwatch.conf:

```
# /etc/logwatch/conf/logwatch.conf
```

```
# Configuración de Logwatch
```

```
# Enviar por correo
```

```
Output = mail
```

```
MailTo = admin@ejemplo.com
```

```
MailFrom = logwatch@ejemplo.com
```

```
# Nivel de detalle (0-10)
```

```
Detail = Med
```

```
# Rango de logs a analizar
```

```
Range = yesterday
```

```
# Formato de salida
```

```
Format = html
```

```
# Servicios a monitorear
```

```
Service = All
```

```
Service = -zz-network
```

```
Service = -zz-sys
```

Script de Alertas Personalizadas

Cree el archivo /usr/local/bin/alertas-seguridad.sh:

```
#!/bin/bash
# /usr/local/bin/alertas-seguridad.sh
# Script de alertas de seguridad básicas

# Configuración
EMAIL="admin@ejemplo.com"
HOSTNAME=$(hostname)
LOG_FILE="/var/log/alertas-seguridad.log"

# Función de envío de alerta
enviar_alerta() {
    local asunto="$1"
    local mensaje="$2"
    local timestamp=$(date '+%Y-%m-%d %H:%M:%S')

    echo "[$timestamp] ALERTA: $asunto" >> "$LOG_FILE"

    # Enviar por correo (requiere mailutils o msmtplib)
    echo -e "Servidor: $HOSTNAME\nFecha: $timestamp\n\n$mensaje" | \
        mail -s "[ALERTA] $asunto - $HOSTNAME" "$EMAIL"
}

# Verificar intentos de SSH fallidos
ssh_fallidos=$(journalctl -u ssh --since "1 hour ago" | grep -c "Failed password")
if [ "$ssh_fallidos" -gt 10 ]; then
    enviar_alerta "Múltiples intentos SSH fallidos" \
        "Se detectaron $ssh_fallidos intentos de acceso SSH fallidos en la última hora."
fi

# Verificar uso de disco
uso_disco=$(df / | awk 'NR==2 {print $5}' | tr -d '%')
if [ "$uso_disco" -gt 90 ]; then
    enviar_alerta "Disco casi lleno" \
        "El disco raíz está al ${uso_disco}% de capacidad."
fi

# Verificar carga del sistema
carga=$(uptime | awk -F'load average:' '{print $2}' | cut -d',' -f1 | tr -d ' ')
cpus=$(nproc)
umbral=$((echo "$cpus * 2" | bc))
if (( $(echo "$carga > $umbral" | bc -l) )); then
    enviar_alerta "Carga del sistema alta" \
        "La carga del sistema ($carga) supera el umbral ($umbral)."
fi

# Verificar servicios críticos
```



```

for servicio in ssh nginx; do
    if ! systemctl is-active --quiet "$servicio"; then
        enviar_alerta "Servicio $servicio caído" \
            "El servicio $servicio no está activo."
    fi
done

# Verificar cambios en archivos críticos (usando auditd)
cambios_passwd=$(ausearch -k identity --start recent 2>/dev/null | grep -c
"type=SYSCALL")
if [ "$cambios_passwd" -gt 0 ]; then
    enviar_alerta "Cambios en archivos de identidad" \
        "Se detectaron $cambios_passwd cambios en archivos de identidad del
sistema."
fi

# Verificar usuarios con UID 0 (además de root)
usuarios_uid0=$(awk -F: '$3 == 0 && $1 != "root" {print $1}' /etc/passwd)
if [ -n "$usuarios_uid0" ]; then
    enviar_alerta "Usuarios con UID 0 detectados" \
        "Usuarios con UID 0 además de root: $usuarios_uid0"
fi

echo "[$(date '+%Y-%m-%d %H:%M:%S')] Verificación completada" >>
"$LOG_FILE"

```

Configurar permisos y cron:

```

# Dar permisos de ejecución
sudo chmod +x /usr/local/bin/alertas-seguridad.sh

# Añadir a cron (cada hora)
echo "0 * * * * root /usr/local/bin/alertas-seguridad.sh" | sudo tee
/etc/cron.d/alertas-seguridad

# Verificar cron
sudo crontab -l

```

5.6. Copias de Seguridad 3-2-1 Cifradas

La **estrategia 3-2-1** es una metodología probada para copias de seguridad: - **3** copias de los datos importantes - **2** tipos diferentes de medios de almacenamiento - **1** copia almacenada fuera del sitio (offsite)

Implementación con Restic

Restic es una herramienta de backup moderna que soporta cifrado, deduplicación y múltiples backends de almacenamiento.

Instalación

Instalar restic

```
sudo apt install restic
```

Verificar versión

```
restic version
```

Inicializar Repositorios

Variables de entorno para la contraseña

```
export RESTIC_PASSWORD="contraseña-segura-compleja"
```

1. Repositorio local (disco externo)

```
sudo restic init --repo /mnt/backup-local
```

2. Repositorio remoto (servidor SSH)

```
sudo restic init --repo sftp:usuario@backup-server:/backups/$(hostname)
```

3. Repositorio en la nube (S3 compatible)

```
export AWS_ACCESS_KEY_ID="tu-access-key"
```

```
export AWS_SECRET_ACCESS_KEY="tu-secret-key"
```

```
sudo restic init --repo s3:s3.ejemplo.com/bucket-backup/$(hostname)
```

Script de Backup Automatizado

Cree el archivo /usr/local/bin/backup-321.sh:

```
#!/bin/bash
```

```
# /usr/local/bin/backup-321.sh
```

```
# Script de backup 3-2-1 cifrado con restic
```

```
set -e
```

```
# === CONFIGURACIÓN ===
```

```
HOSTNAME=$(hostname)
```

```
LOG_FILE="/var/log/backup-321.log"
```

```
EMAIL="admin@ejemplo.com"
```

```
# Repositorios (contraseña desde archivo seguro)
```

```
export RESTIC_PASSWORD_FILE="/root/.restic-password"
```

```
REPO_LOCAL="/mnt/backup-local"
```

```
REPO_REMOTE="sftp:backup@backup-server:/backups/$HOSTNAME"
```

```
REPO_CLOUD="s3:s3.amazonaws.com/mi-bucket-backup/$HOSTNAME"
```

```
# Directorios a respaldar
```

```
BACKUP_DIRS="/etc /home /var/www /var/lib/postgresql /root"
```

```
# Directorios a excluir
```

```
EXCLUDE_DIRS="
--exclude=/home/*/.cache
--exclude=/home/*/.local/share/Trash
--exclude=/var/www/*/node_modules
--exclude=/var/www/*/vendor
--exclude=*.tmp
--exclude=*.log
"
```

```
# Retención de snapshots
```

```
RETENTION="--keep-daily 7 --keep-weekly 4 --keep-monthly 12 --keep-yearly 3"
```

```
# === FUNCIONES ===
```

```
log() {
    echo "[$(date '+%Y-%m-%d %H:%M:%S')] $1" | tee -a "$LOG_FILE"
}
```

```
enviar_notificacion() {
    local estado="$1"
    local mensaje="$2"
    echo -e "Estado: $estado\nServidor: $HOSTNAME\n\n$mensaje" | \
        mail -s "[BACKUP] $estado - $HOSTNAME" "$EMAIL"
}
```

```
verificar_repositorio() {
    local repo="$1"
    local nombre="$2"

    if restic -r "$repo" snapshots &>/dev/null; then
        log "Repositorio $nombre: OK"
        return 0
    else
        log "ERROR: Repositorio $nombre no accesible"
        return 1
    fi
}
```

```
hacer_backup() {
    local repo="$1"
    local nombre="$2"

    log "Iniciando backup a $nombre..."
}
```

```

if restic -r "$repo" backup $BACKUP_DIRS $EXCLUDE_DIRS \
    --tag "$(date +%Y%m%d)" --tag "automatico"; then
    log "Backup a $nombre completado"
    return 0
else
    log "ERROR: Backup a $nombre falló"
    return 1
fi
}

limpiar_snapshots() {
    local repo="$1"
    local nombre="$2"

    log "Limpiando snapshots antiguos en $nombre..."

    if restic -r "$repo" forget $RETENTION --prune; then
        log "Limpieza de $nombre completada"
        return 0
    else
        log "ERROR: Limpieza de $nombre falló"
        return 1
    fi
}

verificar_integridad() {
    local repo="$1"
    local nombre="$2"

    log "Verificando integridad de $nombre..."

    if restic -r "$repo" check; then
        log "Integridad de $nombre: OK"
        return 0
    else
        log "ERROR: Problemas de integridad en $nombre"
        return 1
    fi
}

# === EJECUCIÓN PRINCIPAL ===

log "===== INICIO BACKUP 3-2-1 ====="

errores=0
resumen=""

```

Copia 1: Local

```
if verificar_repositorio "$REPO_LOCAL" "LOCAL"; then
    if hacer_backup "$REPO_LOCAL" "LOCAL"; then
        limpiar_snapshots "$REPO_LOCAL" "LOCAL"
        resumen+="LOCAL: OK\n"
    else
        ((errores++))
        resumen+="LOCAL: ERROR en backup\n"
    fi
else
    ((errores++))
    resumen+="LOCAL: NO DISPONIBLE\n"
fi
```

Copia 2: Remoto (SSH)

```
if verificar_repositorio "$REPO_REMOTO" "REMOTO"; then
    if hacer_backup "$REPO_REMOTO" "REMOTO"; then
        limpiar_snapshots "$REPO_REMOTO" "REMOTO"
        resumen+="REMOTO: OK\n"
    else
        ((errores++))
        resumen+="REMOTO: ERROR en backup\n"
    fi
else
    ((errores++))
    resumen+="REMOTO: NO DISPONIBLE\n"
fi
```

Copia 3: Cloud (offsite)

```
if verificar_repositorio "$REPO_CLOUD" "CLOUD"; then
    if hacer_backup "$REPO_CLOUD" "CLOUD"; then
        limpiar_snapshots "$REPO_CLOUD" "CLOUD"
        resumen+="CLOUD: OK\n"
    else
        ((errores++))
        resumen+="CLOUD: ERROR en backup\n"
    fi
else
    ((errores++))
    resumen+="CLOUD: NO DISPONIBLE\n"
fi
```

Verificación semanal de integridad (solo domingos)

```
if [ "$(date +%u)" -eq 7 ]; then
    log "Verificación semanal de integridad..."
    verificar_integridad "$REPO_LOCAL" "LOCAL"
fi
```

```
log "===== FIN BACKUP 3-2-1 ====="
```

```
# Enviar notificación
```

```
if [ $errores -eq 0 ]; then
```

```
    enviar_notificacion "EXITOSO" "Todas las copias completadas:\n$resumen"
```

```
else
```

```
    enviar_notificacion "ERROR ($errores fallos)" "Resumen:\n$resumen\nRevisar\n$LOG_FILE"
```

```
fi
```

```
exit $errores
```

```
Configuración de Permisos y Cron
```

```
# Crear archivo de contraseña seguro
```

```
echo "contraseña-muy-segura-y-larga" | sudo tee /root/.restic-password
```

```
sudo chmod 600 /root/.restic-password
```

```
# Dar permisos al script
```

```
sudo chmod 700 /usr/local/bin/backup-321.sh
```

```
# Programar backup diario a las 2:00 AM
```

```
echo "0 2 * * * root /usr/local/bin/backup-321.sh" | sudo tee /etc/cron.d/backup-321
```

```
Comandos de Gestión de Backups
```

```
# Configurar contraseña
```

```
export RESTIC_PASSWORD_FILE="/root/.restic-password"
```

```
# Listar snapshots
```

```
sudo restic -r /mnt/backup-local snapshots
```

```
# Ver contenido de un snapshot
```

```
sudo restic -r /mnt/backup-local ls latest
```

```
# Restaurar archivos específicos
```

```
sudo restic -r /mnt/backup-local restore latest --target /tmp/restore --include\n/etc/nginx"
```

```
# Restaurar snapshot completo
```

```
sudo restic -r /mnt/backup-local restore abc123 --target /
```

```
# Comparar dos snapshots
```

```
sudo restic -r /mnt/backup-local diff snapshot1 snapshot2
```

```
# Verificar integridad
```

```
sudo restic -r /mnt/backup-local check
```

```
# Ver estadísticas
```

```
sudo restic -r /mnt/backup-local stats
```

```
# Montar repositorio como sistema de archivos
```

```
sudo mkdir /mnt/restic-mount
```

```
sudo restic -r /mnt/backup-local mount /mnt/restic-mount
```

```
# Después de explorar:
```

```
sudo umount /mnt/restic-mount
```

Checklist Nivel 4: Red, Servicios y Mantenimiento Avanzado

5.1 Arquitectura de Red y DNS

#	Verificación	Comando	Estado Esperado
1.1	nftables está activo	sudo systemctl is-active nftables	active
1.2	Reenvío IP habilitado	cat /proc/sys/net/ipv4/ip_forward	1
1.3	Reglas de firewall cargadas	sudo nft list ruleset grep -c "chain"	>= 3
1.4	DMZ aislada de LAN	sudo nft list ruleset grep "DMZ.*LAN.*drop"	Regla presente
1.5	systemd-resolved activo	sudo systemctl is-active systemd-resolved	active
1.6	DNSSEC habilitado	resolvectl status grep DNSSEC	DNSSEC setting: yes
1.7	DoT habilitado	resolvectl status grep DNSOverTLS	DNSOverTLS setting: yes
1.8	Resolución DNS funciona	resolvectl query google.com	Respuesta exitosa
1.9	WireGuard instalado	dpkg -l grep wireguard	Paquete instalado
1.10	Clave privada WG con permisos	stat -c %a /etc/wireguard/*.key	600
1.11	Interfaz WG activa (si configurada)	ip link show wg0	Estado UP

5.2 SSH Hardening

#	Verificación	Comando	Estado Esperado
2.1	SSH escuchando	ss -tlnp grep :22	Puerto 22 activo
2.2	Sintaxis sshd válida	sudo sshd -t	Sin errores
2.3	Root login deshabilitado	sudo sshd -T grep permitrootlogin	no

#	Verificación	Comando	Estado Esperado
2.4	Autenticación por contraseña deshabilitada	sudo sshd -T \ grep passwordauthen tication	no
2.5	Solo claves Ed25519/RSA	ls /etc/ssh/ssh_host _*_key	Solo ed25519 y rsa
2.6	Sin claves DSA	ls /etc/ssh/ssh_host _dsa_key 2>&1	Archivo no existe
2.7	MaxAuthTries configurado	sudo sshd -T \ grep maxauthtries	<= 3
2.8	X11Forwarding deshabilitado	sudo sshd -T \ grep x11forwarding	no
2.9	Clave Ed25519 del usuario existe	ls ~/.ssh/id_ed255 19.pub	Archivo existe
2.10	FIDO2 configurado (si aplica)	ssh -Q key \ grep sk	Algoritmos sk presentes

5.3 Servicios Web (nginx y Apache)

#	Verificación	Comando	Estado Esperado
3.1	nginx activo	sudo systemctl is-active nginx	active
3.2	Sintaxis nginx válida	sudo nginx -t	syntax is ok
3.3	TLS 1.3 habilitado en nginx	nginx -T \ grep "ssl_protocols"	Incluye TLSv1.3
3.4	server_tokens deshabilitado en nginx	nginx -T \ grep "server_tokens"	off
3.5	HSTS configurado	curl -sI https://localhost \ grep -i strict	Cabecera presente
3.6	CSP configurado	curl -sI https://localhost \ grep -i content-security	Cabecera presente
3.7	X-Content-Type-Options	curl -sI https://localhost \ grep -i x-content-type	nosniff
3.8	OCSP Stapling en nginx	nginx -T \ grep ssl_stapling	on
3.9	Apache activo (si instalado)	sudo systemctl is-active apache2	active
3.10	Sintaxis Apache válida (si instalado)	sudo apache2ctl configtest	Syntax OK

5.4 Gestión de Paquetes

#	Verificación	Comando	Estado Esperado
4.1	Formato deb822 configurado	ls /etc/apt/sources.list.d/debian.sources	Archivo existe
4.2	Repositorios accesibles	sudo apt update 2>&1 \\ grep -c "Err:"	0
4.3	Claves GPG válidas	apt-key list 2>&1 \\ grep -c "expired"	0
4.4	unattended-upgrades instalado	dpkg -l \\ unattended-upgrades	Paquete instalado
4.5	Actualizaciones automáticas activas	sudo systemctl is-enabled unattended-upgrades	enabled
4.6	apt-listbugs instalado	dpkg -l \\ grep apt-listbugs	Paquete instalado
4.7	Configuración de orígenes válida	grep -c "Origins-Pattern" /etc/apt/apt.conf.d/50unattended-upgrades	>= 1
4.8	Sin actualizaciones de seguridad pendientes	sudo unattended-upgrade --dry-run 2>&1 \\ grep -c "will be upgraded"	0 (idealmente)

5.5 Monitorización y Logs

#	Verificación	Comando	Estado Esperado
5.1	journald activo	sudo systemctl is-active systemd-journald	active
5.2	Almacenamiento persistente	grep "^Storage" /etc/systemd/journald.conf	persistent
5.3	FSS (sellado) habilitado	grep "^Seal" /etc/systemd/journald.conf	yes
5.4	Integridad de logs verificable	sudo journalctl --verify 2>&1 \\ grep -c "FAIL"	0
5.5	auditd activo	sudo systemctl is-active auditd	active
5.6	Reglas de auditoría cargadas	sudo auditctl -l \\ grep -c "^_"	>= 10
5.7	Monitoreo de /etc/passwd	sudo auditctl -l \\ grep passwd	Regla presente
5.8	Monitoreo de sudo	sudo auditctl -l \\ grep sudo	Regla presente
5.9	Logs de auditoría accesibles	sudo ausearch -ts today \\ head -1	Sin errores

#	Verificación	Comando	Estado Esperado
5.10	Script de alertas configurado	ls /usr/local/bin/alertas-seguridad.sh	Archivo existe

5.6 Copias de Seguridad 3-2-1

#	Verificación	Comando	Estado Esperado
6.1	restic instalado	restic version	Versión mostrada
6.2	Archivo de contraseña seguro	stat -c %a /root/.restic-password	600
6.3	Repositorio local inicializado	sudo restic -r /mnt/backup-local snapshots 2>&1 \ grep -v "error"	Lista de snapshots
6.4	Script de backup existe	ls /usr/local/bin/backup-321.sh	Archivo existe
6.5	Script con permisos correctos	stat -c %a /usr/local/bin/backup-321.sh	700
6.6	Cron de backup configurado	cat /etc/cron.d/backup-321	Tarea configurada
6.7	Al menos 3 copias (estrategia 3-2-1)	Verificar manual	3 repositorios configurados
6.8	Último backup < 24h	sudo restic -r /mnt/backup-local snapshots --latest 1 --json \ grep time	Fecha reciente
6.9	Integridad del repositorio	sudo restic -r /mnt/backup-local check	Sin errores
6.10	Restauración funcional	Test manual de restauración	Archivos recuperables

Resumen de Comandos de Verificación Rápida

Ejecute este script para una verificación completa del Nivel 4:

```
#!/bin/bash
# /usr/local/bin/verificar-nivel4.sh
# Script de verificación rápida del Nivel 4

echo "=== VERIFICACIÓN NIVEL 4: Red, Servicios y Mantenimiento ==="
echo ""

# Colores
VERDE='\033[0;32m'
ROJO='\033[0;31m'
```

AMARILLO='\033[1;33m'

NC='\033[0m'

```

verificar() {
    local descripcion="$1"
    local comando="$2"
    local esperado="$3"

    resultado=$(eval "$comando" 2>/dev/null)

    if [[ "$resultado" == *"$esperado"* ]] || [[ "$resultado" -ge "$esperado" ]]
2>/dev/null; then
        echo -e "[${VERDE}OK${NC}] $descripcion"
        return 0
    else
        echo -e "[${ROJO}FALLO${NC}] $descripcion (obtenido: $resultado)"
        return 1
    fi
}

echo "--- Servicios de Red ---"
verificar "nftables activo" "systemctl is-active nftables" "active"
verificar "systemd-resolved activo" "systemctl is-active systemd-resolved" "active"
verificar "DNSSEC habilitado" "resolvectl status | grep -c 'DNSSEC setting: yes'"
"1"

echo ""
echo "--- SSH ---"
verificar "SSH activo" "systemctl is-active ssh" "active"
verificar "Sintaxis SSH válida" "sshd -t 2>&1 | grep -c 'error'" "0"
verificar "Root login deshabilitado" "sshd -T | grep 'permitrootlogin no'"
"permitrootlogin no"

echo ""
echo "--- Nginx ---"
verificar "nginx activo" "systemctl is-active nginx" "active"
verificar "Sintaxis nginx válida" "nginx -t 2>&1 | grep -c 'ok'" "1"

echo ""
echo "--- Gestión de Paquetes ---"
verificar "unattended-upgrades activo" "systemctl is-enabled unattended-upgrades"
"enabled"

echo ""
echo "--- Logs y Auditoría ---"
verificar "journald activo" "systemctl is-active systemd-journald" "active"
verificar "auditd activo" "systemctl is-active auditd" "active"

```

```
verificar "Reglas de auditoría" "auditctl -l | grep -c '^-' "10"
```

```
echo ""
```

```
echo "--- Backups ---"
```

```
verificar "restic instalado" "which restic | grep -c restic" "1"
```

```
echo ""
```

```
echo "=== VERIFICACIÓN COMPLETADA ==="
```

Referencias y Recursos Adicionales

- [Manual de Administración de Debian](#)
- [Guía de Seguridad de Debian](#)
- [Documentación de OpenSSH](#)
- [Wiki de WireGuard](#)
- [Documentación de nginx](#)
- [Documentación de Apache HTTP Server](#)
- [Mozilla SSL Configuration Generator](#)
- [Documentación de systemd](#)
- [Manual de auditd](#)
- [Documentación de Restic](#)

Nivel 5: Auditoría, Cumplimiento y Respuesta

Manual de Seguridad Informática para Debian 13 “Trixie”

Introducción

Este nivel aborda las prácticas avanzadas de seguridad que permiten verificar el estado de protección del sistema, responder ante incidentes y medir el progreso en madurez de seguridad. Está diseñado para principiantes, por lo que cada término técnico será explicado en su primera aparición.

6.1. Auditorías Automatizadas (Lynis, OpenSCAP)

Conceptos Fundamentales

Lynis es una herramienta de auditoría de seguridad de código abierto diseñada para sistemas Unix/Linux. Analiza el sistema en busca de vulnerabilidades, errores de configuración y oportunidades de mejora, generando un informe detallado con puntuación y recomendaciones.

OpenSCAP es una implementación de código abierto del protocolo **SCAP** (Security Content Automation Protocol - Protocolo de Automatización de Contenido de Seguridad). SCAP es un conjunto de estándares desarrollados por el NIST (Instituto Nacional de Estándares y Tecnología de EE.UU.) que permite la evaluación automatizada de vulnerabilidades y cumplimiento de políticas de seguridad.

6.1.1. Instalación y Uso de Lynis

Instalación desde repositorios oficiales

Actualizar lista de paquetes

```
sudo apt update
```

Instalar Lynis

```
sudo apt install lynis -y
```

Verificar la versión instalada

```
lynis --version
```

Instalación de la versión más reciente (opcional)

Clonar repositorio oficial

```
cd /opt
```

```
sudo git clone https://github.com/CISOfy/lynis.git
```

Ejecutar desde el directorio clonado

```
cd /opt/lynis
sudo ./lynis audit system
```

Ejecución de auditoría completa

Auditoría completa del sistema

```
sudo lynis audit system
```

Auditoría con informe en formato específico

```
sudo lynis audit system --report-file /var/log/lynis-report.dat
```

Auditoría rápida (sin interacción)

```
sudo lynis audit system --quick
```

Ver solo advertencias y sugerencias

```
sudo lynis audit system --warnings-only
```

Interpretación de resultados

Lynis genera un **índice de endurecimiento** (hardening index) de 0 a 100:

Puntuación	Interpretación
0-49	Seguridad crítica, requiere atención inmediata
50-69	Seguridad básica, mejoras necesarias
70-84	Buena seguridad, optimizaciones recomendadas
85-100	Excelente seguridad

Ver el informe generado

```
sudo cat /var/log/lynis.log
```

Ver sugerencias específicas

```
sudo grep "suggestion" /var/log/lynis-report.dat
```

6.1.2. Instalación y Uso de OpenSCAP

Instalación de OpenSCAP

Instalar OpenSCAP y herramientas asociadas

```
sudo apt update
```

```
sudo apt install libopenscap8 openscap-scanner openscap-utils -y
```

Instalar contenido SCAP para Debian

```
sudo apt install scap-security-guide -y
```

Verificar instalación

```
oscap --version
```

Listar perfiles disponibles

Ver perfiles de seguridad disponibles para el sistema

```
oscap info /usr/share/xml/scap/ssg/content/ssg-debian12-ds.xml
```

Nota: Para Debian 13, puede ser necesario usar el perfil de Debian 12

o descargar contenido actualizado

Ejecución de evaluación SCAP

Crear directorio para resultados

```
sudo mkdir -p /var/log/openscap
```

Ejecutar evaluación con perfil estándar

```
sudo oscap xccdf eval \  
  --profile xccdf_org.ssgproject.content_profile_standard \  
  --results /var/log/openscap/results.xml \  
  --report /var/log/openscap/report.html \  
  /usr/share/xml/scap/ssg/content/ssg-debian12-ds.xml
```

Abrir informe HTML (si hay entorno gráfico)

El informe se encuentra en /var/log/openscap/report.html

Remediación automática

Generar script de remediación basado en resultados

```
sudo oscap xccdf generate fix \  
  --fix-type bash \  
  --result-id "" \  
  /var/log/openscap/results.xml > /tmp/remediation.sh
```

Revisar el script antes de ejecutar

```
cat /tmp/remediation.sh
```

Ejecutar remediación (con precaución)

sudo bash /tmp/remediation.sh

6.1.3. Automatización de Auditorías

Crear tarea programada para Lynis

Crear script de auditoría automática

```
sudo tee /usr/local/bin/auditoria-seguridad.sh << 'EOF'  
#!/bin/bash  
# Script de auditoría de seguridad automatizada  
FECHA=$(date +%Y%m%d_%H%M%S)  
LOG_DIR="/var/log/auditorias"  
mkdir -p $LOG_DIR
```

Ejecutar Lynis

```
lynis audit system --no-colors --quiet \  
  --report-file "$LOG_DIR/lynis_$FECHA.dat" \  
  \
```

```
--log-file "$LOG_DIR/lynis_$FECHA.log"

# Enviar resumen por correo (opcional)
# mail -s "Auditoría de Seguridad $FECHA" admin@empresa.com <
"$LOG_DIR/lynis_$FECHA.log"

echo "Auditoría completada: $FECHA"
EOF

# Dar permisos de ejecución
sudo chmod +x /usr/local/bin/auditoria-seguridad.sh

# Programar ejecución semanal (domingos a las 3:00 AM)
echo "0 3 * * 0 root /usr/local/bin/auditoria-seguridad.sh" | sudo tee
/etc/cron.d/auditoria-seguridad
```

6.2. Simulacros de Incidentes (Ransomware, Fallo TPM)

Conceptos Fundamentales

Ransomware es un tipo de software malicioso (malware) que cifra los archivos del sistema víctima y exige un pago (rescate) para proporcionar la clave de descifrado. Es una de las amenazas más devastadoras en ciberseguridad moderna.

TPM (Trusted Platform Module - Módulo de Plataforma Confiable) es un chip de seguridad integrado en la placa base del ordenador que almacena claves criptográficas y garantiza la integridad del arranque del sistema.

6.2.1. Simulacro de Ataque Ransomware

ADVERTENCIA: Este simulacro debe realizarse ÚNICAMENTE en entornos de prueba aislados, NUNCA en sistemas de producción.

Fase 1: Preparación del entorno de prueba

Crear entorno aislado para simulacro

```
mkdir -p /tmp/simulacro_ransomware/{documentos,imagenes,datos}
```

Crear archivos de prueba

```
for i in {1..10}; do
```

```
    echo "Documento de prueba $i - Contenido confidencial" >
```

```
    "/tmp/simulacro_ransomware/documentos/doc_$i.txt"
```

```
    dd if=/dev/urandom of="/tmp/simulacro_ransomware/imagenes/img_$i.jpg"
```

```
    bs=1024 count=10 2>/dev/null
```

```
done
```

Crear archivo de datos simulado

```
echo "BASE_DE_DATOS_SIMULADA" >
```


/tmp/simulacro_ransomware/datos/database.db

Verificar estructura creada

```
tree /tmp/simulacro_ransomware/ 2>/dev/null || find /tmp/simulacro_ransomware/
-type f
```

Fase 2: Simulación del cifrado (sin daño real)

Script de simulación de ransomware (SOLO PARA PRUEBAS)

```
cat << 'EOF' > /tmp/simular_ransomware.sh
```

```
#!/bin/bash
```

```
# SIMULADOR EDUCATIVO - NO ES RANSOMWARE REAL
```

```
TARGET_DIR="/tmp/simulacro_ransomware"
```

```
LOG_FILE="/tmp/ransomware_simulation.log"
```

```
echo "[$(date)] Simulación de ransomware iniciada" > $LOG_FILE
```

```
# Simular cifrado renombrando archivos
```

```
find "$TARGET_DIR" -type f | while read archivo; do
```

```
    # Solo renombrar, no cifrar realmente
```

```
    mv "$archivo" "${archivo}.ENCRYPTED_SIMULATION"
```

```
    echo "[SIMULADO] Archivo 'cifrado': $archivo" >> $LOG_FILE
```

```
done
```

```
# Crear nota de rescate simulada
```

```
cat > "$TARGET_DIR/LEEME_RESCATE.txt" << 'NOTA'
```

```
=====
SIMULACRO DE RANSOMWARE
```

```
=====
Este es un SIMULACRO educativo.
```

```
Sus archivos han sido "cifrados" (renombrados).
```

En un ataque real:

- NO pague el rescate
- Desconecte el equipo de la red
- Contacte al equipo de seguridad
- Restaure desde copias de seguridad

Para recuperar los archivos de este simulacro:

```
find /tmp/simulacro_ransomware -name "*.ENCRYPTED_SIMULATION" -exec bash
-c 'mv "$1" "${1%.ENCRYPTED_SIMULATION}" _ {} \;
```

```
=====
NOTA
```

```
echo "[$(date)] Simulación completada" >> $LOG_FILE
```

```
EOF
```

```
chmod +x /tmp/simular_ransomware.sh
```

Fase 3: Procedimiento de respuesta ante ransomware

PASO 1: Detección y aislamiento

=====

Desconectar de la red inmediatamente

sudo ip link set eth0 down # O el nombre de su interfaz de red

PASO 2: Identificación del alcance

=====

Buscar archivos con extensiones sospechosas

find / -type f \(-name ".encrypted" -o -name "*.locked" -o -name "*.crypto" \)*
2>/dev/null

Buscar notas de rescate

*find / -type f \(-name "*README*" -o -name "*DECRYPT*" -o -name "*RESTORE*" \)*
\) -mtime -1 2>/dev/null

PASO 3: Preservar evidencia

=====

Crear imagen del sistema (requiere espacio suficiente)

sudo dd if=/dev/sda of=/media/externo/imagen_forense.img bs=4M
status=progress

PASO 4: Documentar el incidente

=====

cat << 'EOF' > /tmp/informe_incidente.txt
INFORME DE INCIDENTE DE SEGURIDAD

=====

Fecha de detección: \$(date)
Sistema afectado: \$(hostname)
Usuario que reporta: \$(whoami)

Síntomas observados:

- [] Archivos con extensión modificada*
- [] Nota de rescate encontrada*
- [] Imposibilidad de abrir archivos*
- [] Rendimiento anormal del sistema*

Acciones tomadas:

- [] Sistema aislado de la red*
- [] Evidencia preservada*
- [] Equipo de seguridad notificado*
- [] Copias de seguridad verificadas*

Próximos pasos:

- 1. Análisis forense*
- 2. Restauración desde backup*
- 3. Investigación del vector de entrada*

4. Actualización de defensas
EOF

6.2.2. Simulacro de Fallo TPM

Preparación del simulacro

Verificar si el sistema tiene TPM

```
sudo dmesg | grep -i tpm  
ls /dev/tpm*
```

Instalar herramientas TPM

```
sudo apt install tpm2-tools -y
```

Verificar estado del TPM

```
sudo tpm2_getcap properties-fixed 2>/dev/null || echo "TPM no disponible o no configurado"
```

Procedimiento de respuesta ante fallo TPM

ESCENARIO: El TPM falla y el sistema no puede arrancar

debido a que las claves de cifrado de disco están protegidas por TPM

PASO 1: Documentar el estado

=====

```
echo "=== Estado del TPM ===" > /tmp/informe_tpm.txt
```

```
sudo tpm2_getcap properties-variable >> /tmp/informe_tpm.txt 2>&1
```

PASO 2: Verificar claves de recuperación LUKS

=====

Listar dispositivos cifrados

```
lsblk -f | grep -i luks
```

Verificar slots de claves disponibles

```
# sudo cryptsetup luksDump /dev/sda3 # Ajustar dispositivo
```

PASO 3: Plan de recuperación

=====

```
cat << 'EOF' > /tmp/plan_recuperacion_tpm.md
```

Plan de Recuperación ante Fallo TPM

Requisitos previos

- Clave de recuperación LUKS almacenada de forma segura
- Medio de arranque alternativo (USB Live)

Procedimiento

Opción A: Recuperación con clave LUKS

1. Arrancar desde USB Live de Debian
2. Desbloquear partición manualmente:

`sudo cryptsetup luksOpen /dev/sdXY nombre_cifrado -key-file /ruta/clave ``` 3.
Montar el sistema de archivos 4. Reconfigurar arranque sin dependencia TPM

Opción B: Restauración completa

22. Arrancar desde medio de recuperación
23. Restaurar sistema desde copia de seguridad
24. Reconfigurar TPM si el hardware es funcional

Lecciones aprendidas

- Mantener claves de recuperación en ubicación segura y separada
- Probar procedimientos de recuperación periódicamente
- Considerar múltiples métodos de desbloqueo EOF

`cat /tmp/plan_recuperacion_tpm.md`

6.3. Forense Digital Básico

Conceptos Fundamentales

Forense digital es la disciplina que aplica técnicas de investigación y análisis para recopilar, preservar y examinar evidencia digital de sistemas informáticos, con el objetivo de determinar qué ocurrió durante un incidente de seguridad, identificar a los responsables y prevenir futuros ataques.

6.3.1. Principios del Análisis Forense

1. **Preservación de evidencia:** No modificar los datos originales
2. **Cadena de custodia:** Documentar quién manipuló la evidencia y cuándo
3. **Reproducibilidad:** Los resultados deben poder verificarse
4. **Documentación:** Registrar cada paso del análisis

6.3.2. Herramientas Forenses en Debian

```
```bash
Instalar herramientas forenses esenciales
sudo apt update
sudo apt install -y \
 sleuthkit \ # Análisis de sistemas de archivos
 autopsy \ # Interfaz gráfica para sleuthkit
 foremost \ # Recuperación de archivos
 dc3dd \ # Copia forense de discos
 volatility3 \ # Análisis de memoria RAM
 binwalk \ # Análisis de firmware y archivos
 hashdeep \ # Verificación de integridad
 exiftool # Metadatos de archivos
```

```
Verificar instalación
which fls mmls foremost dc3dd
```

#### 6.3.3. Procedimientos Forenses Básicos

*Creación de imagen forense*  
# **IMPORTANTE:** Trabajar siempre sobre copias, nunca sobre el original

*# Identificar el disco a copiar*  
lsblk

*# Crear imagen forense con dc3dd (más robusto que dd)*

*# Incluye verificación de hash automática*

```
sudo dc3dd if=/dev/sdX of=/media/forense/imagen.dd \
 hash=sha256 \
 log=/media/forense/imagen.log
```

*# Alternativa con dd tradicional*

```
sudo dd if=/dev/sdX of=/media/forense/imagen.dd bs=4M status=progress
sha256sum /media/forense/imagen.dd > /media/forense/imagen.sha256
```

### *Análisis de sistema de archivos*

*# Ver estructura de particiones de una imagen*

```
mmls /media/forense/imagen.dd
```

*# Ejemplo de salida:*

*# DOS Partition Table*

*# Offset Sector: 0*

*# Units are in 512-byte sectors*

*#*

#	Slot	Start	End	Length	Description
---	------	-------	-----	--------	-------------

#	00:	Meta	0000000000	0000000000	0000000001 Primary Table (#0)
---	-----	------	------------	------------	-------------------------------

#	01:	----	0000000000	0000002047	0000002048 Unallocated
---	-----	------	------------	------------	------------------------

#	02:	00:00	0000002048	0001953791	0001951744 Linux (0x83)
---	-----	-------	------------	------------	-------------------------

*# Listar archivos en una partición (offset 2048)*

```
fls -o 2048 /media/forense/imagen.dd
```

*# Listar archivos eliminados (marcados con \*)*

```
fls -o 2048 -d /media/forense/imagen.dd
```

*# Exportar un archivo específico por su inode*

```
icat -o 2048 /media/forense/imagen.dd 12345 > /tmp/archivo_recuperado
```

### *Recuperación de archivos eliminados*

*# Crear directorio para archivos recuperados*

```
mkdir -p /media/forense/recuperados
```

*# Recuperar archivos con foremost*

```
sudo foremost -t all -i /media/forense/imagen.dd -o /media/forense/recuperados
```

*# Recuperar tipos específicos (documentos, imágenes)*

```
sudo foremost -t doc,pdf,jpg,png -i /media/forense/imagen.dd -o
/media/forense/recuperados_docs
```

*# Ver resultados*

```
cat /media/forense/recuperados/audit.txt
```

### *Análisis de línea temporal*

*# Crear línea temporal de actividad del sistema*

*# (útil para reconstruir eventos)*

*# Generar bodyfile con información de tiempos*

`fls -o 2048 -r -m "/" /media/forense/imagen.dd > /tmp/bodyfile.txt`

*# Convertir a línea temporal ordenada*

`mactime -b /tmp/bodyfile.txt -d > /tmp/timeline.csv`

*# Filtrar por rango de fechas sospechosas*

`mactime -b /tmp/bodyfile.txt -d 2024-01-01..2024-01-31 > /tmp/timeline_enero.csv`

### *Análisis de logs del sistema*

*# Los logs son fundamentales en el análisis forense*

*# Ubicaciones comunes en Debian:*

*# Logs de autenticación*

`sudo cat /var/log/auth.log | grep -E "(Failed|Accepted|session opened)"`

*# Logs del sistema*

`sudo journalctl --since "2024-01-01" --until "2024-01-31"`

*# Buscar actividad sospechosa*

`sudo grep -r "sudo" /var/log/auth.log | tail -50`

`sudo grep -r "COMMAND" /var/log/auth.log | tail -50`

*# Últimos inicios de sesión*

`last -50`

`lastb -50 # Intentos fallidos`

*# Buscar modificaciones recientes en archivos de configuración*

`find /etc -type f -mtime -7 -ls`

### *Verificación de integridad*

*# Crear base de datos de hashes de archivos críticos*

`sudo hashdeep -r -c sha256 /etc /usr/bin /usr/sbin > /var/log/baseline_hashes.txt`

*# Verificar integridad posteriormente*

`sudo hashdeep -r -a -k /var/log/baseline_hashes.txt /etc /usr/bin /usr/sbin`

*# Verificar binarios del sistema contra paquetes*

`sudo debsums -c # Mostrar archivos modificados`

## 6.4. IDS/IPS (Suricata/Fail2ban)

### Conceptos Fundamentales

**IDS (Intrusion Detection System - Sistema de Detección de Intrusiones)** es un sistema que monitorea el tráfico de red o la actividad del sistema en busca de comportamientos maliciosos o violaciones de políticas, generando alertas cuando detecta amenazas.

**IPS (Intrusion Prevention System - Sistema de Prevención de Intrusiones)** es similar al IDS pero además de detectar, puede tomar acciones automáticas para bloquear o prevenir las amenazas detectadas.

**Suricata** es un motor de detección de amenazas de red de alto rendimiento, código abierto, capaz de funcionar como IDS, IPS y monitor de seguridad de red. Analiza el tráfico de red en tiempo real usando reglas predefinidas.

**Fail2ban** es una herramienta que analiza logs del sistema y bloquea automáticamente direcciones IP que muestran comportamiento malicioso (como múltiples intentos fallidos de autenticación).

### 6.4.1. Instalación y Configuración de Suricata

#### Instalación de Suricata

*# Instalar Suricata*

```
sudo apt update
```

```
sudo apt install suricata suricata-update -y
```

*# Verificar versión*

```
suricata --build-info | head -20
```

*# Ver interfaz de red principal*

```
ip addr show
```

*# Nota: Identificar la interfaz (ej: eth0, ens33, enp0s3)*

#### Configuración básica

*# Respalda configuración original*

```
sudo cp /etc/suricata/suricata.yaml /etc/suricata/suricata.yaml.backup
```

*# Editar configuración principal*

```
sudo nano /etc/suricata/suricata.yaml
```

Configuración recomendada en /etc/suricata/suricata.yaml:

*# Sección vars - Definir red local*

```
vars:
```

```
 address-groups:
```

```
 HOME_NET: "[192.168.0.0/16,10.0.0.0/8,172.16.0.0/12]"
```

```
 EXTERNAL_NET: "!$HOME_NET"
```



```
port-groups:
 HTTP_PORTS: "80,443,8080"
 SSH_PORTS: "22"
```

#### *# Sección af-packet - Interfaz de red*

```
af-packet:
 - interface: eth0 # Cambiar por su interfaz
 cluster-id: 99
 cluster-type: cluster_flow
 defrag: yes
```

#### *# Sección outputs - Logs*

```
outputs:
 - eve-log:
 enabled: yes
 filetype: regular
 filename: eve.json
 types:
 - alert:
 payload: yes
 payload-printable: yes
 - http:
 extended: yes
 - dns
 - tls
 - files
 - ssh
```

#### *# Sección rule-files - Reglas activas*

```
rule-files:
 - suricata.rules
```

#### *# Directorio de reglas*

```
default-rule-path: /var/lib/suricata/rules
```

#### *Actualización de reglas*

##### *# Actualizar reglas de detección*

```
sudo suricata-update
```

##### *# Habilitar fuentes de reglas adicionales*

```
sudo suricata-update list-sources
sudo suricata-update enable-source et/open
sudo suricata-update enable-source oisf/trafficid
```

##### *# Actualizar con nuevas fuentes*

```
sudo suricata-update
```

```
Verificar reglas descargadas
ls -la /var/lib/suricata/rules/
```

### *Iniciar Suricata*

```
Probar configuración
sudo suricata -T -c /etc/suricata/suricata.yaml
```

### *# Iniciar servicio*

```
sudo systemctl enable suricata
sudo systemctl start suricata
```

### *# Verificar estado*

```
sudo systemctl status suricata
```

### *# Ver logs en tiempo real*

```
sudo tail -f /var/log/suricata/eve.json | jq '.'
```

### *Monitoreo y alertas*

#### *# Ver alertas de Suricata*

```
sudo tail -f /var/log/suricata/fast.log
```

#### *# Filtrar alertas específicas en eve.json*

```
sudo cat /var/log/suricata/eve.json | jq 'select(.event_type=="alert")'
```

#### *# Estadísticas de Suricata*

```
sudo cat /var/log/suricata/stats.log | tail -50
```

#### *# Script de monitoreo de alertas*

```
cat << 'EOF' > /usr/local/bin/suricata-alertas.sh
#!/bin/bash
Mostrar últimas alertas de Suricata
echo "=== Últimas 20 alertas de Suricata ==="
sudo tail -100 /var/log/suricata/eve.json | \
 jq -r 'select(.event_type=="alert") | \
 "\(.timestamp) | \(.alert.severity) | \(.src_ip):\(.src_port) -> \(.dest_ip):\(.dest_port) \
 | \(.alert.signature)'" | \
 tail -20
EOF
sudo chmod +x /usr/local/bin/suricata-alertas.sh
```

## **6.4.2. Instalación y Configuración de Fail2ban**

### *Instalación de Fail2ban*

#### *# Instalar Fail2ban*

```
sudo apt update
sudo apt install fail2ban -y
```

```
Verificar instalación
fail2ban-client --version
```

### Configuración básica

```
Crear archivo de configuración local (no modificar jail.conf)
sudo cp /etc/fail2ban/jail.conf /etc/fail2ban/jail.local
```

```
Editar configuración
sudo nano /etc/fail2ban/jail.local
```

Configuración recomendada en /etc/fail2ban/jail.local:

#### [DEFAULT]

```
Tiempo de baneo (10 minutos)
bantime = 10m
```

```
Tiempo de ventana para contar intentos (10 minutos)
findtime = 10m
```

```
Número de intentos antes de banear
maxretry = 5
```

```
Acción por defecto: banear IP y enviar correo
action = %(action_mwl)s
```

```
Ignorar IPs locales
ignoreip = 127.0.0.1/8 ::1 192.168.1.0/24
```

```
Backend para monitoreo de logs
backend = systemd
```

```
Email para notificaciones (opcional)
destemail = admin@empresa.com
sender = fail2ban@servidor.com
```

```
=====
JAILS - Servicios a proteger
=====
```

#### [sshd]

```
enabled = true
port = ssh
filter = sshd
logpath = /var/log/auth.log
maxretry = 3
bantime = 1h
```

#### [sshd-ddos]

```
enabled = true
port = ssh
filter = sshd-ddos
logpath = /var/log/auth.log
maxretry = 10
bantime = 48h
```

#### **[apache-auth]**

```
enabled = true
port = http,https
filter = apache-auth
logpath = /var/log/apache2/*error.log
maxretry = 5
```

#### **[apache-badbots]**

```
enabled = true
port = http,https
filter = apache-badbots
logpath = /var/log/apache2/*access.log
maxretry = 2
bantime = 48h
```

#### **[httpd-auth]**

```
enabled = true
port = http,https
filter = httpd-auth
logpath = /var/log/apache2/error.log
maxretry = 5
```

#### **[nginx-http-auth]**

```
enabled = true
port = http,https
filter = nginx-http-auth
logpath = /var/log/nginx/error.log
maxretry = 5
```

#### **[postfix]**

```
enabled = true
port = smtp,465,submission
filter = postfix
logpath = /var/log/mail.log
maxretry = 5
```

#### **[dovecot]**

```
enabled = true
port = pop3,pop3s,imap,imaps
filter = dovecot
```

```
logpath = /var/log/mail.log
maxretry = 5
```

### *Crear filtros personalizados*

*# Crear filtro personalizado para detectar escaneos de servidores web*

```
sudo tee /etc/fail2ban/filter.d/scan-detector.conf << 'EOF'
```

```
[Definition]
```

```
failregex = ^<HOST>
```

```
-.*(GET|POST|HEAD).*/wp-admin|/phpmyadmin|/admin|/.env|/config).*.*$
```

```
ignoreregex =
```

```
EOF
```

*# Añadir jail para el nuevo filtro*

```
sudo tee -a /etc/fail2ban/jail.local << 'EOF'
```

```
[scan-detector]
```

```
enabled = true
```

```
port = http,https
```

```
filter = scan-detector
```

```
logpath = /var/log/nginx/access.log
```

```
 /var/log/apache2/access.log
```

```
maxretry = 3
```

```
bantime = 24h
```

```
EOF
```

### *Iniciar y gestionar Fail2ban*

*# Iniciar servicio*

```
sudo systemctl enable fail2ban
```

```
sudo systemctl start fail2ban
```

*# Verificar estado*

```
sudo systemctl status fail2ban
```

*# Ver estado de todos los jails*

```
sudo fail2ban-client status
```

*# Ver estado de un jail específico*

```
sudo fail2ban-client status sshd
```

*# Ver IPs baneadas actualmente*

```
sudo fail2ban-client status sshd | grep "Banned IP"
```

*# Desbanear una IP manualmente*

```
sudo fail2ban-client set sshd unbanip 192.168.1.100
```

*# Banear una IP manualmente*

```
sudo fail2ban-client set sshd banip 192.168.1.100
```

```
Recargar configuración
sudo fail2ban-client reload
```

### Monitoreo de Fail2ban

```
Ver log de Fail2ban
sudo tail -f /var/log/fail2ban.log
```

```
Script de resumen de actividad
cat << 'EOF' > /usr/local/bin/fail2ban-resumen.sh
#!/bin/bash
echo "=== Resumen de Fail2ban ==="
echo ""
echo "Estado general:"
sudo fail2ban-client status
echo ""
echo "=== IPs actualmente baneadas ==="
for jail in $(sudo fail2ban-client status | grep "Jail list" | sed 's/.*:;//s/,/ /g'); do
 echo "--- $jail ---"
 sudo fail2ban-client status $jail | grep -E "(Currently banned|Banned IP)"
done
echo ""
echo "=== Últimos 10 baneos ==="
sudo grep "Ban" /var/log/fail2ban.log | tail -10
EOF
sudo chmod +x /usr/local/bin/fail2ban-resumen.sh
```

### 6.4.3. Integración Suricata + Fail2ban

```
Crear filtro para alertas de Suricata
sudo tee /etc/fail2ban/filter.d/suricata.conf << 'EOF'
[Definition]
failregex = .*[.*\] <HOST>:\d+ -> .*
ignoreregex =
EOF
```

```
Añadir jail para Suricata
sudo tee -a /etc/fail2ban/jail.local << 'EOF'
```

```
[suricata]
enabled = true
filter = suricata
logpath = /var/log/suricata/fast.log
maxretry = 3
bantime = 1h
action = iptables-allports[name=suricata]
EOF
```

```
Recargar Fail2ban
sudo fail2ban-client reload
```

## 6.5. Métricas de Madurez

### Conceptos Fundamentales

**Métricas de madurez** son indicadores cuantitativos y cualitativos que permiten evaluar el nivel de desarrollo y efectividad de un programa de seguridad informática. Ayudan a identificar fortalezas, debilidades y áreas de mejora, permitiendo tomar decisiones basadas en datos objetivos.

#### 6.5.1. Modelo de Madurez de Seguridad

Nivel	Nombre	Descripción
1	Inicial	Procesos ad-hoc, reactivos, sin documentación
2	Repetible	Procesos básicos documentados, se repiten con éxito
3	Definido	Procesos estandarizados, documentados y comunicados
4	Gestionado	Procesos medidos y controlados cuantitativamente
5	Optimizado	Mejora continua basada en métricas y retroalimentación

#### 6.5.2. KPIs de Seguridad Esenciales

##### *Métricas de Prevención*

*# Script para calcular métricas de prevención*

```
cat << 'EOF' > /usr/local/bin/metricas-prevencion.sh
```

```
#!/bin/bash
```

```
echo "=== MÉTRICAS DE PREVENCIÓN ==="
```

```
echo "Fecha: $(date)"
```

```
echo ""
```

```
Porcentaje de sistemas actualizados
```

```
TOTAL_PAQUETES=$(dpkg -l | grep "^ii" | wc -l)
```

```
ACTUALIZABLES=$(apt list --upgradable 2>/dev/null | grep -v "Listing" | wc -l)
```

```
PORCENTAJE_ACTUALIZADO=$((100 - (ACTUALIZABLES * 100 /
TOTAL_PAQUETES)))
```

```
echo "1. Actualización del sistema: ${PORCENTAJE_ACTUALIZADO}%"
```

```
echo " - Paquetes instalados: $TOTAL_PAQUETES"
```

```
echo " - Paquetes pendientes de actualizar: $ACTUALIZABLES"
```

```
Puntuación Lynis (si está disponible)
```

```
if [-f /var/log/lynis.log]; then
```

```
 LYNIS_SCORE=$(grep "Hardening index" /var/log/lynis.log | tail -1 | grep -oP
'\d+')
```

```

 echo "2. Índice de endurecimiento (Lynis): ${LYNIS_SCORE:-N/A}/100"
fi

Contraseñas que expiran pronto
EXPIRAN_30=$(for user in $(cut -d: -f1 /etc/passwd); do chage -l $user 2>/dev/null
| grep "Password expires" | grep -v "never"; done | wc -l)
echo "3. Usuarios con contraseña por expirar: $EXPIRAN_30"

Servicios expuestos
SERVICIOS_ESCUCHA=$(ss -tlnp | grep LISTEN | wc -l)
echo "4. Servicios escuchando conexiones: $SERVICIOS_ESCUCHA"
EOF
chmod +x /usr/local/bin/metricas-prevencion.sh

```

### *Métricas de Detección*

```

Script para calcular métricas de detección
cat << 'EOF' > /usr/local/bin/metricas-deteccion.sh
#!/bin/bash
echo "=== MÉTRICAS DE DETECCIÓN ==="
echo "Fecha: $(date)"
echo "Período: Últimas 24 horas"
echo ""

```

```

Alertas de Suricata
if [-f /var/log/suricata/eve.json]; then
 ALERTAS_24H=$(cat /var/log/suricata/eve.json | \
 jq -r 'select(.event_type=="alert") | .timestamp' | \
 grep "$(date -d 'yesterday' +%Y-%m-%d)" | wc -l)
 echo "1. Alertas Suricata (24h): $ALERTAS_24H"

 # Por severidad
 echo " Por severidad:"
 for sev in 1 2 3; do
 COUNT=$(cat /var/log/suricata/eve.json | \
 jq -r "select(.event_type=='alert' and .alert.severity==$sev)
| .timestamp" | wc -l)
 echo " - Severidad $sev: $COUNT"
 done
fi

```

```

Baneos de Fail2ban
if [-f /var/log/fail2ban.log]; then
 BANEOS_24H=$(grep "Ban" /var/log/fail2ban.log | \
 grep "$(date -d 'yesterday' +%Y-%m-%d)" | wc -l)
 echo "2. IPs baneadas (24h): $BANEOS_24H"
fi

```

```

Intentos de login fallidos

```



```
LOGINS_FALLIDOS=$(grep "Failed password" /var/log/auth.log 2>/dev/null | \
 grep "$(date -d 'yesterday' +%b %d)" | wc -l)
echo "3. Intentos de login fallidos (24h): $LOGINS_FALLIDOS"

Conexiones SSH únicas
if [-f /var/log/auth.log]; then
 SSH_UNICAS=$(grep "Accepted" /var/log/auth.log | awk '{print $11}' | sort -u |
wc -l)
 echo "4. IPs únicas con acceso SSH: $SSH_UNICAS"
fi
EOF
chmod +x /usr/local/bin/metricas-deteccion.sh
```

### *Métricas de Respuesta*

*# Script para calcular métricas de respuesta*

```
cat << 'EOF' > /usr/local/bin/metricas-respuesta.sh
```

```
#!/bin/bash
```

```
echo "=== MÉTRICAS DE RESPUESTA ==="
```

```
echo "Fecha: $(date)"
```

```
echo ""
```

```
MTTR - Tiempo medio de resolución (simulado con logs de fail2ban)
```

```
echo "1. Tiempo medio de baneo automático:"
```

```
if [-f /var/log/fail2ban.log]; then
```

```
 # Tiempo promedio entre detección y baneo
```

```
 echo " - Fail2ban responde en menos de 1 segundo"
```

```
fi
```

```
Cobertura de backup
```

```
echo "2. Cobertura de copias de seguridad:"
```

```
BACKUPS_RECIENTES=$(find /var/backups -type f -mtime -7 2>/dev/null | wc -l)
```

```
echo " - Backups en últimos 7 días: $BACKUPS_RECIENTES"
```

```
Tiempo desde última auditoría
```

```
if [-f /var/log/lynis.log]; then
```

```
 ULTIMA_AUDITORIA=$(stat -c %y /var/log/lynis.log | cut -d' ' -f1)
```

```
 echo "3. Última auditoría de seguridad: $ULTIMA_AUDITORIA"
```

```
fi
```

```
Documentación de incidentes
```

```
INCIDENTES_DOC=$(find /var/log/incidentes -type f 2>/dev/null | wc -l)
```

```
echo "4. Incidentes documentados: $INCIDENTES_DOC"
```

```
EOF
```

```
chmod +x /usr/local/bin/metricas-respuesta.sh
```

### **6.5.3. Dashboard de Métricas**

*# Script completo de dashboard de seguridad*

```
cat << 'EOF' > /usr/local/bin/dashboard-seguridad.sh
```

```
#!/bin/bash
clear
echo "
echo " DASHBOARD DE SEGURIDAD - $(hostname) "
echo " $(date '+%Y-%m-%d %H:%M:%S') "
echo "
echo ""

Estado del sistema
echo "
echo " ESTADO DEL SISTEMA "
echo "
UPTIME=$(uptime -p)
CARGA=$(uptime | awk -F'load average:' '{print $2}')
echo " Tiempo activo: $UPTIME"
echo " Carga del sistema: $CARGA"
echo "
echo ""

Servicios de seguridad
echo "
echo " SERVICIOS DE SEGURIDAD "
echo "

Suricata
SURICATA_STATUS=$(systemctl is-active suricata 2>/dev/null || echo "no
instalado")
echo -n " | Suricata: "
["$SURICATA_STATUS" == "active"] && echo "✓ ACTIVO" || echo "✗
$SURICATA_STATUS"

Fail2ban
FAIL2BAN_STATUS=$(systemctl is-active fail2ban 2>/dev/null || echo "no
instalado")
echo -n " | Fail2ban: "
["$FAIL2BAN_STATUS" == "active"] && echo "✓ ACTIVO" || echo "✗
$FAIL2BAN_STATUS"

Firewall
UFW_STATUS=$(ufw status 2>/dev/null | grep "Status:" | awk '{print $2}')
echo -n " | Firewall (UFW): "
["$UFW_STATUS" == "active"] && echo "✓ ACTIVO" || echo "✗ INACTIVO"

echo "
echo ""

Métricas rápidas
echo "

```

```

echo " | MÉTRICAS ÚLTIMAS 24 HORAS | "
echo " | "

Intentos SSH fallidos
SSH_FALLIDOS=$(grep "Failed password" /var/log/auth.log 2>/dev/null | wc -l)
echo " | Intentos SSH fallidos: $SSH_FALLIDOS"

IPs baneadas
IPS_BANEADAS=$(fail2ban-client status 2>/dev/null | grep "Currently banned" |
awk '{sum+=$NF} END {print sum}')
echo " | IPs actualmente baneadas: ${IPS_BANEADAS:-0}"

Paquetes por actualizar
ACTUALIZABLES=$(apt list --upgradable 2>/dev/null | grep -v "Listing" | wc -l)
echo " | Actualizaciones pendientes: $ACTUALIZABLES"

echo " | "
echo ""

Resumen de madurez
echo " | "
echo " | NIVEL DE MADUREZ ESTIMADO | "
echo " | "

Calcular puntuación simple
PUNTOS=0
["$SURICATA_STATUS" == "active"] && PUNTOS=$((PUNTOS + 20))
["$FAIL2BAN_STATUS" == "active"] && PUNTOS=$((PUNTOS + 20))
["$UFW_STATUS" == "active"] && PUNTOS=$((PUNTOS + 20))
[$ACTUALIZABLES -lt 10] && PUNTOS=$((PUNTOS + 20))
[-f /var/log/lynis.log] && PUNTOS=$((PUNTOS + 20))

echo " | Puntuación de seguridad: $PUNTOS/100"
if [$PUNTOS -ge 80]; then
 echo " | Nivel: OPTIMIZADO (5/5)"
elif [$PUNTOS -ge 60]; then
 echo " | Nivel: GESTIONADO (4/5)"
elif [$PUNTOS -ge 40]; then
 echo " | Nivel: DEFINIDO (3/5)"
elif [$PUNTOS -ge 20]; then
 echo " | Nivel: REPETIBLE (2/5)"
else
 echo " | Nivel: INICIAL (1/5)"
fi
echo " | "
EOF
chmod +x /usr/local/bin/dashboard-seguridad.sh

```

### 6.5.4. Automatización de Informes

*# Crear informe semanal automatizado*

```
cat << 'EOF' > /usr/local/bin/informe-semanal-seguridad.sh
```

```
#!/bin/bash
```

```
FECHA=$(date +%Y-%m-%d)
```

```
INFORME="/var/log/informes/informe_seguridad_$FECHA.md"
```

```
mkdir -p /var/log/informes
```

```
cat > $INFORME << HEADER
```

```
Informe Semanal de Seguridad
```

```
Fecha: $FECHA
```

```
Sistema: $(hostname)
```

```
Generado automáticamente
```

```

```

```
Resumen Ejecutivo
```

```
HEADER
```

```
Añadir métricas
```

```
echo "### Métricas de Prevención" >> $INFORME
```

```
/usr/local/bin/metricas-prevencion.sh >> $INFORME 2>/dev/null
```

```
echo -e "\n### Métricas de Detección" >> $INFORME
```

```
/usr/local/bin/metricas-deteccion.sh >> $INFORME 2>/dev/null
```

```
echo -e "\n### Métricas de Respuesta" >> $INFORME
```

```
/usr/local/bin/metricas-respuesta.sh >> $INFORME 2>/dev/null
```

```
echo -e "\n--\n*Informe generado automáticamente*" >> $INFORME
```

```
echo "Informe guardado en: $INFORME"
```

```
EOF
```

```
chmod +x /usr/local/bin/informe-semanal-seguridad.sh
```

*# Programar ejecución semanal*

```
echo "0 8 * * 1 root /usr/local/bin/informe-semanal-seguridad.sh" | sudo tee
/etc/cron.d/informe-seguridad
```

```
,
```

## Checklist Nivel 5: Auditoría, Cumplimiento y Respuesta

### Auditorías Automatizadas

9. ☐ **Lynis instalado y configurado**

*# Verificar: debe mostrar versión*  
`lynis --version`

10. ☐ **Auditoría Lynis ejecutada con puntuación  $\geq 70$**

*# Verificar: revisar índice de endurecimiento*  
`grep "Hardening index" /var/log/lynis.log | tail -1`

11. ☐ **OpenSCAP instalado y funcional**

*# Verificar: debe mostrar versión*  
`oscap --version`

12. ☐ **Evaluación SCAP ejecutada**

*# Verificar: debe existir el informe*  
`ls -la /var/log/openscap/report.html`

13. ☐ **Auditorías programadas (cron)**

*# Verificar: debe existir la tarea*  
`cat /etc/cron.d/auditoria-seguridad`

### Simulacros de Incidentes

- ☐ **Entorno de simulacro preparado**

*# Verificar: documentación de simulacros existe*  
`ls /tmp/plan_recuperacion_tpm.md`

- ☐ **Procedimiento de ransomware documentado**

*# Verificar: plantilla de informe existe*  
`cat /tmp/informe_incidente.txt`

- ☐ **Plan de recuperación TPM definido**

*# Verificar: herramientas TPM instaladas (si aplica)*  
`which tpm2_getcap`

- ☐ **Simulacro ejecutado en los últimos 90 días**

*# Verificar: logs de simulacros*  
`find /var/log -name "*simulacro*" -mtime -90`

### Forense Digital

- ☐ **Herramientas forenses instaladas**

*# Verificar: todas las herramientas disponibles*  
`which fls mmls foremost dc3dd hashdeep`

- ☐ **Baseline de hashes creado**

*# Verificar: archivo de hashes existe*  
`ls -la /var/log/baseline_hashes.txt`

- ☐ **Procedimiento de preservación de evidencia documentado**

*# Verificar: capacidad de crear imágenes forenses*  
`which dc3dd && echo "OK: dc3dd disponible"`

- ☐ **Verificación de integridad automatizada**

*# Verificar: debsums funcional*  
`debsums --version`

## IDS/IPS

- ☐ **Suricata instalado y activo**

*# Verificar: servicio corriendo*  
`systemctl is-active suricata`

- ☐ **Reglas de Suricata actualizadas**

*# Verificar: reglas recientes (< 7 días)*  
`find /var/lib/suricata/rules -name "*.rules" -mtime -7 | head -1`

- ☐ **Fail2ban instalado y activo**

*# Verificar: servicio corriendo*  
`systemctl is-active fail2ban`

- ☐ **Jail SSH habilitado en Fail2ban**

*# Verificar: jail sshd activo*  
`fail2ban-client status sshd`

- ☐ **Al menos 3 jails configurados**

*# Verificar: número de jails*  
`fail2ban-client status | grep "Number of jail"`

- ☐ **Logs de IDS monitoreados**

*# Verificar: eve.json tiene datos recientes*  
`tail -1 /var/log/suricata/eve.json | jq '.timestamp'`

## Métricas de Madurez

- ☐ **Scripts de métricas instalados**

```
Verificar: scripts existen
ls /usr/local/bin/metricas-*.sh
```

- ☐ **Dashboard de seguridad funcional**

```
Verificar: script de dashboard existe
ls /usr/local/bin/dashboard-seguridad.sh
```

- ☐ **Informe semanal automatizado**

```
Verificar: cron configurado
cat /etc/cron.d/informe-seguridad
```

- ☐ **Nivel de madurez >= 3 (Definido)**

```
Verificar: ejecutar dashboard y revisar nivel
/usr/local/bin/dashboard-seguridad.sh | grep "Nivel:"
```

## Verificación Global del Nivel 5

```
#!/bin/bash
Script de verificación completa del Nivel 5
echo "=== Verificación Nivel 5: Auditoría, Cumplimiento y Respuesta ==="
TOTAL=0
PASADOS=0

verificar() {
 TOTAL=$((TOTAL + 1))
 if eval "$1" > /dev/null 2>&1; then
 echo "[✓] $2"
 PASADOS=$((PASADOS + 1))
 else
 echo "[✗] $2"
 fi
}

echo -e "\n--- Auditorías ---"
verificar "which lynis" "Lynis instalado"
verificar "which oscap" "OpenSCAP instalado"
verificar "test -f /etc/cron.d/auditoria-seguridad" "Auditorías programadas"

echo -e "\n--- IDS/IPS ---"
verificar "systemctl is-active suricata" "Suricata activo"
verificar "systemctl is-active fail2ban" "Fail2ban activo"
verificar "fail2ban-client status sshd" "Jail SSH configurado"

echo -e "\n--- Forense ---"
verificar "which fls" "Sleuthkit instalado"
verificar "which foremost" "Foremost instalado"
verificar "which hashdeep" "Hashdeep instalado"
```

```

echo -e "\n--- Métricas ---"
verificar "test -x /usr/local/bin/dashboard-seguridad.sh" "Dashboard configurado"
verificar "test -f /etc/cron.d/informe-seguridad" "Informes automatizados"

echo -e "\n=== RESULTADO: $PASADOS/$TOTAL verificaciones pasadas ==="
PORCENTAJE=$((PASADOS * 100 / TOTAL))
echo "Compleitud del Nivel 5: $PORCENTAJE%"

if [$PORCENTAJE -ge 80]; then
 echo "Estado: NIVEL 5 COMPLETADO"
elif [$PORCENTAJE -ge 50]; then
 echo "Estado: NIVEL 5 EN PROGRESO"
else
 echo "Estado: NIVEL 5 REQUIERE ATENCIÓN"
fi

```

## Glosario de Términos

Término	Definición
<b>Lynis</b>	Herramienta de auditoría de seguridad para sistemas Unix/Linux
<b>OpenSCAP</b>	Implementación del protocolo SCAP para evaluación de seguridad
<b>SCAP</b>	Security Content Automation Protocol - estándar de automatización de seguridad
<b>Ransomware</b>	Malware que cifra archivos y exige rescate
<b>TPM</b>	Chip de seguridad para almacenamiento de claves criptográficas
<b>Forense digital</b>	Investigación y análisis de evidencia digital
<b>IDS</b>	Sistema de Detección de Intrusiones
<b>IPS</b>	Sistema de Prevención de Intrusiones
<b>Suricata</b>	Motor IDS/IPS de código abierto
<b>Fail2ban</b>	Herramienta de bloqueo automático de IPs maliciosas
<b>Métricas de madurez</b>	Indicadores del nivel de desarrollo del programa de seguridad
<b>MTTR</b>	Mean Time To Repair - Tiempo medio de reparación
<b>KPI</b>	Key Performance Indicator - Indicador clave de rendimiento



## **Referencias**

- [Documentación oficial de Lynis](#)
- [OpenSCAP User Manual](#)
- [Suricata Documentation](#)
- [Fail2ban Manual](#)
- [NIST Cybersecurity Framework](#)
- [Debian Security Manual](#)